Universidad Champagnat

Facultad de Informática y Diseño

Licenciatura en Sistemas de Información

TESINA DE LICENCIATURA EN SISTEMAS DE INFORMACIÓN

Sistema de traducción de Lengua de Señas Argentina a texto en tiempo real mediante inteligencia artificial

Alumnos: Leandro Provenzano y Jonatan Videla

Año: 2026

Profesor Coordinador: Ing. Mario Alberto Mirallas

Profesor Tutor: Marcelo Palma

Profesor/a de Metodología: **PENDIENTE: nombre del profesor/a de Metodología**

Agradecimientos

PENDIENTE: este apartado es personal y conviene que lo escriban ustedes. Lo que sigue es un borrador de partida para editar libremente.

Este trabajo no habría sido posible sin el acompañamiento de muchas personas. En primer lugar, agradecemos a los docentes de la Licenciatura en Sistemas de Información de la Universidad Champagnat, que a lo largo de la carrera nos dieron las herramientas para encarar un proyecto de esta naturaleza, y en particular al Ing. Mario Alberto Mirallas, coordinador del Seminario de Licenciatura, y a Marcelo Palma, tutor de esta tesina, por la orientación y la paciencia durante todo el proceso.

Agradecemos también al grupo de investigación del Instituto de Investigación en Informática LIDI de la Universidad Nacional de La Plata, cuyo trabajo en la construcción y publicación abierta del conjunto de datos LSA64 hizo posible entrenar y evaluar el modelo de reconocimiento sin depender de una costosa campaña de grabación propia. Publicar datos de investigación de manera abierta es un aporte silencioso que sostiene el trabajo de muchos otros, y este es uno de esos casos.

A nuestras familias, por la comprensión y el apoyo constante en los meses en que la tesina ocupó buena parte de nuestro tiempo, y a nuestros compañeros de cursado, con quienes compartimos dudas, avances y frustraciones.

Por último, a la comunidad sorda argentina, cuya lucha por el reconocimiento de su lengua motivó este proyecto. Esperamos que este trabajo sea un pequeño aporte en la dirección de una comunicación más accesible para todos.

Resumen

Las personas sordas que se comunican en Lengua de Señas Argentina (LSA) enfrentan barreras cotidianas cuando interactúan con personas oyentes que no conocen esa lengua, y la disponibilidad de intérpretes humanos es limitada. Esta tesina presenta el diseño, la implementación y la evaluación de un sistema que reconoce señas aisladas de la LSA a partir del video de una cámara web convencional y las traduce a texto en tiempo real, accesible desde un navegador.

El sistema se organiza en tres capas. Un cliente web captura el video de la cámara, lo envía al servidor mediante un canal **WebSocket** y presenta la traducción junto con un esqueleto de la mano superpuesto a la imagen. Un backend desarrollado en Python con FastAPI extrae, para cada cuadro, los 21 puntos de referencia de cada mano con **MediaPipe** Hands y construye un vector de 138 características normalizado respecto de la posición y el tamaño de la mano. Una red neuronal recurrente bidireccional (**BiLSTM**), entrenada con PyTorch sobre el conjunto de datos público **LSA64** (3200 videos, 64 señas, 10 personas), clasifica la secuencia de vectores. Un módulo de código compartido garantiza que el preprocesamiento sea idéntico durante el entrenamiento y la inferencia.

El modelo se evaluó con validación **leave-one-subject-out**, es decir, midiendo siempre el desempeño sobre una persona que el modelo nunca vio durante el entrenamiento. Bajo ese protocolo se obtuvo una exactitud media de 79,8 % ± 8,9 % sobre las 64 clases, con una variación entre personas de 63,1 % a 89,5 %. El análisis de errores mostró que la principal fuente de variación es la proporción de cuadros en que MediaPipe no detecta las manos, y que las confusiones se concentran en señas que comparten configuración manual y difieren en la ubicación. Se documentan además los defectos metodológicos detectados y corregidos durante el desarrollo —etiquetado incorrecto, fuga de datos entre entrenamiento y prueba, orientación inconsistente de la imagen y entrenamiento no reproducible—, que invalidaban las mediciones iniciales.

Palabras clave: Lengua de Señas Argentina, reconocimiento de lengua de señas, aprendizaje profundo, LSTM bidireccional, MediaPipe, LSA64, aplicación web en tiempo real, accesibilidad.

Abstract

Deaf people who communicate in Argentine Sign Language (LSA) face daily barriers when interacting with hearing people who do not know the language. This thesis presents the design, implementation and evaluation of a web-based system that recognizes isolated LSA signs from a standard webcam and translates them into text in real time. A browser client streams frames over a WebSocket to a FastAPI backend, which extracts hand landmarks with MediaPipe Hands, builds a 138-dimensional normalized feature vector per frame, and classifies the sequence with a bidirectional LSTM trained on the public LSA64 dataset. Evaluated with leave-one-subject-out cross-validation, the model reaches a mean accuracy of 79.8 % ± 8.9 % over 64 signs on unseen signers. Error analysis shows that performance is strongly associated with hand-detection rate and that confusions concentrate on signs sharing handshape but differing in location.

Keywords: Argentine Sign Language, sign language recognition, deep learning, BiLSTM, MediaPipe, LSA64, real-time web application.

Introducción

Contexto

Según la Organización Mundial de la Salud (2021), más de 1500 millones de personas en el mundo viven con algún grado de pérdida auditiva, y alrededor de 430 millones presentan una pérdida discapacitante. Para una parte de ellas, la lengua de señas es su primera lengua y su principal medio de comunicación. La Federación Mundial de Sordos (s.f.) estima que más de 70 millones de personas sordas utilizan alguna de las más de 300 lenguas de señas que existen en el mundo. En la Argentina, la comunidad sorda se comunica en **Lengua de Señas Argentina** (LSA), reconocida por la Ley 27.710 (2023) como una lengua natural y originaria, con una estructura gramatical completa y distinta de la del castellano.

A pesar de ese reconocimiento, la mayor parte de las tecnologías de comunicación fueron diseñadas para la lengua hablada o escrita. En la atención de un comercio, en un trámite administrativo, en un consultorio o en una guardia, la comunicación entre una persona sorda y una persona oyente que no conoce la LSA depende de la presencia de un intérprete, de la escritura o de gestos improvisados. Los intérpretes son escasos y no siempre están disponibles, y la escritura no resuelve el problema para quienes tienen el castellano como segunda lengua.

En la última década, los avances en visión por computadora y aprendizaje profundo hicieron posible reconocer automáticamente gestos y señas a partir de video. Existen sistemas de reconocimiento con alta precisión para lenguas como la Lengua de Señas Americana o la Lengua de Señas Alemana (Koller, 2020), pero los trabajos sobre LSA son pocos y, en su mayoría, se limitan a experimentos de laboratorio sin una interfaz utilizable por el público (ver el Capítulo 5).

Planteamiento del problema

Estado actual. La comunicación entre personas sordas y oyentes que no conocen la LSA es difícil en la vida cotidiana. No existen herramientas accesibles, gratuitas y adaptadas a la LSA que permitan una traducción automática, aunque sea parcial, sin equipamiento especial.

Estado deseado. Disponer de una aplicación accesible desde cualquier navegador, que utilice únicamente la cámara del dispositivo, capaz de reconocer un vocabulario acotado de señas de la LSA y mostrar su traducción al castellano escrito en tiempo real, como apoyo a la comunicación en situaciones cotidianas.

Objetivos

Objetivo general

Desarrollar una aplicación web basada en inteligencia artificial que permita la traducción en tiempo real de señas aisladas de la Lengua de Señas Argentina a texto.

Objetivos específicos

1. Diseñar la arquitectura del sistema y de la red neuronal de reconocimiento.
2. Entrenar una red neuronal sobre un conjunto de datos de LSA, con un preprocesamiento idéntico en el entrenamiento y en la inferencia.
3. Evaluar el modelo mediante un protocolo independiente del sujeto (que mida el desempeño sobre personas no vistas durante el entrenamiento) y analizar sus errores.
4. Desarrollar la aplicación web que capture video en tiempo real, se comunique con el modelo y presente la traducción de forma accesible.
5. Desplegar el sistema en una plataforma en la nube dentro de los límites de una capa gratuita.

El tercer objetivo no figuraba en el anteproyecto y se incorporó durante el desarrollo, al comprobar que las mediciones de exactitud obtenidas con un único reparto de los datos variaban varios puntos según qué personas quedaban en el conjunto de prueba (ver la sección 14.1). Sin una evaluación rigurosa, ninguna cifra de desempeño del modelo podía sostenerse.

Alcances y limitaciones

Alcances

- Aplicación web con traducción en tiempo real desde la cámara del dispositivo, utilizable desde computadoras y dispositivos móviles con navegador moderno.
- Reconocimiento de las 64 señas aisladas del conjunto LSA64 (Ronchetti et al., 2016b), que incluyen colores, alimentos, verbos y sustantivos de uso cotidiano (ver el Anexo A).
- Infraestructura para el reconocimiento de configuraciones manuales estáticas (como las letras del alfabeto manual), con un pipeline de captura y entrenamiento propio. **PENDIENTE: definir el alcance final del abecedario — ver docs/tesina/PLAN.md, decisión D1.**
- Despliegue del sistema como un único servicio en contenedor.

Limitaciones

- El sistema reconoce **señas aisladas**, no lengua de señas continua. No interpreta estructuras gramaticales, el uso del espacio ni los componentes no manuales (expresión facial, movimiento corporal), que en la LSA forman parte del significado.
- El vocabulario está restringido a las señas presentes en los datos de entrenamiento.

- Los resultados cuantitativos se midieron sobre LSA64, grabado en condiciones controladas (fondo uniforme, iluminación constante, vestimenta oscura). El desempeño con cámaras web domésticas y en entornos reales puede ser menor.
- El reconocimiento requiere buena iluminación y que la persona esté de frente a la cámara, con las manos dentro del encuadre.
- El despliegue debe mantenerse dentro de los límites de la capa gratuita de servicios en la nube, lo que restringe la cantidad de usuarios simultáneos.

Justificación

El proyecto responde a una necesidad concreta de inclusión. Una herramienta de traducción automática, aun con un vocabulario limitado, puede facilitar interacciones breves y frecuentes —saludos, consultas, indicaciones— y reducir la dependencia de terceros en situaciones donde no hay intérprete disponible. No pretende reemplazar a los intérpretes humanos, cuya tarea abarca dimensiones lingüísticas y culturales que ningún sistema actual reproduce, sino complementar su labor.

Desde el punto de vista disciplinar, el trabajo integra áreas centrales de la Licenciatura en Sistemas de Información: análisis y diseño de sistemas, desarrollo de aplicaciones web, arquitectura cliente-servidor, aprendizaje automático, pruebas de software y despliegue en la nube. Además, el proyecto aporta un antecedente local poco frecuente: la evaluación independiente del sujeto de un modelo para LSA integrado en una aplicación funcional, con el código y los resultados documentados de forma reproducible.

Metodología de trabajo

El desarrollo se organizó con una adaptación de Scrum para un equipo de dos personas, con iteraciones de dos semanas y un tablero Kanban (ver el Capítulo 7). Las tareas se dividieron en dos frentes de trabajo paralelos: el modelo de reconocimiento (datos, entrenamiento y evaluación) y la aplicación (backend, cliente web, pruebas y despliegue), con integraciones frecuentes a través de ramas de Git.

Estructura del documento

El documento se divide en dos partes. La **Parte I** presenta el marco teórico: la comunidad sorda y la LSA (Capítulo 1), los fundamentos del aprendizaje automático y de su evaluación (Capítulo 2), las redes neuronales y en particular las recurrentes (Capítulo 3), la visión por computadora y la estimación de la pose de la mano (Capítulo 4), el estado del arte en reconocimiento de lengua de señas (Capítulo 5), las tecnologías para aplicaciones web en tiempo real (Capítulo 6) y la metodología de desarrollo (Capítulo 7).

La **Parte II** describe la implementación: el análisis y el diseño del sistema (Capítulo 8), la preparación de los datos y la extracción de características (Capítulo 9), el modelo y su entrenamiento (Capítulo 10), el backend de inferencia (Capítulo 11), el cliente web (Capítulo 12), el aseguramiento de la calidad y el despliegue (Capítulo 13) y los resultados de la evaluación (Capítulo 14). Cierran el trabajo las conclusiones, los anexos, la bibliografía y un glosario de términos técnicos, que aparecen en **negrita** en el lugar del texto donde se definen.

Índice general

Agradecimientos 3

Resumen 4

Abstract 5

Introducción 6

Contexto 6

Planteamiento del problema 6

Objetivos 7

Alcances y limitaciones 7

Justificación 8

Metodología de trabajo 8

Estructura del documento 8

Índice general 10

Índice de figuras 16

Índice de tablas 18

Marco teórico 19

Capítulo 1 — La comunidad sorda y la Lengua de Señas Argentina 20

1.1 Discapacidad auditiva y comunidad sorda 20

1.2 Las lenguas de señas como lenguas naturales 21

1.3 La Lengua de Señas Argentina 21

1.4 Barreras de comunicación y herramientas existentes 23

1.5 Consideraciones éticas 23

Capítulo 2 — Inteligencia artificial y aprendizaje automático 24

2.1 Inteligencia artificial 24

2.2 Aprendizaje automático 24

2.3 Generalización y sobreajuste....25
2.4 Partición de los datos....26
2.5 Métricas de evaluación....27
2.6 Bosques aleatorios....28
Capítulo 3 — Redes neuronales y aprendizaje profundo....29
3.1 La neurona artificial....29
3.2 Redes multicapa y aprendizaje profundo....29
3.3 Entrenamiento....30
3.4 Regularización....31
3.5 Redes neuronales recurrentes....31
3.6 Long Short-Term Memory....32
3.7 LSTM bidireccional....33
3.8 Otras arquitecturas....33
Capítulo 4 — Visión por computadora y estimación de la pose de la mano....34
4.1 Imagen digital y video....34
4.2 Enfoques para el reconocimiento de señas....34
4.3 MediaPipe....35
4.4 Lateralidad y orientación de la imagen....37
4.5 Limitaciones de la detección....37
Capítulo 5 — Reconocimiento automático de lengua de señas: estado del arte....39
5.1 Tareas del área....39
5.2 Conjuntos de datos....39
5.3 Antecedentes sobre LSA64....40
5.4 Aplicaciones existentes....42
5.5 Brecha que aborda el trabajo....42
Capítulo 6 — Aplicaciones web en tiempo real....44
6.1 Arquitectura cliente-servidor....44
6.2 HTTP y el estilo REST....44

6.3 WebSocket 45
6.4 Acceso a la cámara desde el navegador 45
6.5 El backend en Python 46
6.6 Contenedores y despliegue en la nube 46
6.7 Accesibilidad web 47
Capítulo 7 — Metodología de desarrollo 48
7.1 Enfoque ágil 48
7.2 División del trabajo 49
7.3 Control de versiones 49
7.4 Pruebas automatizadas 49
7.5 Rigor experimental 50
7.6 Herramientas 50
7.7 Cronograma 51
Desarrollo de la implementación 52
Capítulo 8 — Análisis y diseño del sistema 53
8.1 Requisitos funcionales 53
8.2 Requisitos no funcionales 53
8.3 Actores y casos de uso 54
8.4 Señas estáticas y dinámicas 54
8.5 Arquitectura del sistema 55
8.6 Modelo de clases del backend 56
8.7 Dinámica de una traducción 57
8.8 Decisiones de diseño 58
Capítulo 9 — Preparación de los datos y extracción de características 60
9.1 El conjunto LSA64 60
9.2 Muestreo temporal 61
9.3 El vector de características 61
9.4 Posiciones fijas por mano 61

9.5 Normalización 62
9.6 Variante con ubicación (v2) 62
9.7 Convención de orientación de la imagen 63
9.8 Datos propios para configuraciones estáticas 64
9.9 Extracción en dos etapas 64
9.10 Calidad de la detección 65
9.11 Errores detectados en el pipeline de datos 66
Capítulo 10 — Modelo de reconocimiento y entrenamiento 67
10.1 Arquitectura de la red 67
10.2 Reparto de los datos 68
10.3 Procedimiento de entrenamiento 69
10.4 Reproducibilidad 69
10.5 Regularización y aumentación de datos 70
10.6 Validación leave-one-subject-out 70
10.7 Artefactos del entrenamiento 71
Capítulo 11 — Backend de inferencia 72
11.1 Organización del código 72
11.2 Arranque y servicios expuestos 72
11.3 El canal WebSocket y la concurrencia 73
11.4 La sesión de traducción 74
11.5 La ventana temporal 75
11.6 Modelos intercambiables 75
11.7 Validación de compatibilidad 76
Capítulo 12 — Cliente web 77
12.1 Estructura 77
12.2 Captura y envío de cuadros 78
12.3 Presentación de los resultados 79
12.4 Esqueleto de las manos 79

12.5 Métricas de desempeño....80
12.6 Accesibilidad....80
12.7 Diseño adaptable....81
Capítulo 13 — Aseguramiento de la calidad y despliegue....82
13.1 Pruebas automatizadas....82
13.2 Seguridad y privacidad....84
13.3 Contenedor....84
13.4 Despliegue....85
Capítulo 14 — Resultados y evaluación....86
14.1 Por qué un único reparto no alcanza....86
14.2 Resultado principal....86
14.3 Curva de entrenamiento y sobreajuste....88
14.4 Análisis por clase....88
14.5 Detección de manos y desempeño....91
14.6 Experimentos de mejora....91
14.7 Comparación con los antecedentes....94
14.8 Desempeño en tiempo real....94
14.9 Discusión....95
14.10 Síntesis....96
Conclusiones....97
Conclusiones generales....97
Inconvenientes resueltos y camino recorrido....97
Contribuciones y aportes....98
Futuro del proyecto....99
Apéndices del trabajo....100
Anexo A — Vocabulario de señas....101
Anexo B — Detalle de la red neuronal....104
B.1 Capas y parámetros....104

B.2 Implementación 104
B.3 Contenido del archivo del modelo 105
Anexo C — Guía de instalación y ejecución 106
C.1 Requisitos 106
C.2 Descarga del modelo de MediaPipe 106
C.3 Ejecución del backend y del cliente 106
C.4 Ejecución con Docker 106
C.5 Reentrenamiento del modelo 107
C.6 Pruebas automatizadas 107
C.7 Regeneración de las figuras y de este documento 107
Anexo D — Cronograma 108
Bibliografía 110
Glosario 114

(Si este índice aparece vacío, abrir el documento en Word y presionar F9 sobre él, o correr build/postproceso.ps1.)

Índice de figuras

Figura 1**.** Distribución de los videos en un esquema dependiente del sujeto (izquierda), donde los videos de prueba de cada persona se mezclan con los de entrenamiento, y en un fold de LOSO (derecha), donde una persona completa queda fuera del entrenamiento. 27
Figura 2**.** Esquema de una celda LSTM. Las compuertas de olvido, entrada y salida regulan qué información se conserva, se escribe y se expone en cada paso de la secuencia.....32
Figura 3**.** Los 21 puntos de referencia de la mano que estima MediaPipe Hands. Las líneas indican las conexiones del esqueleto que el sistema dibuja sobre el video. 36
Figura 4**.** Arquitectura del sistema. El cliente envía cuadros JPEG por WebSocket; el backend extrae las manos, clasifica la secuencia y devuelve el resultado. El código de extracción de características y la definición de la red se comparten entre el backend y el pipeline de entrenamiento. 55
Figura 5**.** Diagrama de secuencia de una sesión de traducción. 58
Figura 6**.** Estructura del vector de características (versión v1). Cada mano ocupa un bloque fijo de 69 valores: un indicador de presencia, las 63 coordenadas normalizadas de sus 21 puntos y los 5 ángulos de flexión de los dedos. 61
Figura 7**.** Pipeline de datos en dos etapas. La extracción con MediaPipe se ejecuta una vez y se guarda en un caché; el entrenamiento y la evaluación leen el caché tantas veces como sea necesario. 64
Figura 8**.** Arquitectura de la red BiLSTM. Cada cuadro de la secuencia alimenta dos LSTM que la recorren en sentidos opuestos; sus estados finales se concatenan y una capa lineal produce una puntuación por seña. 68
Figura 9**.** Documentación de la API generada automáticamente por FastAPI.....73
Figura 10**.** Interfaz del cliente web en su tema claro, antes de iniciar la cámara. 78
Figura 11**.** Interfaz del cliente web en su tema oscuro. 81
Figura 12**.** Exactitud por persona en la validación leave-one-subject-out. La línea indica la media y la banda, un desvío. En rojo, las personas por debajo de la media menos un desvío.....87
Figura 13**.** Pérdida de entrenamiento y exactitud de validación por época del modelo servido. La línea punteada marca la mejor época, cuyos pesos se conservan.....88

Figura 14**.** Exactitud por clase en las predicciones agrupadas del LOSO, ordenada de menor a mayor. En rojo, las clases por debajo de 60 %; en naranja, entre 60 % y 80 %. La línea punteada indica la exactitud global. 89
Figura 15**.** Matriz de confusión de las predicciones agrupadas del LOSO, normalizada por fila. Los números de los ejes corresponden a la numeración de LSA64 (ver el Anexo A). 90
Figura 16**.** Relación entre la retención de cuadros con manos detectadas y la exactitud LOSO por persona. 91
Figura 17**.** Exactitud por persona con los cuadros sin manos filtrados (representación oficial) y con la grilla temporal completa 92
Figura 18**.** Exactitud media sobre las personas S03 y S08 para la configuración de referencia y tres variantes. 93
Figura 19**.** Diagrama de Gantt del proyecto 108

Índice de tablas

Tabla 1**.** Parámetros formales de una seña y su tratamiento en el sistema. 22
Tabla 2**.** Conjuntos de datos de lengua de señas relevantes para el trabajo. 40
Tabla 3**.** Resultados publicados sobre LSA64 según el protocolo de evaluación. 41
Tabla 4**.** Alternativas de comunicación para enviar video al servidor. 45
Tabla 5**.** Herramientas utilizadas en el desarrollo. 50
Tabla 6**.** Requisitos funcionales del sistema. 53
Tabla 7**.** Requisitos no funcionales del sistema. 53
Tabla 8**.** Casos de uso principales. 54
Tabla 9**.** Clases del backend y sus responsabilidades. 56
Tabla 10**.** Detección de manos en el servidor frente a detección en el cliente. 58
Tabla 11**.** Hiperparámetros de la configuración de referencia. 69
Tabla 12**.** Decisiones de accesibilidad del cliente web. 80
Tabla 13**.** Distribución de las pruebas automatizadas. 82
Tabla 14**.** Resultados por partición de la validación leave-one-subject-out. 86
Tabla 15**.** Pares de señas más confundidos en el LOSO. 90
Tabla 16**.** Comparación entre filtrar los cuadros sin manos y conservar la grilla temporal completa. 92
Tabla 17**.** Vocabulario de LSA64 con glosas, cantidad de manos, secuencias conservadas y exactitud LOSO. 101
Tabla 18**.** Capas de la red BiLSTM, dimensiones y cantidad de parámetros. 104
Tabla 19**.** Campos del archivo del modelo entrenado. 105
Tabla 20**.** Etapas del proyecto y su estado. 108
Tabla 21**.** Glosario de términos técnicos. 114

PARTE I

Marco teórico

Capítulo 1 — La comunidad sorda y la Lengua de Señas Argentina

Antes de abordar los aspectos técnicos, conviene precisar qué se intenta reconocer. Un sistema de traducción de lengua de señas no reconoce "gestos" en general, sino unidades de una lengua con su propia fonología, gramática y comunidad de hablantes. Las decisiones de diseño de los capítulos siguientes —qué información extraer de la imagen, qué tipo de modelo usar y cómo evaluarlo— dependen de entender cómo está estructurada una seña.

1.1 Discapacidad auditiva y comunidad sorda

La pérdida auditiva abarca un espectro amplio, desde dificultades leves hasta la sordera profunda, y puede ser congénita o adquirida. La Organización Mundial de la Salud (2021) estima que más de 1500 millones de personas viven con algún grado de pérdida auditiva, de las cuales unos 430 millones presentan una pérdida discapacitante.

Desde el modelo social de la discapacidad, adoptado por la Convención sobre los Derechos de las Personas con Discapacidad (Naciones Unidas, 2006) e incorporado al derecho argentino por la Ley 26.378 (2008), la discapacidad no reside en la persona sino en la interacción entre sus características y las barreras del entorno. Para una persona sorda, la barrera no es la sordera en sí, sino un entorno que presupone que todos oyen y hablan.

Dentro de ese colectivo, la **comunidad sorda** está formada por las personas que comparten la lengua de señas como lengua principal, junto con una identidad y una cultura propias. Esta distinción es relevante para el proyecto: los destinatarios de la herramienta no son todas las personas con pérdida auditiva —muchas de las cuales se comunican oralmente o con audífonos e implantes— sino quienes se comunican en LSA.

1.1.1 Terminología

El término "sordomudo", todavía frecuente en el lenguaje cotidiano y presente en la versión inicial del anteproyecto de esta tesina, es considerado inadecuado por la propia comunidad

sorda[1]. La forma aceptada es **persona sorda**, y en este documento se utiliza exclusivamente esa expresión. En el mismo sentido, se habla de "lengua" de señas y no de "lenguaje", porque se trata de lenguas naturales completas y no de un sistema de comunicación artificial.

1.2 Las lenguas de señas como lenguas naturales

Durante mucho tiempo se consideró que las lenguas de señas eran pantomima o una versión gestual de la lengua oral del entorno. El trabajo de Stokoe (1960) sobre la Lengua de Señas Americana fue el primero en demostrar que las señas no son unidades indivisibles, sino que se componen de un conjunto finito de elementos que se combinan de manera sistemática, del mismo modo en que las palabras de una lengua oral se componen de fonemas. Desde entonces, la lingüística reconoce a las lenguas de señas como lenguas naturales, con todos los niveles de análisis de cualquier lengua: fonología, morfología, sintaxis y pragmática.

Dos propiedades tienen consecuencias directas para el reconocimiento automático:

- **No son universales.** Cada comunidad desarrolló su propia lengua de señas, y estas no se corresponden con las lenguas orales del entorno: la Lengua de Señas Americana y la Lengua de Señas Británica son lenguas distintas, aunque en ambos países se hable inglés. Un modelo entrenado para una lengua de señas no sirve para otra.
- **Son lenguas visoespaciales.** La información se transmite simultáneamente por varios canales: las dos manos, su ubicación respecto del cuerpo, el movimiento, la orientación, la expresión facial y la postura. Mientras una lengua oral es esencialmente secuencial, una seña combina varios elementos en el mismo instante.

1.3 La Lengua de Señas Argentina

La LSA es la lengua de la comunidad sorda argentina. Massone y Machado (1994) realizaron la primera descripción lingüística sistemática de la LSA junto con un vocabulario bilingüe, y establecieron que su estructura es independiente de la del castellano. La Ley 27.710 (2023) reconoció a la LSA como "lengua natural y originaria" que conforma el patrimonio lingüístico y cultural de las personas sordas en todo el territorio nacional, y facultó al Estado a promover su acceso y uso en los distintos ámbitos.

1.3.1 Parámetros formales de una seña

Siguiendo el análisis iniciado por Stokoe (1960) y aplicado a la LSA por Massone y Machado (1994), una seña puede describirse a partir de un conjunto de parámetros formales. La

[1]La mayoría de las personas sordas tienen el aparato fonador intacto y pueden vocalizar; lo que no adquieren de forma natural es la lengua oral, por no acceder a ella auditivamente. "Mudo" describe incorrectamente esa situación y arrastra una connotación de incapacidad que la comunidad rechaza.

Tabla 1 los resume e indica cuáles de ellos son observables por el sistema desarrollado en esta tesina, que trabaja sobre los puntos de referencia de las manos (ver el Capítulo 9).

Tabla *1***.** *Parámetros formales de una seña y su tratamiento en el sistema.*

Parámetro	**Descripción**	**¿Lo captura el sistema?**
Configuración	Forma que adopta la mano: dedos extendidos, flexionados, en contacto	Sí: coordenadas normalizadas de los 21 puntos y ángulos de flexión de cada dedo
Ubicación	Lugar del espacio o del cuerpo donde se realiza la seña	Parcialmente: la versión v1 de las características la descarta al centrar en la muñeca; la versión experimental v2 la incorpora (sección 9.6)
Movimiento	Desplazamiento o cambio de la mano durante la seña	Sí, de forma implícita: el modelo recibe la secuencia completa de cuadros
Orientación	Hacia dónde apunta la palma	Sí, de forma indirecta, a través de la coordenada de profundidad y la geometría de los puntos
Rasgos no manuales	Expresión facial, movimiento de la cabeza y del torso	No: el sistema sólo observa las manos

La segunda fila de la tabla anticipa uno de los hallazgos del Capítulo 14: buena parte de los errores del modelo se concentran en pares de señas que comparten configuración y difieren en la ubicación, precisamente el parámetro que la representación principal del sistema no conserva.

1.3.2 Señas de una y dos manos

En la LSA, como en otras lenguas de señas, existen señas realizadas con una sola mano —la **mano dominante**— y señas bimanuales, en las que ambas manos se mueven de forma simétrica o una actúa como base de la otra. En el conjunto LSA64, 42 de las 64 señas se realizan con una mano y 22 con las dos (Ronchetti et al., 2016b). Un sistema que sólo considere "la primera mano detectada" en cada cuadro no puede distinguir correctamente las señas bimanuales; por eso el diseño de las características reserva una posición fija para cada mano (sección 9.4).

1.3.3 El alfabeto manual

Además del léxico de señas, la LSA cuenta con un **alfabeto manual** o dactilológico, que asigna una configuración de la mano a cada letra del alfabeto castellano. Se usa para deletrear nombres propios, palabras sin seña establecida o términos técnicos. Desde el punto de vista del reconocimiento, la mayoría de las letras son configuraciones estáticas —basta con un cuadro para identificarlas—, aunque algunas incorporan movimiento. Esta diferencia justifica tratar por separado el reconocimiento de letras (clasificación de un vector) y el de señas léxicas (clasificación de una secuencia), como se discute en el Capítulo 8.

1.4 Barreras de comunicación y herramientas existentes

En la práctica, la comunicación entre una persona sorda y una oyente que no sabe LSA se resuelve con alguna de las siguientes alternativas, ninguna de ellas plenamente satisfactoria:

- **Intérpretes de LSA.** Es la solución de mayor calidad, pero los intérpretes son pocos, se concentran en las grandes ciudades y su presencia debe coordinarse con anticipación. No están disponibles para interacciones espontáneas y breves.
- **Videointerpretación remota.** Algunos organismos ofrecen servicios de interpretación a distancia por videollamada. Reducen el problema de la presencia física, pero siguen dependiendo de la disponibilidad de un intérprete.
- **Escritura.** Escribir en papel o en el teléfono es la alternativa más común, pero el castellano escrito es para muchas personas sordas una segunda lengua, aprendida en condiciones desfavorables, y la comunicación resulta lenta y parcial.
- **Aplicaciones de aprendizaje y diccionarios.** Existen diccionarios y aplicaciones para aprender señas, útiles para las personas oyentes interesadas, pero no traducen en tiempo real.

La herramienta propuesta en esta tesina se ubica en un espacio distinto: un apoyo automático, inmediato y sin costo para interacciones acotadas, que no requiere coordinar con terceros ni equipamiento especial. Su principal limitación —el vocabulario reducido y el reconocimiento de señas aisladas— está explicitada desde los alcances del proyecto.

1.5 Consideraciones éticas

Trabajar con una lengua minoritaria y con imágenes de personas implica responsabilidades que se tuvieron en cuenta durante el desarrollo:

- **Representación honesta de las capacidades.** Un sistema que "traduce" sólo 64 señas aisladas no traduce la LSA. Presentarlo como un traductor general generaría expectativas falsas y podría perjudicar a quienes confíen en él en una situación importante. Por eso la interfaz muestra la confianza de cada predicción y el documento declara explícitamente las limitaciones.
- **Datos biométricos.** El video de una persona y la geometría de sus manos son datos personales. El sistema no almacena el video recibido: cada cuadro se procesa en memoria y se descarta. Si se capturan datos propios para entrenar (sección 9.8), es necesario el consentimiento informado de los participantes.
- **Participación de la comunidad.** La evaluación de una herramienta destinada a la comunidad sorda debe incluir a personas sordas. **PENDIENTE: describir la prueba con usuarios de LSA si se realiza — ver docs/tesina/PLAN.md, tarea F4.**

Capítulo 2 — Inteligencia artificial y aprendizaje automático

Este capítulo presenta los conceptos de aprendizaje automático sobre los que se apoya el sistema, con énfasis en un aspecto que resultó central durante el desarrollo: cómo medir correctamente el desempeño de un modelo. Los detalles de las redes neuronales se tratan en el Capítulo 3.

2.1 Inteligencia artificial

Russell y Norvig (2021) organizan las definiciones de **inteligencia artificial** (IA) según dos ejes: si se interesan por el pensamiento o por el comportamiento, y si toman como referencia el desempeño humano o un ideal de racionalidad. La perspectiva que adopta este trabajo es la del comportamiento: un sistema es útil si, dada una entrada —el video de una persona haciendo una seña—, produce la salida correcta —la palabra que esa seña representa—, sin que importe si el mecanismo interno se parece al humano.

Durante décadas, la IA se construyó principalmente con reglas escritas a mano por especialistas. Ese enfoque funciona cuando el conocimiento del dominio puede expresarse de forma explícita, pero fracasa en tareas perceptivas como reconocer una seña: nadie puede escribir las reglas que distinguen la seña "Leche" de la seña "Agua" a partir de los píxeles de una imagen, aunque cualquier persona que sepa LSA las distinga sin esfuerzo. El aprendizaje automático resuelve ese problema invirtiendo el enfoque.

2.2 Aprendizaje automático

El **aprendizaje automático** (*machine learning*) es la rama de la IA que estudia algoritmos capaces de mejorar su desempeño en una tarea a partir de la experiencia, es decir, de los datos (Géron, 2022). En lugar de programar las reglas, se programa un procedimiento que las infiere a partir de ejemplos.

Según el tipo de experiencia disponible, se distinguen tres grandes familias:

- **Aprendizaje supervisado.** Cada ejemplo de entrenamiento está acompañado de la respuesta correcta, llamada **etiqueta**. El algoritmo aprende una función que asigna

entradas a etiquetas. Es el caso de este trabajo: cada video de LSA64 viene etiquetado con la seña que representa.

- **Aprendizaje no supervisado.** Los ejemplos no tienen etiqueta y el objetivo es descubrir estructura en los datos, como grupos de ejemplos similares.
- **Aprendizaje por refuerzo.** Un agente aprende a actuar en un entorno a partir de recompensas y castigos.

2.2.1 Clasificación supervisada

Dentro del aprendizaje supervisado, la **clasificación** es la tarea de asignar cada entrada a una de un conjunto finito de clases. Formalmente, a partir de un conjunto de entrenamiento de pares (x, y), donde x es la representación de la entrada e y su clase, se busca una función f tal que f(x) sea igual a y para entradas nuevas, no vistas durante el entrenamiento.

La representación x se construye a partir de **características** (*features*): valores numéricos que describen la entrada. Elegir buenas características es una de las decisiones más importantes de cualquier sistema de aprendizaje automático. En este trabajo, cada cuadro de video se representa con 138 valores derivados de la geometría de las manos (ver el Capítulo 9), y cada seña es una secuencia de esos vectores.

Los clasificadores modernos no suelen devolver sólo una clase, sino una distribución de probabilidad sobre todas las clases. La clase con mayor probabilidad es la predicción, y esa probabilidad puede interpretarse como la **confianza** del modelo. El sistema aprovecha ese valor para descartar predicciones dudosas (sección 11.4).

2.3 Generalización y sobreajuste

El objetivo del aprendizaje no es memorizar el conjunto de entrenamiento, sino **generalizar**: funcionar bien con datos nuevos. Un modelo puede fallar en ese objetivo de dos maneras opuestas (Goodfellow et al., 2016):

- **Subajuste** (*underfitting*): el modelo es demasiado simple para capturar la relación entre entradas y clases, y comete muchos errores incluso sobre los datos de entrenamiento.
- **Sobreajuste** (*overfitting*): el modelo es tan flexible que aprende también las particularidades irrelevantes del conjunto de entrenamiento —el ruido, o los rasgos de las personas que lo grabaron— y su desempeño cae con datos nuevos.

El síntoma típico del sobreajuste es una brecha creciente entre el error de entrenamiento, que sigue bajando, y el error sobre datos no vistos, que se estanca o sube. Ese patrón se observó en el modelo de esta tesina y se analiza en la sección 14.3. Las técnicas para combatirlo se denominan **regularización** y se describen en la sección 3.4.

2.4 Partición de los datos

Para estimar cómo se comportará el modelo con datos nuevos, es imprescindible reservar una parte de los datos que el modelo nunca vea durante el entrenamiento. La práctica habitual es dividir los datos en tres conjuntos (Géron, 2022):

- **Entrenamiento**: los ejemplos con los que se ajustan los parámetros del modelo.
- **Validación**: ejemplos que se usan durante el desarrollo para tomar decisiones —elegir hiperparámetros, decidir cuándo detener el entrenamiento— sin contaminar la medición final.
- **Prueba** (*test*): ejemplos que se usan una única vez, al final, para medir el desempeño. Si el conjunto de prueba influye en cualquier decisión, deja de ser una estimación honesta.

2.4.1 Validación cruzada

Cuando los datos son pocos, un único reparto produce una estimación inestable: el resultado depende de qué ejemplos cayeron, por azar, en el conjunto de prueba. La **validación cruzada de k particiones** (*k-fold cross-validation*) divide los datos en k grupos, entrena k modelos —cada uno dejando afuera un grupo distinto para la prueba— y promedia los resultados (Kohavi, 1995). Además de la media, el desvío entre particiones informa cuánto varía el desempeño según los datos de prueba.

2.4.2 Fuga de datos y dependencia entre ejemplos

La validación cruzada supone que los ejemplos son independientes entre sí. Esa suposición no se cumple cuando hay grupos de ejemplos fuertemente relacionados. En un conjunto de datos de lengua de señas, las cinco repeticiones de una seña hechas por la misma persona son muy parecidas entre sí: comparten la forma de las manos, la estatura, el ritmo y el estilo personal de quien seña.

Si esas repeticiones se reparten al azar entre entrenamiento y prueba, el modelo puede acertar en la prueba simplemente porque ya vio a esa misma persona haciendo esa misma seña. A esto se lo denomina **fuga de datos** (*data leakage*): información del conjunto de prueba que se filtra al entrenamiento y produce una estimación optimista que no se sostiene en la práctica (Hastie et al., 2009). La solución es particionar **por grupo**: todos los ejemplos de una misma persona deben quedar del mismo lado del reparto.

2.4.3 Evaluación dependiente e independiente del sujeto

En el reconocimiento de lengua de señas, esta distinción tiene nombre propio:

- **Evaluación dependiente del sujeto** (*signer-dependent*): las personas que aparecen en la prueba también aparecen en el entrenamiento. Mide qué tan bien reconoce el modelo a personas que ya conoce.
- **Evaluación independiente del sujeto** (*signer-independent*): las personas de la prueba nunca aparecen en el entrenamiento. Mide qué tan bien generaliza a personas nuevas, que es exactamente la situación de uso de una aplicación pública.

La variante más exigente de la evaluación independiente es la validación **leave-one-subject-out** (LOSO): con N personas, se entrenan N modelos, cada uno dejando afuera a una persona distinta para la prueba. Cada ejemplo del conjunto de datos es predicho exactamente una vez por un modelo que nunca vio a quien lo realizó. La Figura 1 ilustra la diferencia entre ambos esquemas.

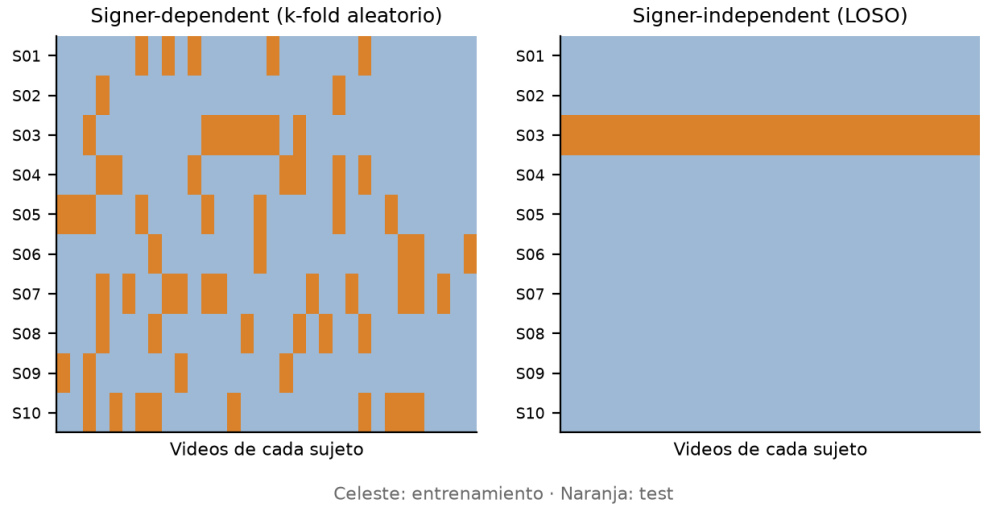


Figura *1. Distribución de los videos en un esquema dependiente del sujeto (izquierda), donde los videos de prueba de cada persona se mezclan con los de entrenamiento, y en un fold de LOSO (derecha), donde una persona completa queda fuera del entrenamiento.*

Como se verá en el Capítulo 5, las cifras de exactitud reportadas en la literatura sobre LSA64 varían enormemente según el protocolo: los trabajos con evaluación dependiente del sujeto reportan valores cercanos al 99 %, mientras que con evaluación independiente los valores son sensiblemente menores. Comparar cifras obtenidas con protocolos distintos no tiene sentido, y por eso esta tesina declara explícitamente el suyo.

2.5 Métricas de evaluación

2.5.1 Exactitud

La **exactitud** (*accuracy*) es la proporción de ejemplos clasificados correctamente:

exactitud = predicciones correctas / total de predicciones

Es la métrica más intuitiva y la más usada en la literatura de reconocimiento de señas aisladas, lo que facilita la comparación. En conjuntos balanceados, como LSA64 —donde cada seña tiene el mismo número de videos—, es además una métrica representativa. Su principal debilidad es que resume todo en un número y oculta qué clases fallan y con cuáles se confunden.

2.5.2 Precisión, exhaustividad y F1

Para una clase c, la **precisión** es la proporción de las predicciones "c" que eran efectivamente c, y la **exhaustividad** (*recall*) es la proporción de los ejemplos de c que el modelo reconoció como tales. La medida **F1** es la media armónica de ambas. Promediando estas métricas sobre todas las clases (promedio *macro*) se obtiene una visión que no se deja dominar por las clases más frecuentes.

2.5.3 Matriz de confusión

La **matriz de confusión** es una tabla en la que la fila i y la columna j contienen la cantidad de ejemplos de la clase i que el modelo predijo como clase j. La diagonal concentra los aciertos; cada valor fuera de la diagonal es un tipo de error concreto. Para un problema de 64 clases es la herramienta de análisis más informativa, porque permite pasar de "el modelo acierta el 80 %" a "el modelo confunde sistemáticamente Aprender con Caramelo", que es un hallazgo interpretable y accionable (sección 14.4).

2.6 Bosques aleatorios

No todo problema de clasificación requiere redes neuronales. El **bosque aleatorio** (*random forest*) propuesto por Breiman (2001) combina muchos árboles de decisión, cada uno entrenado sobre una muestra aleatoria de los ejemplos y considerando en cada división un subconjunto aleatorio de las características. La predicción final es el voto mayoritario de los árboles. Es un algoritmo robusto, rápido de entrenar, poco sensible a los hiperparámetros y difícil de sobreajustar, lo que lo convierte en una **línea base** natural: un punto de comparación simple contra el cual justificar el uso de un modelo más complejo.

Su limitación para este problema es que trabaja con vectores de longitud fija, y no con secuencias de longitud variable. Para aplicarlo a una seña hay que resumir la secuencia en un único vector —por ejemplo, con la media, el desvío, el mínimo y el máximo de cada característica a lo largo del tiempo—, lo que descarta el orden temporal de los movimientos. En el sistema, el bosque aleatorio se utiliza para las configuraciones estáticas, donde un único cuadro basta (sección 8.4), y se propone como línea base para las señas dinámicas (sección 14.7).

Capítulo 3 — Redes neuronales y aprendizaje profundo

El modelo central del sistema es una red neuronal recurrente. Este capítulo recorre los conceptos necesarios para entenderla: la neurona artificial, el entrenamiento por descenso del gradiente, las técnicas de regularización y, finalmente, las redes recurrentes y la arquitectura LSTM bidireccional.

3.1 La neurona artificial

Una **red neuronal artificial** es un modelo compuesto por unidades simples interconectadas, inspiradas lejanamente en las neuronas biológicas. Cada unidad recibe un vector de entradas x, calcula una suma ponderada con un vector de pesos w, le agrega un sesgo b y aplica una **función de activación** no lineal g:

$$salida = g(w \cdot x + b)$$

Los pesos y el sesgo son los **parámetros** del modelo, los valores que se ajustan durante el entrenamiento. La función de activación es indispensable: sin ella, cualquier combinación de neuronas se reduce a una única función lineal y la red no puede representar relaciones complejas. Las activaciones más usadas son la sigmoide σ(z) = 1 / (1 + e^(−z)), que comprime cualquier valor al intervalo (0, 1); la tangente hiperbólica, que lo comprime a (−1, 1); y la unidad lineal rectificada (ReLU), que devuelve max(0, z) (Goodfellow et al., 2016).

3.2 Redes multicapa y aprendizaje profundo

Las neuronas se organizan en **capas**. En un **perceptrón multicapa** (MLP), la entrada atraviesa una o más capas ocultas antes de llegar a la capa de salida, y cada neurona de una capa se conecta con todas las de la capa siguiente. Con suficientes neuronas, un MLP puede aproximar cualquier función continua, lo que explica su versatilidad.

El término **aprendizaje profundo** (*deep learning*) se refiere a redes con muchas capas, en las que cada capa construye representaciones progresivamente más abstractas a partir de la anterior (Goodfellow et al., 2016). Su principal ventaja frente a los métodos clásicos es que aprende las características directamente de los datos en lugar de depender de características

diseñadas a mano. Esa capacidad tiene un costo: requiere muchos datos y mucho cómputo, dos recursos que, en el caso de LSA64, son escasos. Esta tensión explica una decisión central del sistema, desarrollada en el Capítulo 4: en lugar de aprender directamente de los píxeles, la red recibe una representación compacta de las manos calculada por un modelo previamente entrenado.

3.2.1 Capa de salida y función softmax

En un problema de clasificación con K clases, la capa de salida tiene K neuronas, una por clase. Sus valores, llamados *logits*, se transforman en una distribución de probabilidad mediante la función **softmax**, que exponencia cada valor y lo divide por la suma de todos. El resultado son K números positivos que suman uno; el mayor indica la clase predicha y su valor es la confianza del modelo. En el sistema, K = 64.

3.3 Entrenamiento

3.3.1 Función de pérdida

Entrenar una red significa encontrar los parámetros que minimizan una **función de pérdida**, que mide qué tan lejos están las predicciones de las etiquetas correctas. Para clasificación, la pérdida estándar es la **entropía cruzada**: el logaritmo negativo de la probabilidad que el modelo asignó a la clase correcta. Si el modelo asigna probabilidad 1 a la clase correcta, la pérdida es 0; si le asigna una probabilidad cercana a 0, la pérdida crece sin límite.

3.3.2 Descenso del gradiente y retropropagación

El **descenso del gradiente** ajusta cada parámetro en la dirección que más reduce la pérdida, en pasos cuyo tamaño controla la **tasa de aprendizaje**. El gradiente de la pérdida respecto de cada parámetro se calcula eficientemente con el algoritmo de **retropropagación** (*backpropagation*), que aplica la regla de la cadena desde la salida hacia la entrada (Rumelhart et al., 1986).

En la práctica, el gradiente no se calcula sobre todo el conjunto de entrenamiento sino sobre pequeños subconjuntos aleatorios llamados **lotes** (*batches*). Una pasada completa por todos los lotes es una **época**. El entrenamiento consiste en repetir épocas hasta que la pérdida deja de mejorar.

3.3.3 Optimizadores

El descenso del gradiente básico usa la misma tasa de aprendizaje para todos los parámetros. Los optimizadores modernos la adaptan individualmente. **Adam** (Kingma y Ba, 2015) mantiene un promedio móvil del gradiente y de su cuadrado para cada parámetro, lo que

acelera la convergencia y lo hace poco sensible a la elección de la tasa inicial; es el optimizador utilizado por defecto en el sistema. **AdamW** (Loshchilov y Hutter, 2019) es una variante que aplica la penalización de pesos de forma desacoplada del gradiente, y se incorporó para los experimentos de regularización (sección 10.5).

3.4 Regularización

Una red con cientos de miles de parámetros entrenada sobre unos pocos miles de ejemplos tiene capacidad de sobra para memorizarlos. Las técnicas de **regularización** limitan esa capacidad o hacen más difícil la memorización, con el objetivo de mejorar la generalización (Goodfellow et al., 2016). Las utilizadas en este trabajo son:

- **Detención temprana** (*early stopping*). Se evalúa el modelo sobre el conjunto de validación al final de cada época y se conserva el mejor; si no mejora durante un número de épocas fijado (la **paciencia**), el entrenamiento se detiene (Prechelt, 1998).
- **Dropout.** Durante el entrenamiento, cada neurona de una capa se desactiva al azar con una probabilidad p en cada paso. La red no puede depender de ninguna neurona en particular y aprende representaciones más redundantes y robustas (Srivastava et al., 2014). En la inferencia todas las neuronas están activas.
- **Penalización de pesos** (*weight decay*). Agrega a la pérdida un término proporcional a la magnitud de los pesos, lo que favorece soluciones más simples.
- **Aumentación de datos** (*data augmentation*). Genera variantes artificiales de los ejemplos de entrenamiento aplicando transformaciones que no cambian su clase. Para secuencias de landmarks, en este trabajo se implementaron tres: ruido gaussiano sobre las coordenadas, descarte aleatorio de cuadros y cambio de escala temporal (sección 10.5).

3.5 Redes neuronales recurrentes

Un MLP recibe un vector de tamaño fijo y no tiene noción de orden. Una seña, en cambio, es una **secuencia**: una sucesión de posturas de la mano cuyo orden importa —la misma configuración moviéndose hacia arriba o hacia abajo puede ser dos señas distintas— y cuya duración varía de una persona a otra y de una repetición a otra.

Las **redes neuronales recurrentes** (RNN) procesan secuencias elemento por elemento, manteniendo un **estado oculto** que resume lo visto hasta el momento. En cada paso t, la red combina la entrada actual x_t con el estado anterior h_(t−1) para producir el nuevo estado h_t. Los mismos pesos se aplican en todos los pasos, de modo que la red puede procesar secuencias de cualquier longitud con una cantidad fija de parámetros. Al final de la secuencia, el último estado oculto resume la secuencia completa y puede alimentar una capa de clasificación.

3.5.1 El problema del gradiente que se desvanece

Entrenar una RNN implica retropropagar el error a través de todos los pasos de la secuencia. En cada paso, el gradiente se multiplica por factores que suelen ser menores que uno, y después de decenas de pasos se vuelve prácticamente nulo. Bengio et al. (1994) mostraron que, como consecuencia, las RNN simples tienen enormes dificultades para aprender dependencias entre elementos alejados de la secuencia: el comienzo de una seña casi no influye en el aprendizaje cuando la red llega al final.

3.6 Long Short-Term Memory

La arquitectura **LSTM** (*Long Short-Term Memory*), propuesta por Hochreiter y Schmidhuber (1997), resuelve ese problema incorporando, además del estado oculto, una **celda de memoria** c_t por la que la información puede fluir a lo largo de la secuencia con modificaciones mínimas. El acceso a esa memoria está regulado por tres **compuertas**, cada una implementada como una capa con activación sigmoide que produce valores entre 0 (cerrado) y 1 (abierto):

- La **compuerta de olvido** f_t decide qué parte de la memoria anterior se conserva.
- La **compuerta de entrada** i_t decide qué parte de la nueva información candidata ĉ_t se escribe en la memoria.
- La **compuerta de salida** o_t decide qué parte de la memoria se expone como estado oculto.

La actualización de la memoria es $c_t = f_t \otimes c_{(t-1)} \oplus i_t \otimes \hat{c}_t$, y el estado oculto es $h_t = o_t \otimes \tanh(c_t)$, donde $\otimes$ denota el producto elemento a elemento. Como la memoria se actualiza mediante una suma y no mediante multiplicaciones sucesivas, el gradiente puede atravesar muchos pasos sin desvanecerse. La Figura 2 esquematiza la celda.

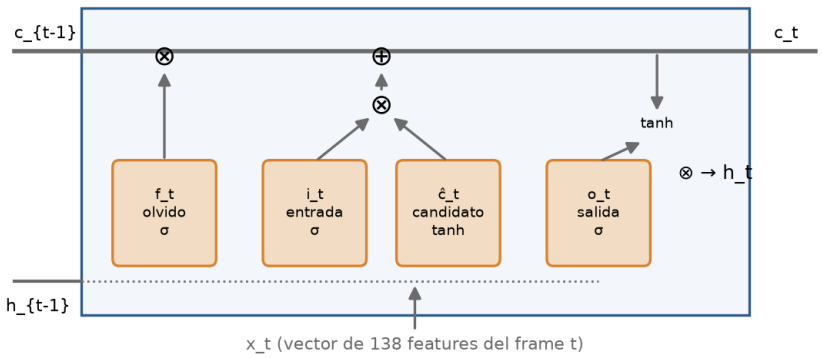


Figura *2. Esquema de una celda LSTM. Las compuertas de olvido, entrada y salida regulan qué información se conserva, se escribe y se expone en cada paso de la secuencia.*

3.7 LSTM bidireccional

Una LSTM procesa la secuencia de izquierda a derecha, de modo que su estado en el paso t sólo conoce el pasado. Cuando se clasifica una secuencia completa —como una seña ya terminada—, no hay razón para limitarse a esa dirección. Una **red recurrente bidireccional** (Schuster y Paliwal, 1997) combina dos redes: una recorre la secuencia hacia adelante y otra hacia atrás. Concatenando el estado final de cada una se obtiene una representación que tiene en cuenta tanto cómo empieza como cómo termina la seña. Esta arquitectura, denominada **BiLSTM**, es la que utiliza el sistema (sección 10.1).

3.7.1 Secuencias de longitud variable

Para entrenar en lotes, todas las secuencias de un lote deben tener la misma longitud, por lo que las más cortas se completan con ceros (*padding*). Si la red procesara esos ceros como si fueran datos, el estado final de las secuencias cortas quedaría contaminado por pasos vacíos. Los marcos de trabajo como PyTorch (Paszke et al., 2019) resuelven esto "empaquetando" las secuencias junto con su longitud real, de modo que la recurrencia se detiene en el último elemento válido de cada una. El modelo del sistema utiliza este mecanismo (ver el Anexo B).

3.8 Otras arquitecturas

El estado del arte en reconocimiento de video utiliza también otras arquitecturas, que se mencionan aquí porque aparecen en los antecedentes del Capítulo 5:

- **Redes convolucionales** (CNN). Aplican filtros que se desplazan sobre la imagen y detectan patrones locales —bordes, texturas, formas— de manera independiente de su posición. Son el estándar para el procesamiento de imágenes.
- **Redes convolucionales 3D**. Extienden los filtros a la dimensión temporal y procesan bloques de cuadros de video, capturando apariencia y movimiento a la vez. Las variantes R(2+1)D separan la convolución espacial de la temporal (Tran et al., 2018). Logran los mejores resultados sobre LSA64, pero requieren mucho más cómputo que un modelo sobre landmarks.
- **Transformers**. Reemplazan la recurrencia por mecanismos de atención que relacionan directamente todos los elementos de la secuencia entre sí (Vaswani et al., 2017). Se proponen como trabajo futuro en las conclusiones.

Capítulo 4 — Visión por computadora y estimación de la pose de la mano

Para que una red neuronal reconozca una seña, primero hay que convertir el video en números. Este capítulo describe cómo se representa una imagen digital, compara los dos grandes enfoques para extraer información de las manos y presenta MediaPipe Hands, la herramienta que utiliza el sistema para obtener la geometría de las manos en cada cuadro.

4.1 Imagen digital y video

Una **imagen digital** es una matriz de píxeles. En una imagen en color, cada píxel se describe con tres valores —rojo, verde y azul (RGB)— entre 0 y 255. Una imagen de 1920 × 1080 píxeles, la resolución de los videos de LSA64, contiene por lo tanto más de seis millones de valores. Un **video** es una sucesión de imágenes, o **cuadros** (*frames*), capturadas a una frecuencia constante medida en cuadros por segundo (fps). LSA64 fue grabado a 60 fps (Ronchetti et al., 2016b), de modo que una seña de dos segundos ocupa unos 120 cuadros.

La **visión por computadora** es el área que estudia cómo extraer información útil de imágenes y video. Su desafío central es la distancia entre la representación cruda —millones de valores de intensidad— y el contenido semántico —"una mano con el índice extendido a la altura del mentón"—. La misma seña produce imágenes muy distintas según la iluminación, el fondo, la ropa, el color de piel, la distancia a la cámara y la posición dentro del encuadre.

4.2 Enfoques para el reconocimiento de señas

Los sistemas de reconocimiento de lengua de señas basados en video siguen, en líneas generales, uno de dos enfoques (Koller, 2020).

4.2.1 Enfoque basado en la apariencia

El modelo recibe directamente los píxeles —usualmente reducidos a una resolución menor— y aprende por sí mismo qué patrones visuales son relevantes. Las redes convolucionales 2D y 3D mencionadas en la sección 3.8 son el ejemplo típico. Este enfoque puede capturar todo lo que está en la imagen, incluidos rasgos que un esqueleto no representa, como la expresión facial. A cambio, requiere muchos datos para aprender a ignorar lo irrelevante —el fondo, la ropa,

la iluminación—, demanda mucho cómputo y es sensible a cambios en las condiciones de grabación.

4.2.2 Enfoque basado en el esqueleto

Un modelo previo, entrenado sobre grandes volúmenes de datos para una tarea general, localiza en cada cuadro un conjunto de puntos de referencia o **landmarks** del cuerpo o de las manos —articulaciones y extremos de los dedos—. El clasificador de señas recibe únicamente las coordenadas de esos puntos.

Para este proyecto, el enfoque basado en el esqueleto ofrece ventajas decisivas:

- **Compacidad.** Un cuadro se reduce de seis millones de valores a unas decenas de coordenadas, lo que permite entrenar un clasificador pequeño, en segundos y sin GPU.
- **Invarianza a factores irrelevantes.** El fondo, la ropa, la iluminación y el color de piel quedan absorbidos por el detector, que ya fue entrenado para ignorarlos. El clasificador trabaja sólo con la geometría.
- **Transferencia.** El detector fue entrenado con cientos de miles de imágenes de manos que ningún proyecto académico podría reunir. Su conocimiento se reutiliza sin costo.
- **Privacidad y ancho de banda.** Las coordenadas no permiten reconstruir la imagen de la persona, y ocupan una fracción mínima del tamaño de un cuadro.

Su desventaja es que el clasificador queda limitado por la calidad del detector: si este no encuentra la mano en un cuadro, esa información se pierde por completo. El Capítulo 14 muestra que esta limitación es, de hecho, la principal fuente de variación en el desempeño del sistema.

4.3 MediaPipe

MediaPipe es un marco de trabajo de código abierto desarrollado por Google para construir procesos de percepción sobre video y audio (Lugaresi et al., 2019). Ofrece soluciones listas para usar —detección de rostros, estimación de la pose corporal, segmentación, seguimiento de manos— optimizadas para ejecutarse en tiempo real en dispositivos comunes, incluidos teléfonos y navegadores.

4.3.1 MediaPipe Hands

La solución de manos de MediaPipe, descripta por Zhang et al. (2020), estima la posición de 21 puntos de referencia por mano a partir de una imagen RGB común. Funciona como un proceso de dos etapas:

1. Un **detector de palmas** localiza las manos en la imagen completa y devuelve un recuadro orientado alrededor de cada una. Detectar la palma, un objeto relativamente rígido, es más fácil que detectar la mano completa con sus dedos articulados.

2. Un **modelo de landmarks** recibe el recorte de cada mano y predice las coordenadas de los 21 puntos. En video, el recuadro de un cuadro se usa para recortar el siguiente, de modo que el detector de palmas sólo se ejecuta cuando se pierde el seguimiento.

   Para cada mano detectada, MediaPipe devuelve:

- Las coordenadas x e y de cada punto, normalizadas al intervalo [0, 1] respecto del ancho y el alto de la imagen.
- Una coordenada z que representa la profundidad relativa de cada punto respecto de la muñeca: valores menores indican puntos más cercanos a la cámara.
- La **lateralidad** (*handedness*): si se trata de una mano izquierda o derecha, junto con una puntuación de confianza.

La Figura 3 muestra la numeración de los 21 puntos: el 0 corresponde a la muñeca, y cada dedo aporta cuatro puntos, desde la articulación con la palma hasta la punta (pulgar: 1 a 4; índice: 5 a 8; medio: 9 a 12; anular: 13 a 16; meñique: 17 a 20).

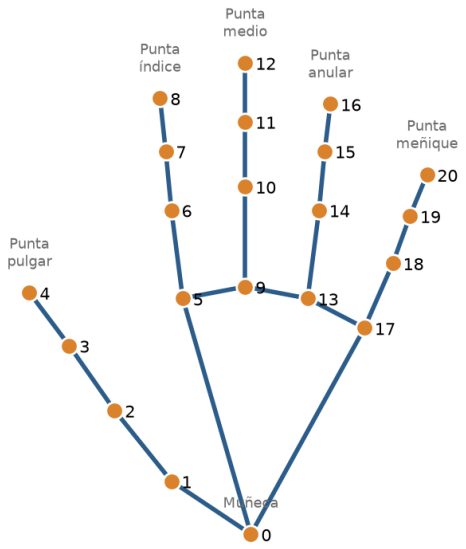


Figura *3. Los 21 puntos de referencia de la mano que estima MediaPipe Hands. Las líneas indican las conexiones del esqueleto que el sistema dibuja sobre el video.*

4.3.2 La API Tasks y la API heredada

MediaPipe ofreció durante años una interfaz de programación conocida como *Solutions* (`mediapipe.solutions.hands`), que es la que utilizan la mayoría de los tutoriales y la que usaba el prototipo de escritorio que sirvió como punto de partida de este proyecto. Al construir el sistema se comprobó que esa interfaz ya no está incluida en las versiones actuales del paquete distribuido por PyPI[2]: en la versión 0.10.33, el atributo `solutions` directamente no existe. El sistema utiliza

[2]Es un problema conocido y reportado en el repositorio oficial de MediaPipe (google-ai-edge/mediapipe, *issues* 6192, 6200 y 6204).

en su lugar la interfaz soportada, **MediaPipe Tasks** (`HandLandmarker`), que ofrece el mismo modelo pero requiere descargar explícitamente su archivo de pesos (`hand_landmarker.task`).

El hallazgo es menor en lo técnico, pero ilustra un riesgo concreto de construir sobre prototipos o tutoriales de terceros sin auditarlos: el código puede dejar de funcionar no por un error propio, sino por cambios en las dependencias. Por eso el proyecto fija las versiones de sus dependencias y documenta el paso de descarga del modelo (ver el Anexo C).

4.4 Lateralidad y orientación de la imagen

La lateralidad que informa MediaPipe se deduce de la imagen: el modelo decide si una mano es izquierda o derecha según cómo se ve. Esto tiene una consecuencia que no es evidente. Las cámaras web suelen mostrarse **espejadas** —como un espejo— porque así resulta natural para el usuario. Si una imagen se procesa espejada, la mano derecha de la persona aparece del lado derecho de la imagen y tiene el aspecto de una mano izquierda; MediaPipe la etiquetará en consecuencia.

Mientras la orientación sea la misma durante el entrenamiento y durante el uso, esto no es un problema: el modelo aprende con una convención y se usa con la misma. El problema aparece cuando las dos etapas usan orientaciones distintas, porque las manos quedan intercambiadas respecto de lo aprendido. La sección 9.7 relata cómo este desajuste se produjo efectivamente en el proyecto, por qué ningún control automático podía detectarlo y cómo se resolvió.

4.5 Limitaciones de la detección

El detector de manos no es infalible. Sus fallas más comunes son:

- **Desenfoque por movimiento.** En los movimientos rápidos, la mano aparece borrosa y el detector puede no encontrarla.
- **Oclusiones.** Cuando una mano tapa a la otra, o cuando la mano pasa por delante de la cara, los puntos ocultos se estiman con poca precisión o la mano no se detecta.
- **Manos fuera del encuadre o de perfil.** Algunas señas se realizan cerca del borde de la imagen o con la mano de canto respecto de la cámara.
- **Apariencias atípicas.** El detector fue entrenado con manos desnudas. En LSA64, las personas usaron guantes de colores fluorescentes para facilitar la segmentación (Ronchetti et al., 2016b), lo que podría afectar la detección de MediaPipe. Esta hipótesis no fue verificada en este trabajo, pero es consistente con la tasa de detección observada (sección 9.10).

Cuando el detector no encuentra una mano, el sistema lo registra explícitamente mediante un indicador de presencia (sección 9.4), en lugar de producir coordenadas en cero que el modelo podría confundir con una mano real.

Capítulo 5 — Reconocimiento automático de lengua de señas: estado del arte

El reconocimiento automático de lengua de señas es un área activa de investigación que combina visión por computadora, aprendizaje automático y lingüística. Este capítulo delimita el problema que aborda la tesina dentro del área, describe los principales conjuntos de datos y revisa los antecedentes específicos sobre la LSA, con especial atención a los protocolos de evaluación con que se obtuvieron sus resultados.

5.1 Tareas del área

Koller (2020) distingue, en su revisión cuantitativa del estado del arte, tres tareas de dificultad creciente:

- **Reconocimiento de señas aisladas.** Cada video contiene una única seña, con un comienzo y un final definidos, y el objetivo es clasificarla dentro de un vocabulario cerrado. Es la tarea que aborda esta tesina.
- **Reconocimiento continuo.** El video contiene una frase completa y el sistema debe producir la secuencia de señas, sin conocer dónde termina una y empieza la siguiente. Agrega el problema de la segmentación y de las transiciones entre señas, que modifican su forma.
- **Traducción.** El sistema debe producir una oración gramatical en la lengua oral, lo que implica resolver las diferencias de orden y estructura entre ambas lenguas. Camgöz et al. (2018) formularon este problema como traducción automática neuronal de extremo a extremo.

La distinción importa para dimensionar correctamente el alcance del sistema: reconocer 64 señas aisladas es un problema útil y no trivial, pero está lejos de la traducción de la LSA.

5.2 Conjuntos de datos

El progreso del área está ligado a la disponibilidad de conjuntos de datos públicos, cuya construcción es costosa: requiere grabar a muchas personas, cada una realizando muchas señas, y etiquetar cada video. La Tabla 2 resume los conjuntos más relevantes para este trabajo.

Tabla *2. Conjuntos de datos de lengua de señas relevantes para el trabajo.*

Conjunto	Lengua	Tipo	Clases	Personas	Videos / imágenes	Referencia
LSA64	Argentina	Señas aisladas	64	10	3200 videos	Ronchetti et al. (2016b)
LSA16	Argentina	Configuraciones manuales	16	10	800 imágenes	Ronchetti et al. (2016a)
WLASL	Americana	Señas aisladas	2000	119	21 083 videos	Li et al. (2020)
AUTSL	Turca	Señas aisladas	226	43	38 336 videos	Sincan y Keles (2020)

5.2.1 LSA64

LSA64 fue desarrollado por el Instituto de Investigación en Informática LIDI de la Universidad Nacional de La Plata y constituye el primer conjunto de datos público de LSA orientado al aprendizaje automático (Ronchetti et al., 2016b). Contiene 3200 videos de 64 señas de uso cotidiano, realizadas por 10 personas no expertas, diestras, con 5 repeticiones de cada seña. Las grabaciones se hicieron en 1920 × 1080 píxeles a 60 cuadros por segundo, con fondo blanco, vestimenta negra y guantes de colores para facilitar el seguimiento de las manos. De las 64 señas, 42 se realizan con una mano y 22 con ambas. Una versión ampliada de su descripción fue publicada posteriormente por los mismos autores (Ronchetti et al., 2023). Se distribuye bajo una licencia Creative Commons de uso no comercial, compatible con un trabajo académico.

LSA64 es el conjunto utilizado para entrenar y evaluar el modelo de señas de esta tesina. Su tamaño es modesto para los estándares del aprendizaje profundo —50 videos por seña—, lo que favorece los enfoques basados en el esqueleto (sección 4.2.2).

5.2.2 LSA16

LSA16, del mismo grupo de investigación, contiene 800 imágenes de 16 configuraciones manuales de la LSA realizadas por 10 personas (Ronchetti et al., 2016a). Las configuraciones manuales son las formas básicas que adopta la mano y se relacionan con las letras del alfabeto manual, aunque no son lo mismo: 16 configuraciones no cubren el abecedario. Se menciona aquí porque es la alternativa pública más cercana para el reconocimiento de configuraciones estáticas, un alcance todavía en definición en este proyecto (ver el Capítulo 8).

5.3 Antecedentes sobre LSA64

La Tabla 3 resume los antecedentes revisados sobre LSA64. La columna de protocolo es la más importante: como se explicó en la sección 2.4.3, una cifra de exactitud no puede interpretarse sin saber si las personas de la prueba estaban o no en el entrenamiento.

Tabla *3. Resultados publicados sobre LSA64 según el protocolo de evaluación.*

Trabajo	Entrada	Modelo	Protocolo	Exactitud
Mindlin (2021)	Video RGB, 128 × 128, 16 cuadros	Red convolucional 3D	Dependiente del sujeto (5 particiones aleatorias)	99,4 %
Marais et al. (2022)	Video RGB	InceptionV3 + GRU	Dependiente del sujeto	97,03 %
Marais et al. (2022)	Video RGB	InceptionV3 + GRU	Independiente del sujeto	74,22 %
Akdag y Baykan (2024)	Video RGB y fusión de características	Red R3(2+1)D	Independiente del sujeto	98,53 %
Esta tesina	Landmarks de MediaPipe (138 valores por cuadro)	BiLSTM	Independiente del sujeto (LOSO, 10 particiones)	79,8 % ± 8,9 %

5.3.1 Mindlin (2021)

La tesina de licenciatura de Mindlin (2021), dirigida por Franco Ronchetti en la Universidad Nacional de La Plata, entrenó modelos convolucionales 3D, convolucionales recurrentes y recurrentes sobre LSA64, con distintos preprocesamientos del video, y alcanzó un 99,4 % de exactitud con el mejor de ellos. Es el antecedente que se citó en el anteproyecto de este trabajo.

Al revisar su metodología se constató un dato que el anteproyecto omitía: la evaluación se realizó con validación cruzada de 5 particiones armadas tomando videos al azar de cada clase, y el propio autor aclara que "esta separación no es independiente del sujeto". Es decir, las personas de la prueba también estaban en el entrenamiento. El 99,4 % mide, por lo tanto, la capacidad de reconocer señas de personas conocidas, y no es comparable con el resultado de esta tesina, que mide el desempeño sobre personas nuevas. El mismo trabajo concluye que LSA64 "no representa un desafío para el estado del arte" bajo ese protocolo, y propone como trabajo futuro el uso de datos con desafíos del mundo real.

5.3.2 Marais et al. (2022)

El trabajo de Marais et al. (2022) es el antecedente más directamente relacionado con el problema de evaluación. Con una misma arquitectura —una red convolucional InceptionV3 para extraer características de cada cuadro seguida de una red recurrente GRU—, compararon ambos protocolos sobre LSA64: 97,03 % dependiente del sujeto frente a 74,22 % independiente del sujeto. La caída de más de 20 puntos, con el mismo modelo y los mismos datos, cuantifica la magnitud del optimismo que introduce mezclar personas entre entrenamiento y prueba.

5.3.3 Akdag y Baykan (2024)

Akdag y Baykan (2024) reportaron 98,53 % de exactitud independiente del sujeto sobre LSA64, con una red convolucional 3D que combina bloques residuales y convoluciones espaciotemporales separadas, operando sobre el video completo y fusionando distintas características. Es el mejor resultado independiente del sujeto encontrado, y muestra que el techo del problema es alto. El costo es un modelo pesado sobre video crudo, que requiere una GPU para la inferencia y no es compatible con una aplicación web en la capa gratuita de un servicio en la nube, que es una restricción explícita de este proyecto.

5.3.4 Posicionamiento del resultado propio

El resultado de esta tesina, 79,8 % ± 8,9 %, se ubica por encima del resultado independiente del sujeto de Marais et al. (2022) y claramente por debajo del de Akdag y Baykan (2024). La diferencia con este último se explica principalmente por la representación de entrada: el modelo propio no ve la imagen, sino sólo la geometría de las manos que entrega MediaPipe, y pierde la información de los cuadros donde el detector falla (sección 14.5). A cambio, el modelo tiene 290 880 parámetros, se entrena en segundos sobre CPU y permite la inferencia en tiempo real en un servidor modesto. La elección no es la de mayor exactitud posible, sino la de mejor equilibrio entre exactitud y viabilidad para el objetivo de una aplicación web accesible.

5.4 Aplicaciones existentes

Fuera del ámbito académico existen aplicaciones de reconocimiento de lengua de señas, en su mayoría para la Lengua de Señas Americana y basadas en el alfabeto manual, y diversas aplicaciones para el aprendizaje de la LSA que funcionan como diccionarios. En la revisión realizada para el anteproyecto no se encontraron aplicaciones accesibles desde el navegador que reconozcan señas de la LSA en tiempo real a partir de la cámara del dispositivo. Los trabajos académicos sobre LSA64 revisados se limitan a experimentos de laboratorio sin una interfaz de uso.

5.5 Brecha que aborda el trabajo

A partir de la revisión, se identifica la siguiente brecha: los trabajos existentes sobre LSA reportan resultados de clasificación sobre videos pregrabados, muchas veces con protocolos dependientes del sujeto, y no integran el modelo en un sistema utilizable. Esta tesina aporta:

1. Un modelo para las 64 señas de LSA64 evaluado con un protocolo independiente del sujeto (LOSO), con análisis por persona y por clase.
2. La integración de ese modelo en una aplicación web en tiempo real, que funciona con la cámara del dispositivo y está diseñada para ejecutarse en infraestructura gratuita.

3. La documentación, con evidencia, de los errores metodológicos que producen resultados optimistas en este tipo de sistemas y de cómo detectarlos.

Capítulo 6 — Aplicaciones web en tiempo real

El modelo de reconocimiento sólo es útil si llega a las personas. Este capítulo presenta las tecnologías que permiten llevarlo a una aplicación web: la arquitectura cliente-servidor, los protocolos de comunicación, las interfaces del navegador para acceder a la cámara, el marco de trabajo del backend, los contenedores para el despliegue y las pautas de accesibilidad.

6.1 Arquitectura cliente-servidor

En una **arquitectura cliente-servidor**, un programa —el servidor— ofrece un servicio a través de la red, y otros programas —los clientes— lo consumen enviándole solicitudes. En una aplicación web, el cliente es el navegador del usuario, que ejecuta una interfaz escrita en HTML, CSS y JavaScript, y el servidor es un proceso que corre en una computadora remota.

Para este proyecto, la división de responsabilidades entre ambos extremos es una decisión de diseño con consecuencias: dónde se ejecuta la detección de manos determina cuántos datos viajan por la red, cuánto cómputo necesita el servidor y qué tan potente debe ser el dispositivo del usuario. La sección 8.8 analiza las alternativas.

6.2 HTTP y el estilo REST

El protocolo **HTTP** es la base de la web: el cliente envía una solicitud —por ejemplo, `GET /health`— y el servidor devuelve una respuesta. Cada intercambio es independiente y lo inicia siempre el cliente. Fielding (2000) formalizó el estilo arquitectónico **REST**, que organiza un servicio como un conjunto de recursos identificados por direcciones y manipulados mediante los métodos estándar de HTTP, sin estado entre solicitudes. Es el estilo dominante para las interfaces de programación web (API), y el sistema lo utiliza para sus servicios de consulta (sección 11.2).

REST es adecuado para operaciones aisladas, pero poco eficiente para un flujo continuo como el video. Enviar cada cuadro como una solicitud HTTP independiente implica repetir en cada envío las cabeceras del protocolo y, si no se reutilizan las conexiones, el costo de establecerlas. Con ocho cuadros por segundo, ese costo se multiplica.

6.3 WebSocket

El protocolo **WebSocket** (Fette y Melnikov, 2011) establece, a partir de una solicitud HTTP inicial, un canal persistente y bidireccional entre cliente y servidor. Una vez abierto, cualquiera de los dos extremos puede enviar mensajes —de texto o binarios— en cualquier momento, con una sobrecarga mínima por mensaje. Es el protocolo estándar para aplicaciones con comunicación continua, como chats, juegos o tableros en vivo.

La Tabla 4 compara las alternativas consideradas para transportar el video al servidor.

Tabla *4. Alternativas de comunicación para enviar video al servidor.*

Criterio	HTTP por cuadro (REST)	WebSocket	WebRTC
Conexión	Una solicitud por cuadro	Persistente	Persistente, entre pares
Sobrecarga por cuadro	Alta (cabeceras completas)	Baja	Baja
Respuesta del servidor	En la misma solicitud	Mensaje independiente en el mismo canal	Canal de datos aparte
Estado por cliente	Hay que reconstruirlo en cada solicitud	Natural: una conexión, una sesión	Natural
Complejidad	Baja	Baja	Alta (señalización, NAT, códecs)
Soporte en FastAPI	Nativo	Nativo	Requiere bibliotecas externas

Se eligió WebSocket por su equilibrio entre eficiencia y simplicidad. Además, la persistencia de la conexión resuelve naturalmente un requisito del sistema: cada usuario necesita su propio estado —el historial reciente de predicciones y la ventana de cuadros acumulados—, y una conexión WebSocket delimita exactamente la vida de ese estado (sección 11.3). WebRTC, diseñado para videollamadas, ofrecería mejor manejo del video pero agrega una complejidad que no se justifica para el alcance del proyecto.

6.4 Acceso a la cámara desde el navegador

Los navegadores modernos permiten acceder a la cámara mediante la interfaz `MediaDevices.getUserMedia()`, que solicita el permiso del usuario y devuelve un flujo de video que puede mostrarse en un elemento `<video>` (Mozilla, s.f.). Por razones de seguridad, el acceso sólo está permitido en contextos seguros: páginas servidas por HTTPS o desde la propia máquina (`localhost`). Este requisito condiciona el despliegue, porque la aplicación publicada debe servirse con HTTPS para que la cámara funcione.

Para enviar un cuadro al servidor, el cliente lo dibuja sobre un elemento `<canvas>` y lo codifica como imagen JPEG. El canvas permite además transformar la imagen antes de enviarla

—por ejemplo, reducir su resolución o espejarla horizontalmente—, operaciones que el sistema utiliza para disminuir el volumen de datos y para respetar la convención de orientación de la sección 4.4.

6.5 El backend en Python

6.5.1 FastAPI

Python es el lenguaje dominante en aprendizaje automático: PyTorch, scikit-learn (Pedregosa et al., 2011), OpenCV y MediaPipe ofrecen sus interfaces principales en Python. Escribir el backend en el mismo lenguaje permite que el servidor reutilice exactamente el mismo código de preprocesamiento que se usa para entrenar, que es un requisito central del diseño (sección 8.5).

FastAPI es un marco de trabajo web para Python orientado a la construcción de API (Ramírez, s.f.). Se basa en el estándar **ASGI**, que a diferencia de su antecesor WSGI soporta programación asincrónica y conexiones de larga duración como WebSocket. Entre sus características se destacan la validación automática de datos a partir de las anotaciones de tipos de Python y la generación automática de documentación de la API en formato OpenAPI, navegable desde el propio servidor. El servidor ASGI utilizado para ejecutarlo es **Uvicorn**.

6.5.2 Concurrencia y trabajo bloqueante

FastAPI atiende muchas conexiones simultáneas con un único hilo mediante un **bucle de eventos**: mientras una conexión espera datos de la red, el bucle atiende a otra. El modelo funciona bien mientras cada tarea ceda el control rápidamente. Sin embargo, el procesamiento de un cuadro —decodificar el JPEG, ejecutar MediaPipe y la red neuronal— es cómputo intensivo y sincrónico: si se ejecuta directamente dentro del bucle, lo bloquea, y todas las demás conexiones esperan. Con dos usuarios simultáneos, las traducciones se serializan y la latencia de cada uno se duplica.

La solución estándar es delegar el trabajo bloqueante a un conjunto de hilos auxiliares (*thread pool*), dejando el bucle de eventos libre para la comunicación. El sistema aplica esta técnica (sección 11.3), con la salvedad de que el detector de MediaPipe no admite ser usado por varios hilos a la vez, por lo que su acceso se protege con un cerrojo.

6.6 Contenedores y despliegue en la nube

6.6.1 Docker

Una aplicación de aprendizaje automático depende de muchas bibliotecas con versiones específicas, algunas con componentes nativos. Reproducir ese entorno en otra máquina es una

fuente habitual de errores. **Docker** empaqueta la aplicación junto con su sistema operativo base, sus dependencias y su configuración en una **imagen**, que se ejecuta como un **contenedor** aislado de forma idéntica en cualquier máquina que tenga Docker instalado (Merkel, 2014; Docker Inc., s.f.). La imagen se describe en un archivo de texto, el `Dockerfile`, que se versiona junto con el código.

6.6.2 Plataformas como servicio y capa gratuita

Las **plataformas como servicio** (PaaS), como Render, Railway o Google Cloud Run, ejecutan contenedores sin que el desarrollador administre servidores. Varias ofrecen una **capa gratuita** con restricciones típicas: poca memoria y CPU compartida, suspensión del servicio tras un período de inactividad —con un arranque lento en la siguiente solicitud— y un número limitado de servicios. Desplegar dentro de esas restricciones, como exige el anteproyecto, obliga a tomar decisiones concretas: servir el cliente y la API desde un único servicio, mantener la imagen liviana y no descargar recursos al arrancar (sección 13.4).

6.7 Accesibilidad web

Una herramienta cuyo propósito es la inclusión debe ser, ella misma, accesible. Las **Pautas de Accesibilidad para el Contenido Web** (WCAG) del World Wide Web Consortium (2023) organizan los requisitos en cuatro principios: el contenido debe ser **perceptible**, **operable**, **comprensible** y **robusto**. Cada pauta tiene criterios verificables clasificados en tres niveles de conformidad (A, AA y AAA). Algunos criterios de nivel AA relevantes para el proyecto son:

- Contraste mínimo de 4,5:1 entre el texto y su fondo.
- Tamaño mínimo de 24 × 24 píxeles para los objetivos táctiles.
- Foco del teclado visible en todos los controles.
- Información que cambia dinámicamente anunciada a las tecnologías de asistencia, como los lectores de pantalla, mediante regiones vivas (`aria-live`).

Cabe una observación: los usuarios directos de la herramienta —una persona sorda que seña y una persona oyente que lee la traducción— no necesariamente usan lectores de pantalla, pero la accesibilidad no se agota en un perfil de usuario. Una persona con baja visión, o con una discapacidad motriz, puede ser cualquiera de los dos interlocutores. La sección 12.6 detalla cómo se aplicaron estas pautas.

Capítulo 7 — Metodología de desarrollo

Este capítulo describe cómo se organizó el trabajo: el marco de gestión adoptado, la división de tareas, el manejo del código fuente y las prácticas que se incorporaron para que los resultados experimentales fueran confiables y reproducibles.

7.1 Enfoque ágil

El proyecto combinaba dos frentes con incertidumbre alta: un modelo de aprendizaje automático cuyo desempeño no podía conocerse de antemano, y una aplicación cuyos requisitos de latencia y usabilidad sólo podían validarse con el sistema funcionando. En ese contexto, planificar todo el desarrollo al comienzo era poco realista. Se adoptó por eso un enfoque ágil, basado en los valores del Manifiesto Ágil (Beck et al., 2001): priorizar el software funcionando, responder al cambio y trabajar en ciclos cortos con entregas incrementales.

7.1.1 Scrum adaptado

El marco elegido fue **Scrum** (Schwaber y Sutherland, 2020), adaptado a un equipo de dos personas. Scrum organiza el trabajo en iteraciones de duración fija llamadas **sprints**, cada una con un objetivo y un incremento verificable, y define eventos de planificación, revisión y retrospectiva. La guía oficial prevé tres roles —dueño del producto, desarrolladores y *Scrum Master*—, que en un equipo de dos no tienen sentido como roles separados. Las adaptaciones fueron:

- **Sprints de dos semanas**, con una reunión de planificación al inicio y una revisión con retrospectiva al final.
- **Sin roles formales.** Las decisiones de alcance se tomaron en conjunto, y los profesores coordinador y tutor actuaron como referentes externos en las revisiones de avance.
- **Integración frecuente** entre los dos frentes de trabajo, para detectar temprano las incompatibilidades entre el modelo y la aplicación.

7.1.2 Tablero Kanban

Las tareas se gestionaron en un tablero **Kanban** con tres columnas —por hacer, en progreso y terminado—, que hace visible el estado del trabajo y limita la cantidad de tareas abiertas al mismo tiempo. **PENDIENTE: indicar la herramienta efectivamente usada (GitHub Projects o Trello) e incluir una captura del tablero.**

7.2 División del trabajo

El trabajo se organizó en dos frentes, cada uno con un responsable principal:

- **Modelo de reconocimiento** (Jonatan Videla): preparación de los datos de LSA64, extracción de características, entrenamiento de la red, validación leave-one-subject-out y experimentos de mejora.
- **Aplicación** (Leandro Provenzano): arquitectura del sistema, backend de inferencia, cliente web, accesibilidad, pruebas automatizadas, contenedor de despliegue y documentación.

PENDIENTE: el anteproyecto asignaba los roles de forma inversa (Leandro: red neuronal; Jonatan: aplicación web). Confirmar la división real y unificar el criterio con la sección de Recursos Humanos.

Ambos frentes compartían un punto de contacto crítico: el módulo de extracción de características, que debe producir exactamente los mismos vectores durante el entrenamiento y durante el uso. Ese módulo se diseñó desde el comienzo como código compartido entre los dos frentes (sección 8.5), y su contrato se documentó en el propio código.

7.3 Control de versiones

El código se gestionó con **Git** y se alojó en un repositorio de GitHub (Chacon y Straub, 2014). Se trabajó con ramas por línea de trabajo, que se integraban mediante fusiones (*merge*) explícitas:

- `main`: versión estable del proyecto.
- `test`: rama de integración, donde se combinan los avances de los dos frentes y se verifica que el sistema funcione de punta a punta.
- `lean`: desarrollo de la aplicación (backend y cliente).
- `Modulo-LSA64`: desarrollo del pipeline de datos y del modelo sobre LSA64.

Los mensajes de los *commits* siguieron una convención con prefijos que indican el tipo de cambio —`feat` para funcionalidad nueva, `fix` para correcciones, `chore` para mantenimiento—, y se redactaron explicando no sólo qué se cambió sino por qué. Ese historial resultó valioso como registro de las decisiones: varios apartados de este documento, en particular los de la sección 9.7 y el Capítulo 14, se reconstruyeron a partir de él.

7.4 Pruebas automatizadas

Desde el inicio se escribieron **pruebas automatizadas** con pytest para los componentes con lógica no trivial: la normalización de las características, la carga y validación de modelos, el suavizado temporal y el pipeline de datos de LSA64. Una práctica que resultó especialmente útil fue escribir una **prueba de regresión** por cada error real encontrado: una prueba que falla con el

código defectuoso y pasa con el corregido, documentada con la etiqueta `REGRESIÓN` y una explicación del problema. Así, cada error corregido queda protegido contra su reaparición. El Capítulo 13 describe la suite completa.

7.5 Rigor experimental

El desarrollo del modelo se trató como un proceso experimental, con prácticas tomadas de la investigación científica:

- **Un protocolo de evaluación fijo.** Todos los resultados que se informan provienen del mismo protocolo (LOSO) y del mismo conjunto de características, de modo que son comparables entre sí.
- **Reproducibilidad.** Todas las fuentes de aleatoriedad —el reparto de los datos, la inicialización de los pesos, el dropout y el orden de los lotes— se controlan con una semilla. Dos ejecuciones con la misma configuración producen el mismo resultado, lo que permite atribuir cualquier diferencia a los cambios introducidos y no al azar (sección 10.4).
- **Línea base inmutable.** Cada nueva opción de entrenamiento se incorporó desactivada por defecto, de modo que una ejecución sin opciones reprodujera exactamente el resultado de referencia.
- **Hipótesis explícitas.** Cada experimento se planteó como una hipótesis con un resultado esperado, y los resultados negativos se documentaron con el mismo detalle que los positivos (sección 14.6).
- **Registro de resultados.** Cada ejecución guarda un reporte en formato JSON con sus hiperparámetros, su reparto de datos y sus métricas, versionado en el repositorio.

7.6 Herramientas

La Tabla 5 resume las herramientas utilizadas.

Tabla *5. Herramientas utilizadas en el desarrollo.*

Área	Herramienta	Uso
Lenguajes	Python 3.12, JavaScript, HTML5, CSS3	Backend y modelo; cliente web
Aprendizaje automático	PyTorch, scikit-learn, NumPy	Red BiLSTM, bosque aleatorio y MLP, cálculo numérico
Visión por computadora	MediaPipe (API Tasks), OpenCV	Detección de landmarks, decodificación de video e imágenes
Backend	FastAPI, Uvicorn	API REST, WebSocket, servidor ASGI
Pruebas	pytest	Pruebas unitarias, de regresión y de integración
Despliegue	Docker	Imagen del servicio
Gestión	Git, GitHub	Control de versiones y trabajo colaborativo
Documentación	Markdown, matplotlib	Documentación técnica y figuras de resultados

Área	Herramienta	Uso
Edición	Visual Studio Code	Editor principal

PENDIENTE: decidir si se declara el uso de asistentes de programación basados en IA durante el desarrollo y la documentación. Si se usaron, se recomienda declararlo aquí con una frase que describa el uso (por ejemplo, como apoyo en revisión de código y redacción, con verificación humana de todo resultado).

7.7 Cronograma

El Anexo D presenta el cronograma del proyecto en forma de diagrama de Gantt, con las etapas cumplidas y las restantes hasta la entrega.

PARTE II

Desarrollo de la implementación

Capítulo 8 — Análisis y diseño del sistema

Este capítulo inicia la segunda parte del trabajo. Presenta los requisitos del sistema, sus casos de uso y la arquitectura adoptada, junto con las decisiones de diseño más relevantes y las alternativas descartadas.

8.1 Requisitos funcionales

La Tabla 6 enumera los requisitos funcionales, derivados de los objetivos y alcances del proyecto.

Tabla *6.* *Requisitos funcionales del sistema.*

Código	Requisito	Estado
RF-01	Capturar video de la cámara del dispositivo desde el navegador, con permiso del usuario	Implementado
RF-02	Reconocer, en tiempo real, las 64 señas del vocabulario de LSA64	Implementado
RF-03	Mostrar la seña reconocida junto con la confianza del modelo y la estabilidad de la detección	Implementado
RF-04	Acumular las señas reconocidas en un texto que el usuario pueda copiar, corregir o borrar	Implementado
RF-05	Dibujar el esqueleto de las manos detectadas sobre el video, con opción a ocultarlo	Implementado
RF-06	Permitir ajustar la frecuencia de envío de cuadros	Implementado
RF-07	Informar al usuario el estado de la conexión y reconectar automáticamente si se interrumpe	Implementado
RF-08	Reconocer configuraciones manuales estáticas (alfabeto manual)	Parcial: pipeline implementado, datos pendientes
RF-09	Exponer el estado del servicio y la información del modelo cargado	Implementado

8.2 Requisitos no funcionales

Tabla *7.* *Requisitos no funcionales del sistema.*

Código	Categoría	Requisito
RNF-01	Desempeño	La latencia entre el envío de un cuadro y la recepción de su resultado debe permitir una interacción fluida (objetivo: menos de 300 ms en una red local)
RNF-02	Exactitud	El desempeño del modelo debe medirse sobre personas no vistas en el entrenamiento

Código	Categoría	Requisito
RNF-03	Portabilidad	El cliente debe funcionar en navegadores modernos de escritorio y móviles, sin instalación
RNF-04	Accesibilidad	La interfaz debe cumplir los criterios de nivel AA de WCAG 2.2 aplicables
RNF-05	Costo	El despliegue debe funcionar dentro de la capa gratuita de una plataforma en la nube
RNF-06	Privacidad	El video de los usuarios no debe almacenarse
RNF-07	Concurrencia	Usuarios simultáneos no deben interferir entre sí en sus resultados
RNF-08	Mantenibilidad	El preprocesamiento debe estar definido en un único lugar para entrenamiento e inferencia
RNF-09	Robustez	El servidor debe arrancar aunque falte un componente, informando el problema en lugar de fallar

8.3 Actores y casos de uso

El sistema tiene dos actores humanos. La **persona que seña** se ubica frente a la cámara y realiza las señas; la **persona que lee** observa la traducción en la pantalla. En el uso típico —un mostrador de atención, por ejemplo— ambas comparten el mismo dispositivo. Un tercer actor, el **desarrollador**, interactúa con el sistema para entrenar y publicar nuevos modelos. La Tabla 8 resume los casos de uso.

Tabla *8***.** *Casos de uso principales.*

Caso de uso	Actor	Descripción
CU-01 Iniciar traducción	Persona que seña	Inicia la cámara; el sistema solicita permiso, abre la conexión con el servidor y comienza a enviar cuadros
CU-02 Realizar una seña	Persona que seña	Realiza una seña frente a la cámara; el sistema muestra el esqueleto de las manos y, al estabilizarse la predicción, la seña reconocida
CU-03 Leer la traducción	Persona que lee	Lee la seña actual y el texto acumulado
CU-04 Editar el texto	Cualquiera	Copia el texto, borra la última seña o limpia todo
CU-05 Ajustar opciones	Cualquiera	Cambia la frecuencia de envío, oculta el esqueleto o cambia el tema visual
CU-06 Detener	Cualquiera	Detiene la cámara y cierra la conexión
CU-07 Entrenar y publicar un modelo	Desarrollador	Extrae características, entrena, evalúa y copia el modelo al servidor

8.4 Señas estáticas y dinámicas

Una decisión temprana del diseño fue tratar como problemas distintos el reconocimiento de configuraciones estáticas y el de señas con movimiento. El prototipo de escritorio que sirvió

de punto de partida intentaba representar ambos con un mismo esquema, lo que en la práctica impedía entrenar correctamente cualquiera de los dos.

- **Configuraciones estáticas** (como la mayoría de las letras del alfabeto manual): un único cuadro contiene toda la información. Cada muestra es un vector de 138 valores y el problema es una clasificación tabular. Se resuelve con un bosque aleatorio o un perceptrón multicapa, que se entrenan y comparan con el script `ml/train.py`.
- **Señas dinámicas** (como las de LSA64): la información está en la evolución temporal. Cada muestra es una secuencia de vectores de longitud variable y se resuelve con una BiLSTM (Capítulo 10).

El backend sirve uno u otro tipo de modelo según el archivo cargado, sin cambios en el resto del sistema (sección 11.6). El reconocimiento dinámico sobre LSA64 es el que está completo y evaluado. Para el estático existe el pipeline completo de captura y entrenamiento, y una prueba de concepto con dos letras capturadas por uno de los autores, pero no un conjunto de datos suficiente para reportar resultados (sección 9.8).

8.5 Arquitectura del sistema

La arquitectura se organiza en tres capas —cliente, backend y modelo—, más un módulo de código compartido. La Figura 4 presenta la vista general.

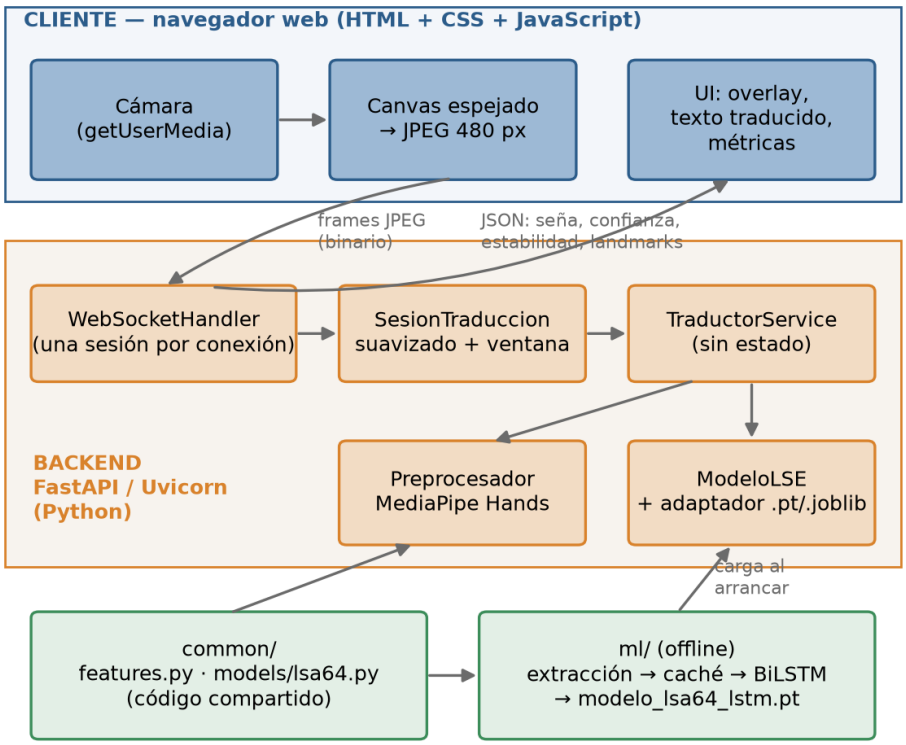


Figura *4. Arquitectura del sistema. El cliente envía cuadros JPEG por WebSocket; el backend extrae las manos, clasifica la secuencia y devuelve el resultado. El código de extracción de características y la definición de la red se comparten entre el backend y el pipeline de entrenamiento.*

- **Cliente** (`frontend/`). Una página web que captura el video, lo espeja y lo codifica como JPEG, lo envía al servidor, y presenta los resultados. No contiene lógica de reconocimiento.
- **Backend** (`backend/`). Un servicio FastAPI que recibe los cuadros por WebSocket, detecta las manos con MediaPipe, construye el vector de características, mantiene la ventana de cuadros de cada conexión, invoca al modelo y suaviza los resultados. Sirve además los archivos del cliente, de modo que todo el sistema es un único servicio.
- **Modelo** (`ml/`). Los scripts que extraen las características de LSA64, entrenan la red y la evalúan. Se ejecutan fuera de línea; su producto es un archivo de pesos que el backend carga al arrancar.
- **Código compartido** (`common/`). La extracción y normalización de características (`features.py`) y la definición de la red (`models/lsa64.py`).

8.5.1 El módulo compartido

El módulo `common/` no figuraba en el diseño inicial y es, probablemente, la decisión de arquitectura más importante del proyecto. En el prototipo de escritorio, la extracción de características estaba escrita dos veces: una en el script de captura y otra en el de reconocimiento. Si una de las dos copias se modifica y la otra no, el modelo recibe durante el uso vectores distintos de los que vio al entrenar. No se produce ningún error: el modelo simplemente predice mal, y el síntoma —baja exactitud— suele atribuirse al modelo o a la falta de datos, no al preprocesamiento.

Con un único módulo importado por ambos lados, ese tipo de divergencia es imposible por construcción. El módulo define además una **versión del esquema de características** (`FEATURE_VERSION`), que se guarda dentro de cada modelo entrenado y que el backend verifica al cargarlo: si no coincide con la versión vigente, el servidor se niega a usar el modelo y lo informa en lugar de servir predicciones sin sentido. La sección 9.7 muestra que el módulo compartido, aun así, no alcanzó para prevenir todos los desajustes.

8.6 Modelo de clases del backend

La Tabla 9 presenta las clases del backend y sus responsabilidades. El diseño parte de un diagrama de clases elaborado al inicio del proyecto, con dos modificaciones que se justifican a continuación.

Tabla *9.* *Clases del backend y sus responsabilidades.*

Clase	Archivo	Responsabilidad
`Frame`	`backend/app/frame.py`	Encapsula los bytes recibidos y los decodifica a una matriz de imagen
`Preprocesador`	`backend/app/preprocesador.py`	Ejecuta MediaPipe sobre la imagen y construye el vector de características

Clase	Archivo	Responsabilidad
`ModeloLSE`	`backend/app/modelo_lse.py`	Carga el modelo, valida su compatibilidad y produce predicciones
`ModeloAdaptador` y subclases	`backend/app/modelos/`	Encapsulan el formato del modelo (scikit-learn o PyTorch)
`Prediccion`	`backend/app/prediccion.py`	Objeto de valor con la etiqueta, la confianza y la estabilidad
`TraductorService`	`backend/app/traductor_service.py`	Orquesta preprocesamiento y modelo; no guarda estado de clientes
`SesionTraduccion`	`backend/app/traductor_service.py`	Estado de una conexión: suavizado, ventana de cuadros e historial
`WebSocketHandler`	`backend/app/websocket_handler.py`	Acepta conexiones y ejecuta el ciclo de recepción y envío

Primera modificación: la clase `SesionTraduccion`. En el diagrama original, `TraductorService` guardaba el historial de predicciones como atributo propio. Pero el servicio se construye una única vez al arrancar el servidor y se comparte entre todas las conexiones, así que ese atributo era, en la práctica, estado global: con dos usuarios conectados, las predicciones de uno se mezclaban con las del otro. La solución fue separar lo que es de la aplicación —el preprocesador y el modelo, que pueden compartirse— de lo que es de cada conversación, que pasó a una clase nueva instanciada por conexión. El caso ilustra un límite de los diagramas de clases: son correctos en lo estructural, pero no distinguen entre objetos de aplicación y objetos de sesión, y esa diferencia es la que determina el comportamiento con varios usuarios.

Segunda modificación: la cámara vive en el cliente. El diagrama original incluía una clase `CamaraCaptura` del lado del servidor. Sin embargo, la cámara pertenece al dispositivo del usuario y sólo es accesible mediante las interfaces del navegador. La captura se implementó en JavaScript, en el cliente, y lo único que viaja al servidor son los cuadros ya capturados.

8.7 Dinámica de una traducción

La Figura 5 muestra el diagrama de secuencia de una sesión de traducción. Una vez abierta la conexión, el ciclo se repite a la frecuencia configurada en el cliente: el navegador envía un cuadro, el backend lo procesa fuera del bucle de eventos, y el resultado vuelve como un mensaje JSON que incluye la seña, su confianza, la estabilidad, los puntos de las manos para el dibujo y el tiempo de procesamiento del servidor.

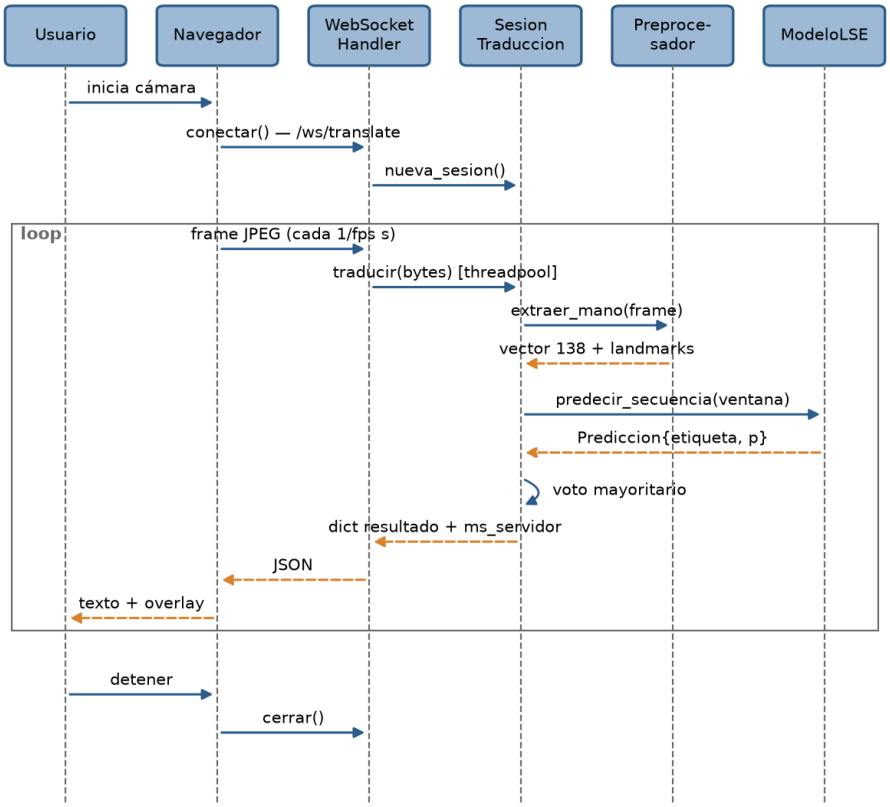


Figura *5. Diagrama de secuencia de una sesión de traducción.*

8.8 Decisiones de diseño

8.8.1 Dónde ejecutar la detección de manos

La arquitectura implementada envía cuadros completos al servidor y ejecuta MediaPipe allí. Existe una alternativa: MediaPipe tiene una versión para navegador (`@mediapipe/tasks-vision`, en JavaScript y WebAssembly), que permitiría detectar las manos en el dispositivo del usuario y enviar al servidor sólo los vectores de características. La Tabla 10 compara ambas opciones.

Tabla *10. Detección de manos en el servidor frente a detección en el cliente.*

Criterio	En el servidor (implementado)	En el cliente (alternativa)
Datos por cuadro	Imagen JPEG (decenas de KB)	Vector de 138 valores (menos de 1 KB)
Cómputo del servidor	Alto: MediaPipe por cada cuadro de cada usuario	Bajo: sólo la red neuronal
Requisitos del dispositivo	Mínimos	Debe ejecutar MediaPipe con fluidez
Garantía de preprocesamiento idéntico	Total: el mismo código Python que en el entrenamiento	Requiere una segunda implementación en JavaScript
Adecuación a la capa gratuita	Limitada: pocos usuarios simultáneos	Mejor

Se eligió la detección en el servidor porque garantiza que el preprocesamiento sea idéntico al del entrenamiento —el requisito RNF-08—, que es la fuente de errores más grave identificada en el proyecto. La alternativa es más eficiente y escalable, pero duplicaría la extracción de características en dos lenguajes, precisamente lo que el módulo compartido busca evitar. Queda como mejora futura, condicionada a una verificación automática de que ambas implementaciones producen los mismos vectores.

8.8.2 Cliente sin marco de trabajo

El anteproyecto preveía desarrollar el cliente con React. Durante el diseño se optó por HTML, CSS y JavaScript sin marco de trabajo, separados en tres archivos por responsabilidad. La interfaz consiste en una única pantalla con un video, un panel de resultados y algunos controles: un marco de trabajo agregaría un proceso de compilación, dependencias y un segundo servidor durante el desarrollo, sin un beneficio proporcional. Además, al no requerir compilación, el backend puede servir los archivos del cliente directamente y todo el sistema se despliega como un único servicio, lo que ahorra recursos en la capa gratuita. **PENDIENTE: validar con el tutor este cambio respecto del anteproyecto.**

8.8.3 Otras decisiones

- **PyTorch en lugar de TensorFlow.** El anteproyecto mencionaba ambos. Se usó PyTorch por su soporte directo para secuencias de longitud variable empaquetadas (sección 3.7.1) y porque permite servir el modelo con la versión para CPU de la biblioteca.
- **Scripts en lugar de cuadernos Jupyter.** El entrenamiento se implementó como scripts de Python con parámetros de línea de comandos, en lugar de cuadernos. Los scripts se versionan mejor, se prueban con pytest y se ejecutan de forma reproducible, requisitos que surgieron al detectar los problemas de reproducibilidad de la sección 10.4.
- **Entrenamiento local en lugar de Google Colab.** La red tiene menos de 300 000 parámetros y se entrena en segundos en una CPU. El paso costoso es la extracción de landmarks con MediaPipe, que corre en CPU y se paraleliza por procesos: con los 8 núcleos de un equipo de escritorio tarda unos 47 minutos, mientras que en la capa gratuita de Colab, con dos núcleos virtuales, tardaría varias horas. Una GPU no acelera este paso.

Capítulo 9 — Preparación de los datos y extracción de características

Este capítulo describe cómo se transforman los videos de LSA64 en las secuencias de vectores que recibe la red neuronal: la organización del conjunto de datos, el diseño del vector de características, la convención de orientación de la imagen, el proceso de extracción en dos etapas y el análisis de la calidad de la detección.

9.1 El conjunto LSA64

Las características generales de LSA64 se presentaron en la sección 5.2.1. Para el procesamiento interesan dos aspectos de su organización.

Nomenclatura de los archivos. Cada video se nombra con el patrón `CCC_SSS_RRR.mp4`, donde CCC es el número de seña (001 a 064), SSS el de la persona (001 a 010) y RRR el de la repetición (001 a 005). La etiqueta y la persona de cada video se obtienen, por lo tanto, del nombre del archivo. La función que realiza esa lectura (`parse_lsa64_stem`) fue el origen del error más grave del proyecto, descripto en la sección 9.11.

Glosas. La lista oficial de señas se publica sólo en inglés, aunque el conjunto sea de LSA. Para la aplicación y para este documento se tradujeron las glosas al castellano (ver el Anexo A), con dos criterios que van más allá de la traducción literal:

- **Localismos rioplatenses.** La seña 020, publicada como *Sweet milk*, es "Dulce de leche", y la 045, *Barbecue*, es "Asado".
- **Orden alfabético de la lista original.** Los tramos 029 a 043, 044 a 049 y 050 a 064 de la lista están ordenados alfabéticamente en castellano —no en inglés—, lo que indica que la lista original estaba en castellano y lo publicado es su traducción. Ese orden permitió resolver traducciones ambiguas: la seña 037, *Ship*, se ubica entre "Música" y "Ninguno", por lo que su glosa es "Nave" y no "Barco"; la 051, *Thanks*, se ubica entre "Aceptar" y "Apagar", por lo que es "Agradecer" y no "Gracias".

Además, se corrigió un error de una de las copias del conjunto disponibles en internet, que glosaba las señas 002 y 008 ambas como *Red*; la página del mantenedor del conjunto indica que la 008 es *Pink* ("Rosa"). Las glosas se mantienen en un archivo de configuración (`ml/lsa64_labels.json`) separado del entrenamiento, de modo que corregir una glosa no requiere volver a extraer las características (sección 10.7).

9.2 Muestreo temporal

Los videos de LSA64 tienen 60 cuadros por segundo, una resolución temporal mayor que la necesaria para describir el movimiento de una seña. La extracción toma uno de cada dos cuadros (`frame_step = 2`), lo que equivale a 30 cuadros por segundo, y limita cada secuencia a un máximo de 120 cuadros muestreados, es decir, 4 segundos; las secuencias más largas se truncan en ese punto.

Este muestreo tiene una consecuencia para el uso en tiempo real que se discute en la sección 14.8: el modelo aprende las señas a 30 cuadros por segundo, mientras que el cliente web envía, por defecto, 8 cuadros por segundo.

9.3 El vector de características

Cada cuadro se representa con un vector de **138 valores**. La Figura 6 muestra su estructura: dos bloques de 69 valores, uno para cada mano, en posiciones fijas.

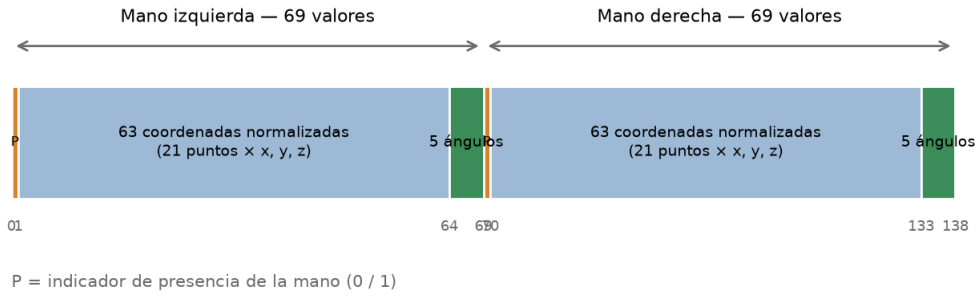


Figura *6. Estructura del vector de características (versión v1). Cada mano ocupa un bloque fijo de 69 valores: un indicador de presencia, las 63 coordenadas normalizadas de sus 21 puntos y los 5 ángulos de flexión de los dedos.*

Cada bloque contiene:

- **1 indicador de presencia** (0 o 1), que vale 1 si la mano fue detectada en el cuadro.
- **63 coordenadas normalizadas**: las coordenadas x, y, z de los 21 puntos, después de la normalización descripta en la sección 9.5.
- **5 ángulos de flexión**, uno por dedo.

Si una mano no se detecta, su bloque completo vale cero, incluido el indicador de presencia. Este indicador permite al modelo distinguir entre "la mano no está" y "la mano está con sus puntos en el origen de coordenadas", dos situaciones que sin él serían idénticas.

9.4 Posiciones fijas por mano

El prototipo de escritorio tomaba "la primera mano detectada" en cada cuadro. MediaPipe no garantiza ningún orden entre las manos que detecta, de modo que, en una seña bimanual, la

mano derecha podía aparecer primero en un cuadro y segunda en el siguiente. El mismo gesto producía vectores distintos según un detalle irrelevante del detector, y las señas bimanuales —22 de las 64 de LSA64— no podían reconocerse.

El sistema asigna cada mano a su bloque según la **lateralidad** que informa MediaPipe: el primer bloque es siempre la mano izquierda y el segundo, la derecha. Así, cada posición del vector tiene un significado estable en todos los cuadros y en todas las muestras.

9.5 Normalización

Las coordenadas que entrega MediaPipe están referidas a la imagen: una misma configuración de la mano produce coordenadas distintas si la mano está a la izquierda o a la derecha del encuadre, o más cerca o más lejos de la cámara. Para que el modelo aprenda la forma de la mano y no su posición en la imagen, las coordenadas se normalizan en dos pasos (función `normalize_landmarks` del módulo compartido):

1. **Traslación.** Se resta a todos los puntos la posición de la muñeca (punto 0), que pasa a ser el origen.
2. **Escala.** Se dividen todas las coordenadas por la distancia entre la muñeca y la base del dedo medio (punto 9), una medida de referencia del tamaño de la propia mano.

Después de la normalización, la misma configuración genera vectores prácticamente idénticos sin importar la distancia a la cámara ni el lugar del encuadre. Las pruebas automatizadas del módulo verifican estas dos invarianzas (sección 13.1).

Los **ángulos de flexión** se calculan sobre los puntos normalizados. Para cada dedo se toman tres puntos —la muñeca, la articulación intermedia y la punta— y se calcula el ángulo en la articulación intermedia a partir del producto escalar de los dos vectores que la unen con los extremos. Un dedo extendido produce un ángulo cercano a 180 grados (π radianes) y uno flexionado, un ángulo menor. Estos cinco valores resumen en forma explícita una información que la red podría deducir de las coordenadas, pero que así recibe directamente.

9.6 Variante con ubicación (v2)

La normalización respecto de la muñeca tiene un costo: al centrar cada mano en su propia muñeca, el vector conserva la **forma** de la mano pero descarta **dónde** está. Como se vio en la sección 1.3.1, la ubicación es uno de los parámetros formales de una seña, y dos señas con igual configuración y movimiento, realizadas a distinta altura, resultan indistinguibles en la versión v1 del vector.

Para medir el efecto de esa pérdida se implementó una variante experimental, **v2**, que agrega a cada bloque la posición de la muñeca sin normalizar: tres valores por mano, con lo que

el vector pasa de 138 a 144 valores. Esta variante tiene una limitación conocida: la posición queda referida al encuadre de la cámara y no al cuerpo de la persona. En LSA64 eso es aceptable porque la cámara está fija y las personas se ubicaron sobre marcas en el piso, pero con una cámara web cada usuario se ubica distinto. Una versión utilizable en producción debería expresar la posición respecto de un punto del cuerpo —por ejemplo, los hombros o la nariz, obtenidos con el modelo de pose de MediaPipe—.

La versión del esquema forma parte de los metadatos de cada modelo, y el backend actual rechaza explícitamente un modelo v2, porque no genera esa variante. El resultado del experimento se presenta en la sección 14.6.

9.7 Convención de orientación de la imagen

La sección 4.4 explicó que MediaPipe deduce la lateralidad de cada mano a partir de la imagen, y que una imagen espejada intercambia izquierda y derecha. Como el vector tiene posiciones fijas por lateralidad, si el entrenamiento y la inferencia procesan imágenes con orientación opuesta, cada mano cae en el bloque contrario al aprendido.

Este desajuste se produjo efectivamente durante el desarrollo, en dos oportunidades:

1. El script de captura de datos propios espejaba la imagen antes de procesarla, como es habitual con las cámaras web. El cliente web, en cambio, mostraba el video espejado mediante una propiedad de CSS (`transform: scaleX(-1)`), pero esa propiedad sólo afecta la presentación: el cuadro que se dibujaba en el canvas y se enviaba al servidor salía **sin espejar**.
2. El pipeline de LSA64, desarrollado después, procesaba los videos sin espejar, mientras que la captura y la inferencia trabajaban con imágenes espejadas. Como 42 de las 64 señas se hacen con una sola mano, el modelo entrenado así predecía señas incorrectas al usarse desde el cliente.

El aspecto más llamativo es que **ningún control automático podía detectarlo**: los vectores tenían la longitud correcta, la versión del esquema coincidía y no se producía ninguna excepción. El único síntoma era una exactitud baja sin explicación aparente, que es el síntoma que uno tiende a atribuir al modelo o a la falta de datos.

La solución fue fijar una convención única y explícita en el módulo compartido, la constante `ESPEJADO_CANONICO`: **toda imagen se procesa espejada**. Se eligió esa orientación porque mantenía válidos los datos ya capturados y porque es la natural para el usuario, que se ve como en un espejo. El cliente espeja el cuadro en el canvas antes de codificarlo, y el pipeline de LSA64 espeja cada cuadro antes de pasarlo a MediaPipe. Además, la orientación pasó a formar parte de los metadatos del caché de características (sección 9.9), de modo que un caché generado con otra orientación se invalida automáticamente.

El episodio muestra un límite de la estrategia del módulo compartido: evita la duplicación del **código** de extracción, pero no protege contra diferencias en cómo se **prepara la entrada** antes de llegar a ese código. La mitigación fue documentar el contrato junto al esquema que protege y convertirlo, donde fue posible, en un dato verificable.

9.8 Datos propios para configuraciones estáticas

Para el reconocimiento de configuraciones estáticas se desarrolló un script de captura (`ml/capture_dataset.py`) que abre la cámara, detecta las manos y guarda cada muestra como un archivo JSON con su vector de características y sus metadatos: etiqueta, persona, sesión y condición de iluminación. El script de entrenamiento correspondiente (`ml/train.py`) agrupa las muestras por sesión al dividirlas, de modo que todas las muestras de una misma sesión de captura queden del mismo lado, y aborta con un mensaje explicativo si alguna clase tiene una única sesión, porque en ese caso el reparto dejaría clases enteras fuera del entrenamiento.

Como prueba de concepto se capturaron 120 muestras de dos letras (A y C), en dos sesiones, realizadas por uno de los autores. Esta cantidad permite verificar que el pipeline funciona de punta a punta, pero no constituye un conjunto de datos del que puedan extraerse conclusiones. **PENDIENTE: según la decisión D1 del plan, completar con la captura del abecedario (varias personas y sesiones, con consentimiento informado) o incorporar LSA16, y reportar los resultados en el Capítulo 14.**

9.9 Extracción en dos etapas

Procesar los 3200 videos de LSA64 implica ejecutar MediaPipe sobre más de 130 000 cuadros de alta resolución. En las primeras versiones, cada entrenamiento repetía esa extracción, de modo que probar un cambio en la red —que entrena en segundos— costaba casi una hora de espera. El pipeline se reorganizó en dos etapas independientes, que muestra la Figura 7.

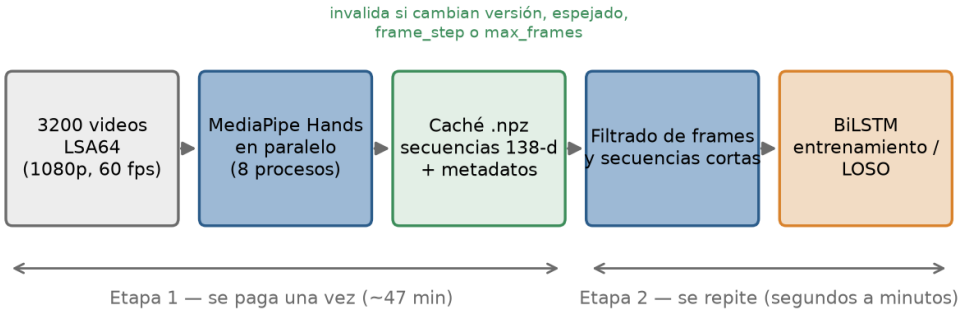


Figura *7. Pipeline de datos en dos etapas. La extracción con MediaPipe se ejecuta una vez y se guarda en un caché; el entrenamiento y la evaluación leen el caché tantas veces como sea necesario.*

Etapa 1: extracción (`ml/extract_features.py`). Recorre los videos, extrae la secuencia de vectores de cada uno y guarda el resultado en un único archivo comprimido de NumPy (`.npz`), junto con la etiqueta y la persona de cada secuencia. La extracción se paraleliza en varios procesos, cada uno con su propia instancia de MediaPipe; con 8 procesos en un procesador de 8 núcleos tardó 47 minutos.

Etapa 2: entrenamiento (`ml/train_lsa64.py`, `ml/evaluate_loso.py`). Lee el caché y entrena o evalúa el modelo. Un entrenamiento completo tarda segundos; una validación LOSO de 10 particiones, alrededor de una hora.

9.9.1 Metadatos y compatibilidad del caché

Un caché es válido sólo para la configuración con que fue generado. Si se cambia el muestreo temporal, el esquema de características o la orientación, un caché anterior produciría datos incompatibles. Para evitar errores silenciosos, el caché guarda sus parámetros de generación —versión del formato, versión y longitud del vector, `frame_step`, `max_frames` y orientación— y, antes de usarlo, el entrenamiento los compara con la configuración pedida. Ante cualquier diferencia, el caché se rechaza con un mensaje que indica qué parámetro no coincide.

9.10 Calidad de la detección

No todos los cuadros tienen manos detectadas. La extracción guarda siempre la secuencia completa, incluidos los cuadros sin manos, y el filtrado se decide al entrenar. En la configuración oficial se descartan los cuadros donde no se detectó ninguna mano y, después, las secuencias que quedan con menos de 8 cuadros. Con ese criterio, de los 3200 videos se conservan **2862 secuencias**: 338 videos (10,6 %) se descartan por no tener suficientes cuadros con manos detectadas.

La pérdida no es uniforme. Por persona, la proporción de cuadros con manos detectadas —que se denomina **retención**— varía entre 58,5 % y 75,7 %. Por seña, la diferencia es aún mayor: algunas clases conservan casi todas sus muestras y otras pierden más de la mitad, como "Paciencia", que conserva 22 de sus 50 videos, u "Hombre", que conserva 23. Son señas en las que MediaPipe no logra detectar las manos con regularidad, posiblemente por oclusiones, movimientos rápidos o los guantes de color usados en la grabación (sección 4.5). La sección 14.5 muestra que la retención está asociada con la exactitud del modelo.

[[PENDIENTE (tarea M3 del plan): probar umbrales de detección más bajos y el modo VIDEO de MediaPipe, que usa seguimiento entre cuadros, para medir cuántas secuencias se recuperan.]]

9.11 Errores detectados en el pipeline de datos

Durante la revisión del pipeline de LSA64 se detectaron errores que invalidaban todas las mediciones previas. Se documentan aquí porque constituyen uno de los aprendizajes centrales del trabajo:

1. **Etiquetado incorrecto.** La función que obtenía la etiqueta descartaba el primer campo del nombre de archivo —el número de seña— y armaba la etiqueta con la persona y la repetición. El modelo se entrenaba con 50 clases artificiales en lugar de las 64 señas reales, y cualquier exactitud medida en esas condiciones no medía nada.
2. **Fuga de datos.** Por el mismo motivo, la persona de cada video no se identificaba y el reparto de los datos se hacía por archivo: la misma persona aparecía en entrenamiento y en prueba, lo que producía una evaluación dependiente del sujeto sin advertirlo (sección 2.4.2).
3. **Orientación inconsistente**, descripta en la sección 9.7.
4. **Entrenamiento no reproducible**, descripto en la sección 10.4.
5. **Ausencia de caché**, que multiplicaba el costo de cada experimento, resuelta con la extracción en dos etapas.

Cada uno de estos errores tiene hoy una prueba automatizada que lo reproduce (sección 13.1).

Capítulo 10 — Modelo de reconocimiento y entrenamiento

Este capítulo describe la red neuronal que clasifica las señas, el procedimiento de entrenamiento, los mecanismos que garantizan la validez y la reproducibilidad de las mediciones, y la implementación de la validación leave-one-subject-out.

10.1 Arquitectura de la red

El clasificador es una red **BiLSTM** de una capa, implementada en PyTorch en el módulo compartido (`common/models/lsa64.py`), de modo que la misma definición se usa para entrenar y para inferir. La Figura 8 muestra su estructura:

1. **Entrada**: una secuencia de T vectores de 138 características, con T variable entre 8 y 120.
2. **Capa LSTM bidireccional** con 128 unidades por dirección. Las secuencias de un lote se empaquetan con su longitud real, de modo que la recurrencia se detiene en el último cuadro válido de cada una (sección 3.7.1).
3. **Concatenación** del estado oculto final de la dirección hacia adelante y de la dirección hacia atrás, que da un vector de 256 valores que resume la secuencia completa.
4. **Dropout** con probabilidad 0,2.
5. **Capa lineal** de 256 a 64 valores, uno por seña, seguida de softmax para obtener las probabilidades.

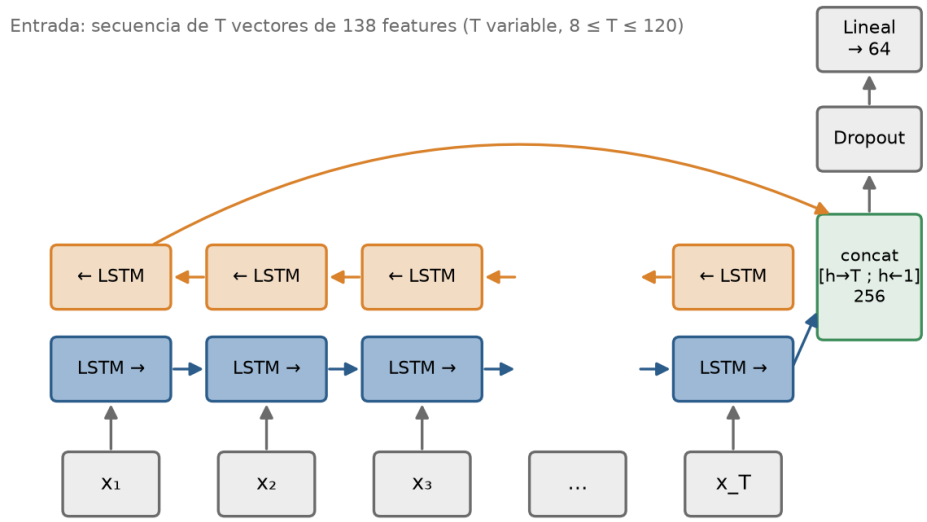


Figura *8. Arquitectura de la red BiLSTM. Cada cuadro de la secuencia alimenta dos LSTM que la recorren en sentidos opuestos; sus estados finales se concatenan y una capa lineal produce una puntuación por seña.*

La red tiene **290 880 parámetros**: 274 432 en la capa LSTM bidireccional y 16 448 en la capa lineal. Es un modelo deliberadamente pequeño: con unas 2000 secuencias de entrenamiento por partición, una red más grande sólo aumentaría el sobreajuste. El Anexo B detalla las dimensiones de cada capa.

10.2 Reparto de los datos

Se utilizan dos esquemas de reparto, con propósitos distintos.

Reparto único, para entrenar el modelo que se sirve en la aplicación. Las 10 personas se ordenan al azar con una semilla fija y se asignan 7 al entrenamiento, 1 a la validación y 2 a la prueba. El reparto es siempre **por persona**: todas las secuencias de una persona quedan del mismo lado. Con la semilla 42, el entrenamiento usa 2014 secuencias, la validación 256 y la prueba 592.

Validación leave-one-subject-out, para medir el desempeño (sección 10.6). Es el protocolo con el que se reportan los resultados.

10.2.1 Verificación de fuga

Después del error de etiquetado de la sección 9.11, que mezclaba a las personas entre los conjuntos sin que nada lo advirtiera, se incorporó una verificación explícita: antes de entrenar, el sistema calcula el conjunto de personas de cada partición y comprueba que sean disjuntos. Si encuentra una persona en más de una partición, lo informa en la consola y lo registra en el reporte de resultados, en el campo `fugas_detectadas`. Una exactitud medida sobre personas ya vistas en el entrenamiento no es un resultado válido para esta tesina, y el reporte lo deja asentado.

10.3 Procedimiento de entrenamiento

El entrenamiento minimiza la entropía cruzada con el optimizador AdamW. La Tabla 11 resume los hiperparámetros de la configuración de referencia.

Tabla *11.* *Hiperparámetros de la configuración de referencia.*

Hiperparámetro	Valor	Comentario
Unidades LSTM por dirección	128	Estado concatenado de 256
Dropout antes de la capa lineal	0,2	
Optimizador	AdamW	Con penalización de pesos 0 equivale a Adam
Tasa de aprendizaje	0,001	
Penalización de pesos	0	
Tamaño de lote	8	
Épocas máximas	40 (reparto único) / 60 (LOSO)	
Paciencia de la detención temprana	8 épocas	
Muestreo temporal (`frame_step`)	2	30 cuadros por segundo efectivos
Longitud máxima de la secuencia	120 cuadros	
Longitud mínima de la secuencia	8 cuadros	Las más cortas se descartan
Semilla	42	

Al final de cada época se calcula la exactitud sobre el conjunto de validación. Cuando mejora, se guarda una copia de los pesos; si no mejora durante 8 épocas seguidas, el entrenamiento se detiene y se restauran los mejores pesos guardados. En el reparto único, la mejor época fue la 11 y el entrenamiento se detuvo en la 19 (sección 14.3).

10.4 Reproducibilidad

En una primera versión, la semilla sólo determinaba el reparto de los datos. La inicialización de los pesos, las máscaras de dropout y el orden de los lotes quedaban librados al azar, y dos ejecuciones sobre los mismos datos diferían en varios puntos de exactitud. En esas condiciones es imposible saber si un cambio mejora el modelo o si la diferencia es ruido.

La función `set_seed` fija las semillas de Python, NumPy y PyTorch, y el cargador de datos usa un generador propio también inicializado con la semilla, porque el desordenamiento de los lotes no depende de la semilla global. Con esto, el entrenamiento es determinista: misma configuración, mismo resultado, época por época. Se comprobó además que filtrar los cuadros vacíos de un caché de secuencias completas reconstruye bit a bit el caché anterior —las 2862 secuencias—, y que el resultado del LOSO se reproduce exactamente incluso después de repetir la extracción completa con MediaPipe.

10.5 Regularización y aumentación de datos

Para atacar el sobreajuste observado (sección 14.3) se incorporaron al entrenamiento las técnicas de la sección 3.4, todas como opciones de línea de comandos **desactivadas por defecto**, de modo que una ejecución sin opciones reproduzca exactamente la configuración de referencia:

- `--dropout p`: probabilidad de dropout antes de la capa lineal.
- `--weight-decay λ`: penalización de pesos, aplicada en forma desacoplada por AdamW.
- `--aug-frame-drop p`: descarta cada cuadro con probabilidad p. Simula a una persona con mala detección de manos, que es justamente el caso en que el modelo falla.
- `--aug-time-scale s`: cambia la duración de la secuencia por un factor aleatorio entre 1 − s y 1 + s, remuestreando por vecino más cercano. Simula señas más rápidas o más lentas.
- `--aug-noise σ`: suma ruido gaussiano a las coordenadas normalizadas. Sólo se aplica a los cuadros con manos detectadas: aplicarlo a un cuadro vacío crearía una "mano fantasma" en el origen.

La aumentación se aplica en cada época con una variante aleatoria distinta de cada secuencia, y sólo sobre el conjunto de entrenamiento: nunca sobre validación ni prueba, que deben medir el desempeño sobre datos reales. Los resultados de estas técnicas se presentan en la sección 14.6.

10.6 Validación leave-one-subject-out

El script `ml/evaluate_loso.py` implementa el protocolo LOSO sobre el mismo caché y con el mismo ciclo de entrenamiento que el reparto único, de modo que los resultados son comparables. Para cada una de las 10 personas:

- La persona es el conjunto de **prueba**.
- La persona siguiente, en orden cíclico, es el conjunto de **validación**, usado para la detención temprana.
- Las 8 personas restantes son el conjunto de **entrenamiento**.

La asignación es determinista: no depende de ninguna semilla y cada persona cumple una vez el rol de prueba y una vez el de validación. El script entrena los 10 modelos y genera un reporte JSON con la exactitud de cada partición, la media y el desvío, la exactitud por clase y por persona, y las predicciones de las 10 particiones agrupadas. Como cada secuencia del conjunto es predicha exactamente una vez por un modelo que nunca vio a esa persona, esas predicciones agrupadas permiten construir una matriz de confusión de todo el conjunto de datos (sección 14.4).

El LOSO no guarda modelos: es un protocolo de medición. El modelo que se sirve en la aplicación es el del reparto único.

10.6.1 Evaluación parcial sobre personas difíciles

Una validación LOSO completa tarda alrededor de una hora, demasiado para explorar muchas variantes. Por otro lado, el reparto único no sirve para elegir entre variantes, porque las dos personas con peor desempeño —la 03 y la 08— caen en su conjunto de entrenamiento y el reparto es ciego a ellas. Se agregó por eso la opción `--solo-sujetos`, que ejecuta sólo las particiones de las personas indicadas. Evaluar una variante sobre las personas 03 y 08 tarda unos diez minutos y permite descartar rápidamente las que no mejoran donde el modelo falla. Las variantes prometedoras deben confirmarse luego con el LOSO completo.

10.7 Artefactos del entrenamiento

El entrenamiento produce dos archivos:

- **El modelo** (`backend/models/modelo_lsa64_lstm.pt`): un *checkpoint* de PyTorch que contiene los pesos y los metadatos necesarios para usarlo: la lista de clases, la versión y la longitud del vector de características, el tamaño de la capa oculta, el muestreo temporal y la longitud mínima de secuencia. El backend no necesita ningún dato externo para reconstruir la red.
- **El resumen** (`backend/models/modelo_lsa64_lstm.json`): los hiperparámetros, el reparto de personas, la exactitud de validación y de prueba, la exactitud por clase y la historia de la pérdida y la exactitud por época.

10.7.1 Glosas y orden de las clases

La traducción de las etiquetas canónicas (`clase_01` a `clase_64`) a las glosas en castellano produjo un error instructivo. La primera implementación traducía las etiquetas antes de entrenar. Pero el codificador de etiquetas asigna los índices de las clases en orden alfabético, y con las glosas el orden cambia —"Aceptar" pasa a ser la primera clase—, lo que altera la correspondencia entre clases y neuronas de salida y, con ella, el resultado del entrenamiento. Se midió una diferencia de exactitud de 0,8125 a 0,8159, con la mejor época pasando de la 11 a la 32, sobre los mismos datos. Una opción de presentación no puede modificar el resultado de un experimento.

La corrección consistió en entrenar siempre con las etiquetas canónicas, cuyo orden alfabético coincide con el numérico, y aplicar las glosas al final, sólo sobre los nombres que se guardan en el modelo y en el reporte. La prueba de que la corrección es correcta es que, con las glosas aplicadas, el modelo reentrenado reproduce exactamente el resultado de referencia: exactitud de prueba de 0,8125, mejor época 11, detención en la 19.

Capítulo 11 — Backend de inferencia

El backend recibe los cuadros de video, los convierte en predicciones y las devuelve al cliente. Este capítulo describe su organización, su ciclo de vida, el manejo de conexiones concurrentes, el suavizado temporal de las predicciones y los mecanismos que impiden servir un modelo incompatible.

11.1 Organización del código

El backend es una aplicación FastAPI ubicada en `backend/app/`, cuyas clases se presentaron en la sección 8.6. Su punto de entrada, `main.py`, construye el grafo de dependencias al arrancar: crea el `Preprocesador` y el `ModeloLSE`, los combina en un `TraductorService` y entrega este último al `WebSocketHandler`. Ninguna clase crea por su cuenta los objetos que necesita; los recibe ya construidos, lo que permite reemplazarlos por dobles de prueba en las pruebas automatizadas (sección 13.1).

11.2 Arranque y servicios expuestos

11.2.1 Selección del modelo

Al arrancar, el backend decide qué modelo cargar con el siguiente orden de prioridad:

1. El archivo indicado en la variable de entorno `LSA_MODEL_PATH`, si está definida. Permite probar otro modelo sin modificar el código.
2. La BiLSTM de LSA64 (`modelo_lsa64_lstm.pt`), si existe.
3. El modelo de configuraciones estáticas (`modelo_lse.joblib`).

11.2.2 Arranque tolerante a fallas

En un repositorio recién clonado pueden faltar el archivo de pesos de MediaPipe —que se descarga aparte— o el modelo entrenado. Una primera versión del backend fallaba al arrancar en esos casos, con un error poco informativo. La versión actual captura esas situaciones, registra un aviso que indica el comando que hay que ejecutar para resolverlas, y arranca igual. Los servicios que dependen de los componentes faltantes responden con un mensaje de error explicativo en lugar de fallar. La inicialización se realiza en el manejador del ciclo de vida de la aplicación (`lifespan`), que además libera los recursos nativos de MediaPipe al apagar el servidor.

11.2.3 Servicios

El backend expone dos servicios HTTP y un canal WebSocket. FastAPI genera automáticamente su documentación navegable, que muestra la Figura 9.

- `GET /health`: estado del servicio. Informa si el preprocesador y el modelo están listos, qué modelo está cargado, sus clases, y si el sistema está en condiciones de traducir. Es el servicio que consultan las plataformas en la nube para saber si el contenedor está sano.
- `GET /model/info`: información que el cliente necesita para funcionar: las clases, el umbral de confianza, la versión de las características, si el modelo requiere una secuencia de cuadros y de qué longitud, y las conexiones del esqueleto de la mano para dibujarlo. El esqueleto se sirve desde el backend, en lugar de estar escrito también en el JavaScript, por la misma razón por la que existe el módulo compartido: una única definición, sin copias que puedan quedar desactualizadas.
- `WS /ws/translate`: el canal de traducción, descripto a continuación.

Además, el backend sirve los archivos estáticos del cliente en la raíz del sitio, de modo que todo el sistema funciona como un único servicio.

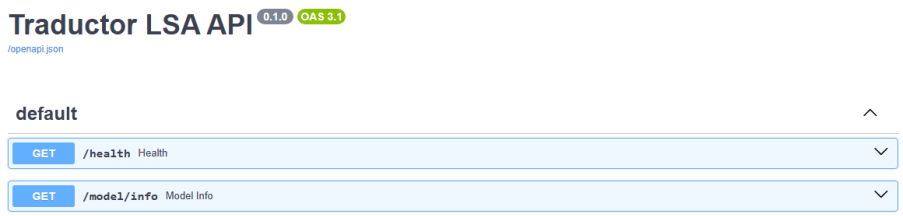


Figura *9. Documentación de la API generada automáticamente por FastAPI.*

11.3 El canal WebSocket y la concurrencia

Cuando un cliente se conecta a `/ws/translate`, el `WebSocketHandler` acepta la conexión y crea una `SesionTraduccion` exclusiva para ella. Luego entra en un ciclo: espera un mensaje binario con un cuadro JPEG, lo traduce y responde con un mensaje de texto en formato JSON. El ciclo termina cuando el cliente se desconecta, momento en que la sesión se descarta.

Como se explicó en la sección 6.5.2, el procesamiento de un cuadro es cómputo sincrónico y pesado. Si se ejecutara directamente en el manejador asincrónico, bloquearía el bucle de eventos y serializaría a todos los usuarios conectados. Por eso la traducción se delega a un hilo auxiliar mediante `run_in_threadpool`:

```
resultado = await run_in_threadpool(sesion.traducir, datos)
```

El detector de MediaPipe, en cambio, se comparte entre todas las conexiones —crear uno por conexión multiplicaría el uso de memoria— y no admite llamadas simultáneas desde varios hilos. El `Preprocesador` protege su uso con un cerrojo (`threading.Lock`). El resultado es que la detección de manos de distintos usuarios se ejecuta de a una por vez, pero el resto del trabajo

—decodificación, red neuronal, comunicación— avanza en paralelo, y una conexión lenta no bloquea a las demás.

Si el procesamiento de un cuadro falla —por ejemplo, porque llegan bytes que no son una imagen válida—, el error se registra y se devuelve al cliente como un mensaje con un campo `error`, sin cerrar la conexión.

11.4 La sesión de traducción

`SesionTraduccion` contiene todo el estado de una conversación: el búfer de suavizado, la ventana de cuadros y el historial de señas reconocidas.

11.4.1 Formato de la respuesta

Por cada cuadro recibido, la sesión devuelve un mensaje como el siguiente:

```
{
  "seña": "Leche",
  "confianza": 0.9132,
  "estabilidad": 0.8,
  "valida": true,
  "landmarks": [{"mano": "Right", "puntos": [[0.4812, 0.6623], ...]}],
  "manos": 1,
  "ms_servidor": 41.27
}
```

Los puntos de las manos viajan sólo con sus coordenadas x e y, redondeadas a cuatro decimales: con dos manos son 84 números, menos de 1 KB, una fracción despreciable frente al cuadro JPEG que se envió. El campo `ms_servidor` es el tiempo de procesamiento medido en el servidor, que el cliente usa para separar el costo de cómputo del costo de red (sección 12.5).

11.4.2 Suavizado por voto mayoritario

Una predicción por cuadro, presentada directamente, parpadea: un cuadro borroso o una transición entre señas producen predicciones aisladas incorrectas. La sesión estabiliza el resultado con un **voto mayoritario** sobre las últimas 10 predicciones. La etiqueta que se informa es la más votada, y sólo se informa si el búfer tiene al menos 5 predicciones.

En la primera implementación, sólo se agregaban al búfer las predicciones válidas. El efecto era que el búfer nunca se vaciaba: cuando la persona bajaba las manos, el voto seguía devolviendo la última seña como válida indefinidamente. La corrección fue registrar también los cuadros sin una seña válida, como votos "vacíos". Así, cuando la persona deja de señar, los votos vacíos ganan la mayoría y el resultado se apaga solo.

11.4.3 Confianza y estabilidad

La versión inicial del suavizado reemplazaba la confianza del modelo por la proporción de votos de la etiqueta ganadora, de modo que el campo `confianza` significaba cosas distintas

según el punto del sistema en que se lo mirara, y ambos valores se comparaban contra el mismo umbral. Se separaron en dos campos:

- **Confianza**: la probabilidad que asigna el modelo a la seña, promediada sobre los cuadros que votaron por ella. Responde a la pregunta "¿qué tan seguro está el modelo?".
- **Estabilidad**: la proporción de votos de la seña ganadora en el búfer. Responde a "¿qué tan consistente fue la detección en el tiempo?".

Una predicción se considera **válida** sólo si ambos valores superan su umbral (0,6 en ambos casos). Además de ser más correcta, la separación permite distinguir dos situaciones con causas y soluciones distintas: un modelo que duda —confianza baja— y un modelo seguro cuya detección parpadea —estabilidad baja—.

11.5 La ventana temporal

Con un modelo de secuencias, una predicción requiere varios cuadros. La sesión acumula los vectores de los cuadros con manos detectadas en una ventana deslizante de longitud fija y, cuando la ventana se completa, la entrega al modelo; a partir de ahí, cada cuadro nuevo desplaza al más antiguo y produce una predicción. Mientras la ventana no está completa, la sesión devuelve una predicción vacía, pero con los puntos de las manos, para que el cliente pueda dibujar el esqueleto desde el primer cuadro.

Si un cuadro llega sin manos, la ventana se vacía: la secuencia acumulada dejó de ser continua y se considera que la seña terminó. La ventana, como el búfer de suavizado, es estado de una conversación y vive en la sesión; si estuviera en el servicio compartido, las señas de dos usuarios se mezclarían en la misma secuencia.

La longitud de la ventana se toma del propio modelo: es la longitud mínima de secuencia con que fue entrenado, 8 cuadros. Esta elección es la principal limitación conocida de la integración en tiempo real y se analiza en la sección 14.8: el modelo fue entrenado con señas completas muestreadas a 30 cuadros por segundo, y en la aplicación recibe ventanas de 8 cuadros a la frecuencia de envío del cliente.

11.6 Modelos intercambiables

`ModeloLSE` es la única clase del backend que conoce cómo se carga y se consulta un modelo, y delega los detalles de cada formato en un **adaptador**. Una fábrica elige el adaptador según la extensión del archivo:

- `SklearnJoblibAdapter`, para los modelos de scikit-learn (`.joblib`) de las configuraciones estáticas. Además de cargar el modelo, verifica que las columnas de probabilidades correspondan a las clases en el orden correcto: en scikit-learn, la columna i corresponde a `classes_[i]`, que no necesariamente es la clase i.

- `TorchCheckpointAdapter`, para la BiLSTM de PyTorch (`.pt`). Reconstruye la red con la definición del módulo compartido y los metadatos del checkpoint, y la pone en modo de evaluación (sin dropout).

Gracias a este diseño, migrar de un modelo estático a uno secuencial, o agregar un tercer formato, no requiere modificar el servicio, el manejador WebSocket ni el cliente.

11.7 Validación de compatibilidad

Al cargar un modelo, `ModeloLSE` verifica que su versión de características y la longitud de su vector coincidan con las que genera el backend. Si no coinciden, rechaza el modelo con un mensaje que explica la diferencia y cómo resolverla. Esta verificación existe porque el desajuste entre el preprocesamiento del entrenamiento y el de la inferencia no produce errores, sino predicciones incorrectas (sección 8.5.1).

La implementación de esta validación tuvo un error sutil, detectado y corregido con una prueba de regresión. En la primera versión, el modelo se asignaba al objeto antes de validarlo. Si la validación fallaba, la excepción se registraba, pero el modelo rechazado quedaba igualmente cargado: el servicio `/health` informaba que había un modelo listo y el sistema servía precisamente el modelo que acababa de rechazar. La corrección consiste en realizar todas las validaciones sobre variables locales y asignar el modelo al objeto sólo cuando todas pasaron.

Capítulo 12 — Cliente web

El cliente es la parte del sistema con la que interactúan las personas. Este capítulo describe su estructura, el ciclo de captura y envío de cuadros, la presentación de los resultados, las métricas de desempeño que calcula y las decisiones de accesibilidad.

12.1 Estructura

El cliente es una página web de una sola pantalla, separada en tres archivos por responsabilidad:

- `frontend/index.html`: la **estructura**, con marcado semántico (encabezado, secciones con título, controles agrupados) y los atributos de accesibilidad.
- `frontend/styles.css`: la **presentación**, organizada a partir de variables de diseño —colores, espaciados, tipografías— que definen un tema claro y uno oscuro.
- `frontend/app.js`: el **comportamiento**: acceso a la cámara, comunicación con el servidor, dibujo del esqueleto, métricas y edición del texto.

Una primera versión reunía todo en un único archivo HTML con estilos y código embebidos. La separación facilitó el mantenimiento y permite que cada archivo se lea de forma independiente.

La Figura 10 muestra la interfaz en su tema claro. A la izquierda se ubica el video con el esqueleto de las manos superpuesto y, en su esquina, la seña detectada; debajo, los botones para iniciar y detener la cámara. A la derecha, tres paneles muestran la detección (confianza, estabilidad y manos detectadas), el rendimiento y las opciones. En la parte inferior se acumula el texto traducido.

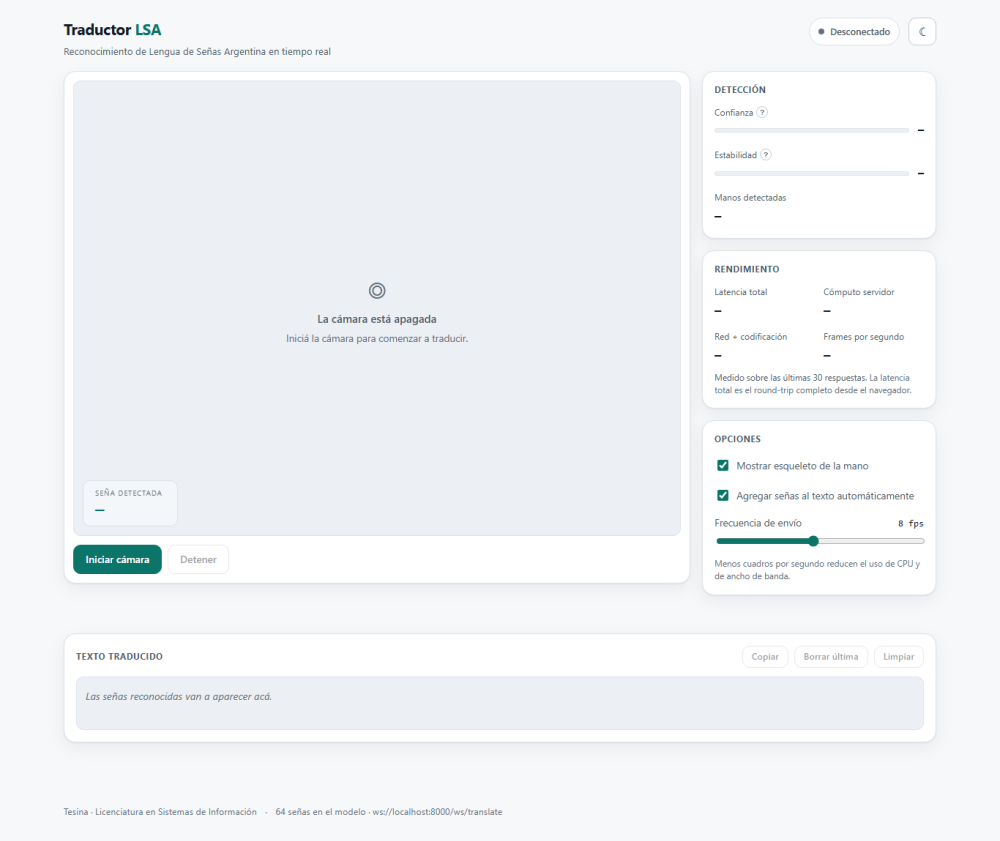


Figura *10. Interfaz del cliente web en su tema claro, antes de iniciar la cámara.*

PENDIENTE: agregar una captura con la cámara activa, el esqueleto dibujado y una seña reconocida (requiere hacerla con una cámara real; ver tarea F1 del plan).

12.2 Captura y envío de cuadros

Al presionar "Iniciar cámara", el cliente solicita acceso a la cámara con `getUserMedia`, pidiendo una resolución ideal de 960 × 720 píxeles, y muestra el video en pantalla. En paralelo, consulta `/model/info` para conocer las clases y las conexiones del esqueleto, y abre el canal WebSocket.

Un temporizador envía cuadros a la frecuencia configurada, de 8 cuadros por segundo por defecto, ajustable por el usuario. Para cada envío, el cliente:

1. Dibuja el cuadro actual del video en un canvas oculto de 480 píxeles de ancho, **espejado** horizontalmente mediante una transformación del contexto de dibujo (sección 9.7).
2. Codifica el canvas como JPEG con calidad 0,7.
3. Envía los bytes resultantes como un mensaje binario y registra el instante del envío.

La reducción a 480 píxeles y la compresión JPEG reducen el volumen de datos sin afectar la detección: MediaPipe trabaja internamente con imágenes de menor resolución.

12.2.1 Control de contrapresión

Si el servidor tarda más en procesar un cuadro que el intervalo entre envíos, los cuadros se acumulan en una cola que crece sin límite: la latencia aumenta constantemente y el usuario ve resultados cada vez más atrasados. Para evitarlo, el cliente lleva la cuenta de los cuadros enviados que todavía no tienen respuesta y **no envía uno nuevo si ya hay dos pendientes**. Con ese límite, el servidor siempre tiene trabajo, pero nunca acumula atraso.

Este control tiene también una consecuencia metodológica. Sin él, la latencia medida en el cliente dejaría de reflejar el tiempo de procesamiento y pasaría a medir, sobre todo, el tiempo de espera en la cola. Es un error de medición fácil de cometer y difícil de notar, porque el número resultante parece razonable al principio y empeora gradualmente.

12.2.2 Reconexión

Si la conexión WebSocket se interrumpe —por un corte de red o un reinicio del servidor, frecuente en las capas gratuitas—, el cliente intenta reconectarse automáticamente con **espera exponencial**: 0,5 segundos, luego 1, 2, 4 y hasta un máximo de 8 segundos entre intentos. Así se recupera rápido de cortes breves sin saturar al servidor con intentos durante una caída prolongada. El indicador de estado del encabezado informa en todo momento si el cliente está conectado, conectando o desconectado.

12.3 Presentación de los resultados

Cada respuesta del servidor actualiza la interfaz:

- La **seña detectada** se muestra sobre el video, en la esquina inferior.
- Las barras de **confianza** y **estabilidad** muestran ambos valores en porcentaje. Un ícono de ayuda junto a cada una explica qué significa.
- El contador de **manos detectadas** permite al usuario saber si está bien ubicado frente a la cámara.
- Cuando una seña se confirma como válida y es distinta de la anterior, se **agrega al texto traducido**. La comparación con la seña anterior evita que una seña sostenida se escriba muchas veces. El agregado automático puede desactivarse.

El texto traducido puede copiarse al portapapeles, corregirse borrando la última seña o limpiarse por completo.

12.4 Esqueleto de las manos

El servidor devuelve, junto con cada predicción, las coordenadas de los puntos de las manos detectadas. El cliente las dibuja sobre un canvas superpuesto al video, uniendo los puntos según las conexiones del esqueleto recibidas de `/model/info`. Este dibujo cumple una función

práctica: le muestra al usuario, en tiempo real, si el sistema está "viendo" sus manos. Si el esqueleto no aparece, el problema es de detección —iluminación, encuadre, distancia— y no de reconocimiento.

Las coordenadas llegan en el sistema de la imagen ya espejada, y el video también se muestra espejado. Por eso el canvas del esqueleto no lleva ninguna transformación adicional: los puntos caen directamente sobre las manos que ve el usuario. Los grosores de las líneas se escalan según el tamaño del video para mantener la proporción en cualquier pantalla.

12.5 Métricas de desempeño

El panel de rendimiento muestra cuatro valores, promediados sobre las últimas 30 respuestas:

- **Latencia total**: el tiempo entre el envío de un cuadro y la recepción de su respuesta, medido en el navegador.
- **Cómputo del servidor**: el tiempo de procesamiento que el servidor informa en cada respuesta.
- **Red y codificación**: la diferencia entre los dos anteriores, que corresponde a la transmisión por la red más la codificación y decodificación del JPEG.
- **Cuadros por segundo** efectivamente procesados.

Separar la latencia en sus dos componentes permite decidir con datos dónde optimizar. Si domina el cómputo del servidor, conviene optimizar el modelo o el preprocesamiento; si domina la red, conviene reducir el tamaño de los cuadros o trasladar la detección al cliente (sección 8.8.1). Las mediciones se presentan en la sección 14.8.

12.6 Accesibilidad

Las decisiones de accesibilidad se tomaron de forma explícita, siguiendo las pautas WCAG 2.2 (sección 6.7). La Tabla 12 las resume.

Tabla *12***.** *Decisiones de accesibilidad del cliente web.*

Decisión	Motivo
Región viva (`aria-live="polite"`) sólo en el texto traducido	Cambia únicamente al confirmarse una seña. Las métricas se actualizan unas ocho veces por segundo: anunciadas por un lector de pantalla harían la herramienta inutilizable
El canvas del esqueleto se oculta a las tecnologías de asistencia	Es decorativo: la información que transmite está disponible como texto en el panel de detección
Barras con `role="progressbar"` y `aria-valuetext`	Un valor leído como "Confianza: 72 por ciento" es más claro que un número sin unidad
Mensajes de error con `role="alert"` y estado de conexión con `role="status"`	Se anuncian sin que el usuario tenga que buscarlos

Decisión	Motivo
Objetivos táctiles de al menos 24 × 24 píxeles	Criterio 2.5.8 (nivel AA). Los íconos de ayuda miden 16 píxeles y se agrandan con un área transparente
Contraste verificado en ambos temas	Todos los pares de color superan 4,5:1; el mínimo medido es 4,93:1
Foco visible en todos los controles	Permite el uso completo con teclado
Respeto de `prefers-reduced-motion`	Desactiva las animaciones para quienes lo configuraron en su sistema
Tema claro y oscuro	Sigue la preferencia del sistema operativo y puede cambiarse manualmente; la elección se recuerda

La Figura 11 muestra el tema oscuro, que se aplica automáticamente cuando el sistema operativo del usuario lo tiene configurado.

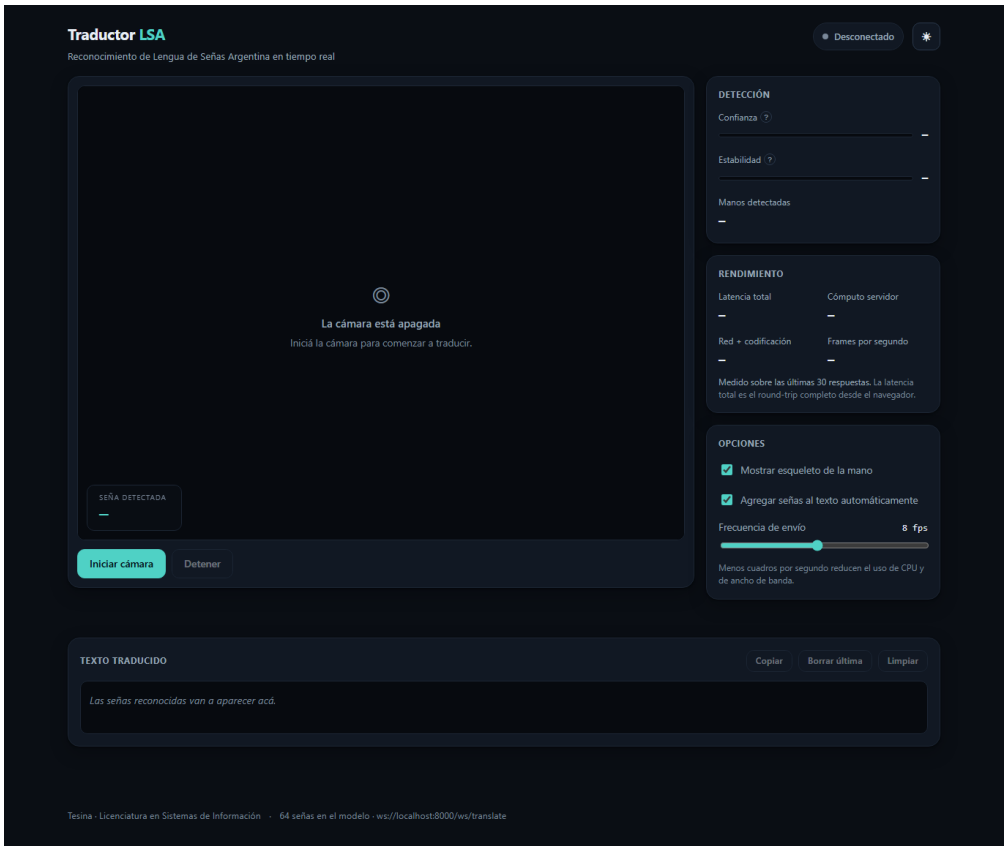


Figura *11. Interfaz del cliente web en su tema oscuro.*

12.7 Diseño adaptable

La interfaz se adapta al ancho de la pantalla: en pantallas angostas, los paneles laterales pasan a ubicarse debajo del video, en una sola columna, y los botones ocupan todo el ancho para facilitar su uso táctil. **PENDIENTE: verificar en un teléfono real; en una prueba con un navegador sin interfaz gráfica a 400 píxeles de ancho algunos paneles parecían desbordar a la derecha. Ver tarea F2 del plan.**

Capítulo 13 — Aseguramiento de la calidad y despliegue

Este capítulo describe las pruebas automatizadas del sistema, las consideraciones de seguridad y privacidad, y el empaquetado del sistema para su despliegue en una plataforma en la nube.

13.1 Pruebas automatizadas

El proyecto cuenta con **116 pruebas automatizadas** escritas con pytest, organizadas en dos carpetas: `tests/`, para el backend y el módulo compartido, y `ml/tests/`, para el pipeline de datos y entrenamiento de LSA64. Todas se ejecutan sin cámara, sin un modelo entrenado y, salvo las de integración, sin MediaPipe: utilizan **dobles de prueba** —objetos que imitan la interfaz de un componente real con un comportamiento controlado— y datos sintéticos. Esto permite ejecutarlas en cualquier máquina en menos de un minuto. La Tabla 13 resume su distribución.

Tabla *13. Distribución de las pruebas automatizadas.*

Archivo	Pruebas	Qué verifica
`tests/test_features.py`	14	Invarianza a traslación y escala, posiciones fijas por mano, indicador de presencia, serialización de los puntos para el dibujo
`tests/test_modelo_lse.py`	14	Carga de modelos, rechazo de esquemas incompatibles, correspondencia entre columnas y clases
`tests/test_traductor_service.py`	20	Suavizado, separación de confianza y estabilidad, aislamiento entre sesiones, ventana temporal
`tests/test_backend_e2e.py`	9	Arranque de la aplicación, servicios HTTP, ida y vuelta de un cuadro por WebSocket, conexiones simultáneas
`ml/tests/test_lsa64_data.py`	9	Lectura de la nomenclatura de LSA64, etiqueta y persona de cada video, reparto por persona, guardado e invalidación del caché
`ml/tests/test_lsa64_training.py`	9	Uso del caché sin releer los videos, rechazo de cachés incompatibles, filtrado de cuadros vacíos, longitud mínima, detección de fuga
`ml/tests/test_lsa64_loso.py`	8	Una partición por persona, particiones completas y disjuntas, determinismo, detección de una fuga inyectada
`ml/tests/test_lsa64_augment.py`	15	Aumentación: identidad con las opciones apagadas, que no toque los cuadros vacíos ni el indicador de presencia, determinismo con semilla, sólo en entrenamiento

Archivo	Pruebas	Qué verifica
`ml/tests/test_lsa64_labels_map.py`	6	Que las glosas se apliquen al final sobre las etiquetas canónicas, conserven el orden y nunca lleguen al caché
`ml/tests/test_features_posicion.py`	12	Variante v2 del vector: índices derivados del ancho real, compatibilidad con v1
Total	**116**	

13.1.1 Pruebas de regresión

Varias de las pruebas son **pruebas de regresión**: reproducen un error real encontrado durante el desarrollo, fallan con el código defectuoso y pasan con el corregido. Cada una documenta, en su descripción, qué fallaba y por qué. Algunos ejemplos:

- Que un modelo rechazado por esquema incompatible no quede cargado (sección 11.7).
- Que el suavizado decaiga cuando la persona baja las manos (sección 11.4.2).
- Que dos conexiones simultáneas no compartan el búfer de suavizado (sección 8.6).
- Que la etiqueta de un video de LSA64 sea la seña y no la persona (sección 9.11).
- Que aplicar las glosas no cambie el índice de ninguna clase (sección 10.7.1).

El valor de estas pruebas no está sólo en verificar el código actual, sino en impedir que un cambio futuro reintroduzca un error que ya costó encontrar.

13.1.2 Pruebas de integración

Las pruebas de `test_backend_e2e.py` levantan la aplicación completa con el cliente de pruebas de FastAPI y ejercitan los servicios reales: consultan `/health` y `/model/info`, verifican que se sirvan los archivos del cliente, envían un cuadro JPEG por WebSocket y validan la estructura de la respuesta, comprueban que un cuadro corrupto produzca un mensaje de error sin cerrar la conexión y que dos conexiones simultáneas no se interfieran. Requieren el archivo de pesos de MediaPipe; si no está, se omiten automáticamente en lugar de fallar.

13.1.3 Limitaciones de las pruebas

Las pruebas verifican la lógica del sistema, pero no la calidad del reconocimiento con personas reales frente a una cámara, que sólo puede evaluarse experimentalmente (Capítulo 14). Tampoco hay pruebas automatizadas del código JavaScript del cliente. **PENDIENTE: configurar la ejecución automática de las pruebas en cada envío al repositorio (por ejemplo, con GitHub Actions) — tarea Q1 del plan.**

13.2 Seguridad y privacidad

- **Sin almacenamiento de video.** Los cuadros recibidos se decodifican, se procesan en memoria y se descartan. El servidor no guarda imágenes ni secuencias de los usuarios (requisito RNF-06).
- **Usuario sin privilegios.** El contenedor ejecuta la aplicación con un usuario sin privilegios de administrador, de modo que una eventual vulnerabilidad en una dependencia tenga un alcance limitado.
- **Carga de modelos.** El formato de PyTorch permite, en general, ejecutar código arbitrario al cargar un archivo de pesos. El backend sólo carga el modelo versionado en el propio repositorio, generado por los scripts del proyecto, y nunca archivos provistos por los usuarios.
- **Orígenes permitidos (CORS).** Durante el desarrollo, el servidor acepta solicitudes de cualquier origen. Para la versión desplegada debe restringirse al dominio de la aplicación. **PENDIENTE: tarea D2 del plan — hay una propuesta en la rama `mejoras/despliegue`.**
- **HTTPS.** Los navegadores sólo permiten el acceso a la cámara en contextos seguros (sección 6.4), por lo que la versión publicada debe servirse con HTTPS. Las plataformas en la nube consideradas proveen certificados de forma automática.

13.3 Contenedor

El sistema se empaqueta en una única imagen de Docker, definida en el `Dockerfile` de la raíz del repositorio, que sirve la API, el canal WebSocket y el cliente. Las decisiones principales son:

- **Imagen base liviana** (`python:3.12-slim`), con sólo la biblioteca de sistema que MediaPipe requiere. Se utiliza la versión de OpenCV sin interfaz gráfica, que no necesita las bibliotecas de ventanas.
- **Dependencias en una capa propia.** El archivo de dependencias se copia e instala antes que el código. Docker reutiliza esa capa mientras las dependencias no cambien, de modo que modificar el código no obliga a reinstalar MediaPipe y PyTorch en cada construcción.
- **Modelo de MediaPipe descargado durante la construcción.** Si se descargara al arrancar, cada despliegue —y cada reinicio del contenedor, que en las capas gratuitas ocurre con frecuencia por inactividad— dependería de que el servidor de Google sea accesible desde la plataforma. Descargándolo en la construcción, la imagen queda autocontenida.
- **Modelo entrenado incluido en la imagen.** El modelo se versiona junto con el código y se copia a la imagen; nunca se entrena en producción.
- **Puerto configurable.** Las plataformas como Render y Railway indican el puerto en la variable de entorno `PORT`; el contenedor la respeta y usa el 8000 en forma local.

13.3.1 Tamaño de la imagen

PyTorch es necesario para servir la BiLSTM, pero la distribución por defecto incluye las bibliotecas de CUDA para GPU, que ocupan varios gigabytes y no se usan: la inferencia de una ventana de cuadros por vez se resuelve en CPU en pocos milisegundos. La distribución sólo para CPU ocupa una fracción de ese tamaño. Usarla es importante para la capa gratuita, que limita el espacio y penaliza los tiempos de arranque de imágenes grandes. **PENDIENTE: incorporar la instalación de PyTorch para CPU en el Dockerfile (propuesta en la rama `mejoras/despliegue`) y registrar aquí el tamaño final de la imagen.**

13.4 Despliegue

El anteproyecto establece que el despliegue debe mantenerse dentro de la capa gratuita de una plataforma en la nube. El diseño del sistema contempla esa restricción en varios puntos: un único servicio para la API y el cliente, que ocupa un solo lugar de los que permite la capa gratuita; una imagen autocontenida; un servicio de estado para los controles de salud de la plataforma; y un cliente que se reconecta solo cuando el servicio se reinicia.

La principal limitación esperable es la capacidad de cómputo. Cada cuadro de cada usuario requiere ejecutar MediaPipe en el servidor, y la CPU compartida de una capa gratuita puede atender a pocos usuarios simultáneos con latencias aceptables. La alternativa de trasladar la detección al cliente (sección 8.8.1) es la vía para escalar si fuera necesario.

PENDIENTE: desplegar efectivamente el sistema (tarea D1 del plan) y documentar aquí la plataforma elegida, la dirección pública, los pasos seguidos, el tiempo de arranque en frío y la latencia medida desde el navegador contra el servidor publicado.

Capítulo 14 — Resultados y evaluación

Este capítulo presenta los resultados de la evaluación del modelo y del sistema. Todos los números provienen de los reportes versionados en el repositorio (`ml/reports/` y `backend/models/modelo_lsa64_lstm.json`), y todas las figuras se generan a partir de ellos con un script (`docs/tesina/figuras/generar_figuras.py`), de modo que pueden reproducirse.

14.1 Por qué un único reparto no alcanza

La primera estimación del desempeño se obtuvo con el reparto único descripto en la sección 10.2: 7 personas para entrenar, 1 para validar y 2 para probar. Tres ejecuciones sobre datos prácticamente idénticos, con pequeñas variaciones de la configuración, dieron exactitudes de prueba de **86,05 %**, **78,55 %** y **81,25 %**. Una diferencia de 7,5 puntos entre ejecuciones casi iguales no permite afirmar nada: ni cuál es el desempeño del modelo ni si un cambio lo mejora.

El problema no es de medición, sino de muestra. Con dos personas en la prueba, la estimación depende fuertemente de quiénes sean esas dos personas. Este hallazgo motivó la incorporación del tercer objetivo específico —evaluar con un protocolo independiente del sujeto— y la adopción de la validación leave-one-subject-out como protocolo oficial de la tesina.

14.2 Resultado principal

Con la validación LOSO de 10 particiones (sección 10.6), sobre las 2862 secuencias de LSA64 que superan el filtrado y las 64 clases, el modelo obtuvo:

> *Exactitud independiente del sujeto:* ***79,8 % ± 8,9 %*** *(media ± desvío entre las 10 particiones).*

La exactitud calculada sobre las 2862 predicciones agrupadas —cada secuencia predicha una vez por un modelo que nunca vio a esa persona— es de 79,9 %, prácticamente igual a la media de las particiones. La ejecución completa tardó 55 minutos. La Tabla 14 detalla cada partición y la Figura 12 la representa gráficamente.

Tabla *14. Resultados por partición de la validación leave-one-subject-out.*

Persona de prueba	Persona de validación	Secuencias de entrenamiento	Secuencias de prueba	Mejor época	Exactitud
S01	S02	2270	286	34	86,4 %

Persona de prueba	Persona de validación	Secuencias de entrenamiento	Secuencias de prueba	Mejor época	Exactitud
S02	S03	2274	306	7	75,5 %
S03	S04	2283	282	12	63,1 %
S04	S05	2309	297	15	82,2 %
S05	S06	2310	256	16	87,1 %
S06	S07	2262	296	20	80,7 %
S07	S08	2297	304	14	89,5 %
S08	S09	2297	261	9	66,3 %
S09	S10	2288	304	24	85,5 %
S10	S01	2306	270	26	81,9 %
Media ± desvío					**79,8 % ± 8,9 %**

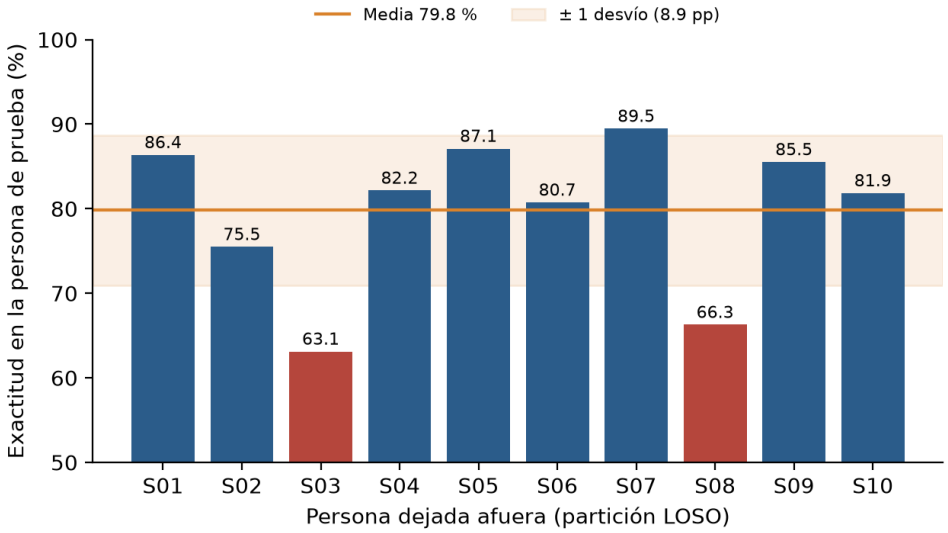


Figura *12*. *Exactitud por persona en la validación leave-one-subject-out. La línea indica la media y la banda, un desvío. En rojo, las personas por debajo de la media menos un desvío.*

El desvío es un hallazgo, no un defecto de la medición. Entre la persona con mejor resultado (S07, 89,5 %) y la de peor resultado (S03, 63,1 %) hay 26 puntos. Ninguna partición es un valor atípico según la regla habitual de 1,5 veces el rango intercuartílico: el límite inferior es 62,8 % y el peor valor es 63,1 %. La variación es, por lo tanto, una propiedad del problema: el modelo funciona bien para la mayoría de las personas y notablemente peor para dos de ellas, S03 y S08. Un número único, obtenido con un reparto que dejara a esas dos personas en el entrenamiento, habría ocultado este comportamiento.

14.3 Curva de entrenamiento y sobreajuste

La Figura 13 muestra la evolución del entrenamiento del modelo servido (reparto único, semilla 42). La pérdida de entrenamiento desciende de forma sostenida, de 2,55 en la primera época a 0,16 en la última, mientras que la exactitud de validación deja de mejorar a partir de la época 11 (81,6 %) y oscila alrededor del 78 % hasta que la detención temprana corta el entrenamiento en la época 19.

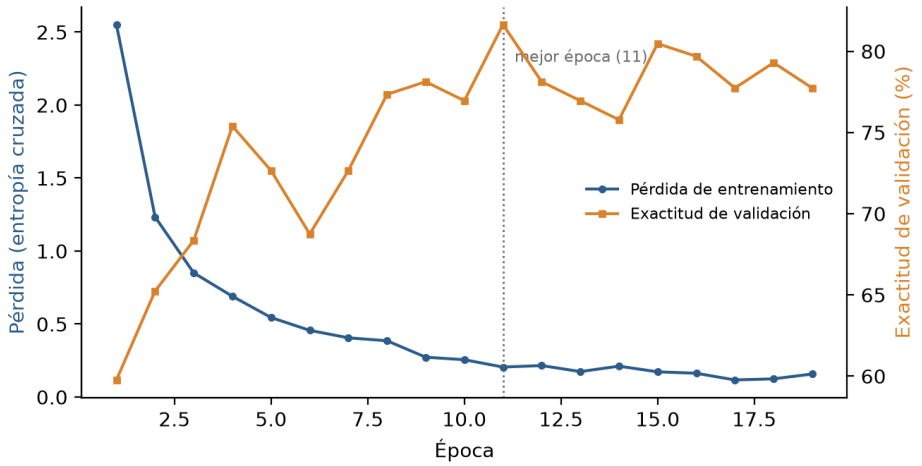


Figura *13. Pérdida de entrenamiento y exactitud de validación por época del modelo servido. La línea punteada marca la mejor época, cuyos pesos se conservan.*

Es el patrón típico del sobreajuste (sección 2.3): a partir de cierto punto, el modelo sigue aprendiendo particularidades del conjunto de entrenamiento que no generalizan a la persona de validación. La detención temprana evita que el modelo empeore, pero no resuelve la causa. Las técnicas de regularización evaluadas para atacarla se presentan en la sección 14.6.

El modelo servido obtuvo una exactitud de prueba de **81,25 %** sobre las personas S01 y S02, coherente con el resultado LOSO de esas dos personas (86,4 % y 75,5 %).

14.4 Análisis por clase

La exactitud no es pareja entre las 64 señas. La Figura 14 ordena las clases según su exactitud en las predicciones agrupadas del LOSO. La mediana es 81,4 %; 34 clases superan el 80 % y 18 superan el 90 %, con cuatro reconocidas sin errores (Rojo, Cajón, Agua y Dónde). En el otro extremo, 8 clases quedan por debajo del 60 %.

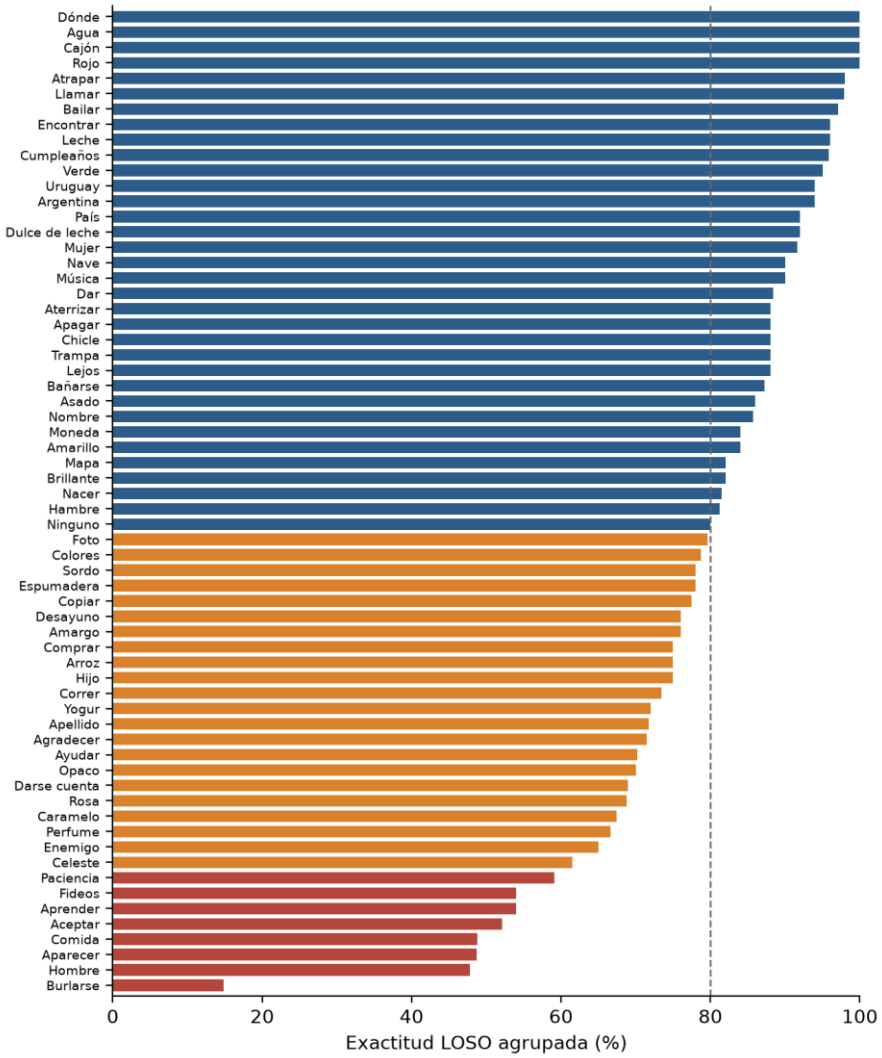


Figura *14. Exactitud por clase en las predicciones agrupadas del LOSO, ordenada de menor a mayor. En rojo, las clases por debajo de 60 %; en naranja, entre 60 % y 80 %. La línea punteada indica la exactitud global.*

Las clases con peor desempeño son Burlarse (14,8 %), Hombre (47,8 %), Aparecer (48,7 %), Comida (48,8 %), Aceptar (52,1 %), Aprender (54,0 %), Fideos (54,0 %) y Paciencia (59,1 %). Varias de ellas coinciden con las clases que más secuencias perdieron por falta de detección de las manos (sección 9.10): Hombre conserva 23 de sus 50 videos, Paciencia 22 y Burlarse 27. Con menos de la mitad de los datos, esas clases están subrepresentadas en el entrenamiento.

14.4.1 Señas de una y dos manos

Las 42 señas de una mano alcanzan una exactitud de 81,9 % y las 22 señas bimanuales, de 76,4 %. La diferencia es esperable: en una seña bimanual hay dos manos que detectar en cada cuadro, y la probabilidad de perder alguna es mayor. De los 574 errores del LOSO, 170 confunden una seña de una mano con una de dos o viceversa.

14.4.2 Matriz de confusión

La Figura 15 presenta la matriz de confusión de las 2862 predicciones agrupadas, normalizada por fila. La diagonal concentra la mayor parte de las predicciones; los errores se reparten de forma dispersa, con algunos pares que destacan.

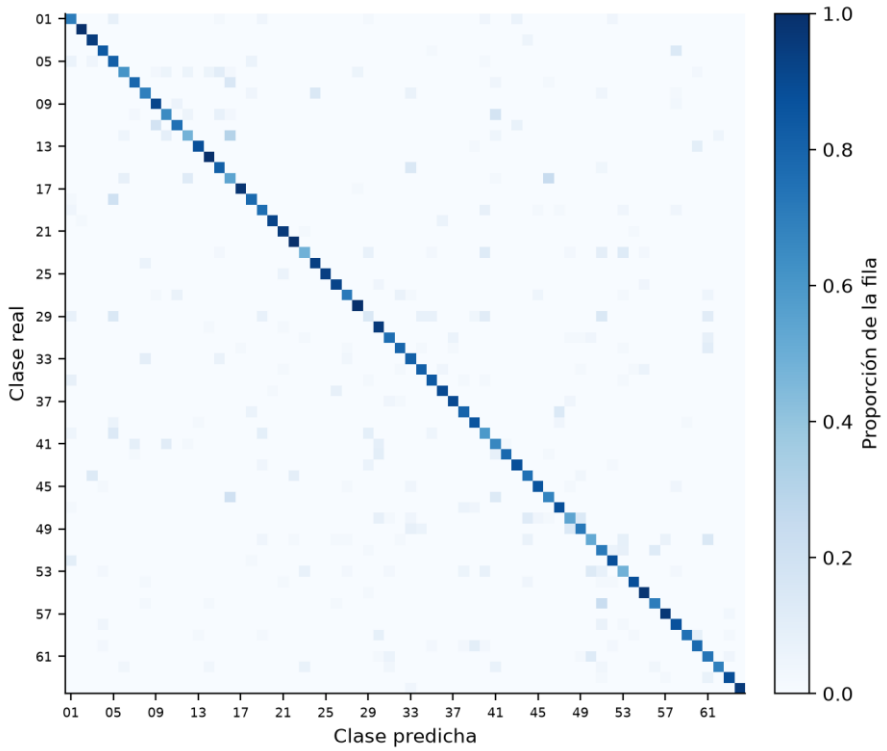


Figura *15. Matriz de confusión de las predicciones agrupadas del LOSO, normalizada por fila. Los números de los ejes corresponden a la numeración de LSA64 (ver el Anexo A).*

La Tabla 15 enumera los pares más confundidos.

Tabla *15. Pares de señas más confundidos en el LOSO.*

Seña real	Predicha como	Casos	Secuencias de la seña real
Aprender	Caramelo	12	50
Ayudar	Agradecer	11	47
Espumadera	Brillante	10	50
Caramelo	Aprender	9	46
Hijo	Mujer	8	44
Hombre	Aprender	7	23
Amarillo	Bañarse	7	50
Colores	Aprender	7	47
Rosa	Argentina	7	48
Enemigo	Perfume	7	40
Ninguno	Chicle	7	50
Fideos	Yogur	7	50

La confusión más frecuente es simétrica: Aprender se confunde con Caramelo y Caramelo con Aprender. Según el análisis del modelo servido, las confusiones se concentran en señas que comparten configuración manual y difieren principalmente en la **ubicación**, precisamente el parámetro que la normalización respecto de la muñeca descarta (sección 9.6). [[PENDIENTE (tarea M4 del plan): verificar con los videos de LSA64, par por par, si las señas más confundidas efectivamente comparten configuración y difieren en ubicación, y describirlo aquí con una figura de ejemplo.]]

14.5 Detección de manos y desempeño

La sección 9.10 mostró que la proporción de cuadros con manos detectadas varía entre personas. La Figura 16 relaciona esa retención con la exactitud LOSO de cada persona.

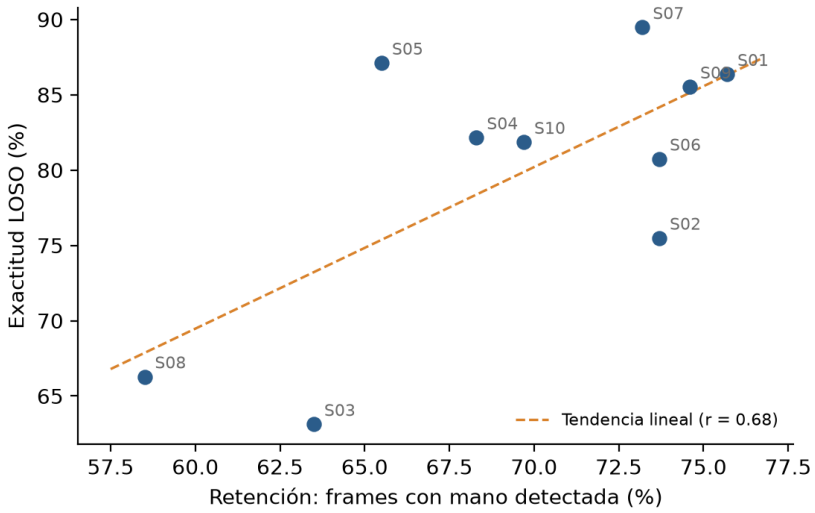


Figura *16. Relación entre la retención de cuadros con manos detectadas y la exactitud LOSO por persona.*

La correlación es de **0,68**: las personas con peor detección tienden a obtener peor exactitud, y las dos personas con peor resultado, S03 y S08, son también las dos con peor retención (63,5 % y 58,5 %). Con 10 personas, la correlación es sugestiva pero no concluyente. La interpretación es consistente con el diseño del sistema: el clasificador sólo ve lo que MediaPipe detecta, y una persona cuyas manos se pierden en un tercio de los cuadros le entrega al modelo secuencias incompletas.

14.6 Experimentos de mejora

14.6.1 Hipótesis descartada: preservar la grilla temporal

Hipótesis. Al descartar los cuadros sin manos, la secuencia se comprime de forma irregular: un video de 44 cuadros muestreados puede quedar en 18. Se sospechaba que ese "salto"

en el tiempo, y no sólo la pérdida de información, perjudicaba al modelo. Si así fuera, conservar los cuadros vacíos —con sus vectores en cero, manteniendo el ritmo original de la seña— debería mejorar el desempeño, especialmente en las personas con peor detección.

Resultado. La hipótesis resultó falsa. La Tabla 16 y la Figura 17 comparan ambas representaciones con el mismo protocolo LOSO y el mismo caché.

Tabla *16. Comparación entre filtrar los cuadros sin manos y conservar la grilla temporal completa.*

Persona	Retención	Cuadros filtrados	Grilla completa	Diferencia
S01	75,7 %	86,4 %	79,4 %	−7,0
S02	73,7 %	75,5 %	72,9 %	−2,6
S03	63,5 %	63,1 %	62,8 %	−0,4
S04	68,3 %	82,2 %	80,1 %	−2,0
S05	65,5 %	87,1 %	74,6 %	−12,5
S06	73,7 %	80,7 %	75,7 %	−5,1
S07	73,2 %	89,5 %	82,2 %	−7,2
S08	58,5 %	66,3 %	72,8 %	+6,5
S09	74,6 %	85,5 %	85,9 %	+0,3
S10	69,7 %	81,9 %	78,9 %	−3,0
Media		**79,8 %**	**76,5 %**	**−3,3**
Desvío		**8,9 %**	**6,4 %**	

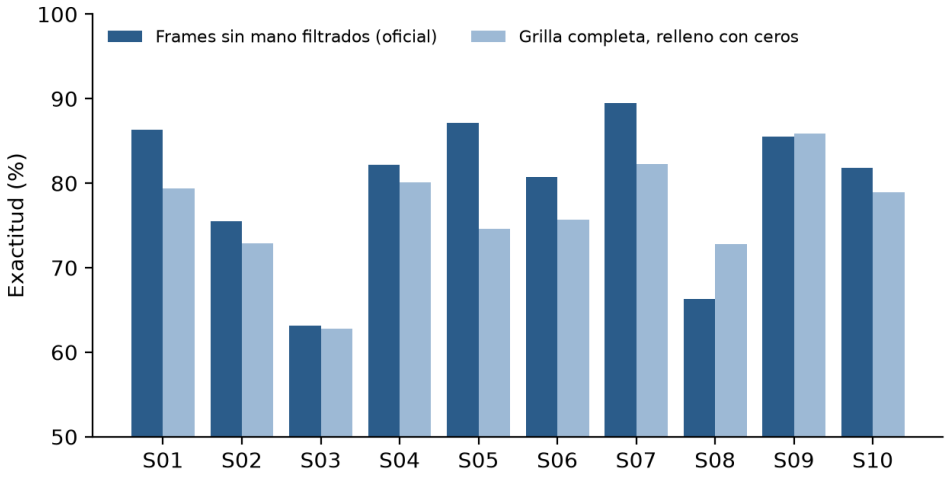


Figura *17. Exactitud por persona con los cuadros sin manos filtrados (representación oficial) y con la grilla temporal completa.*

Ocho de las diez personas empeoraron. El desvío baja, pero nivelando hacia abajo: el máximo cae de 89,5 % a 85,9 % mientras el mínimo se mantiene alrededor del 63 %. La correlación entre retención y exactitud apenas cambia (de 0,68 a 0,60), lo que indica que rellenar con ceros no desacopla el desempeño de la calidad de la detección, que era el objetivo.

Interpretación. El problema no es que los cuadros se hayan quitado, sino que faltan. Un vector en cero conserva la posición temporal pero no aporta información, y obliga a la red a dedicar capacidad a ignorar un tercio de su entrada; con un modelo que ya sobreajusta, el balance es negativo. La BiLSTM tolera la compresión irregular del tiempo mejor de lo que se suponía. Sólo la persona S08, la de peor retención, mejoró notablemente (+6,5), pero S05, con 65,5 % de retención, empeoró 12,5 puntos, de modo que no hay un patrón. **La representación con cuadros filtrados se mantiene como oficial**, y un relleno de los huecos sólo debería volver a evaluarse con un mecanismo distinto, como la interpolación o una máscara explícita.

14.6.2 Regularización y representación con ubicación

Para evaluar rápidamente las técnicas contra el sobreajuste se utilizó la evaluación parcial sobre las dos personas más difíciles, S03 y S08 (sección 10.6.1). La Figura 18 resume los resultados.

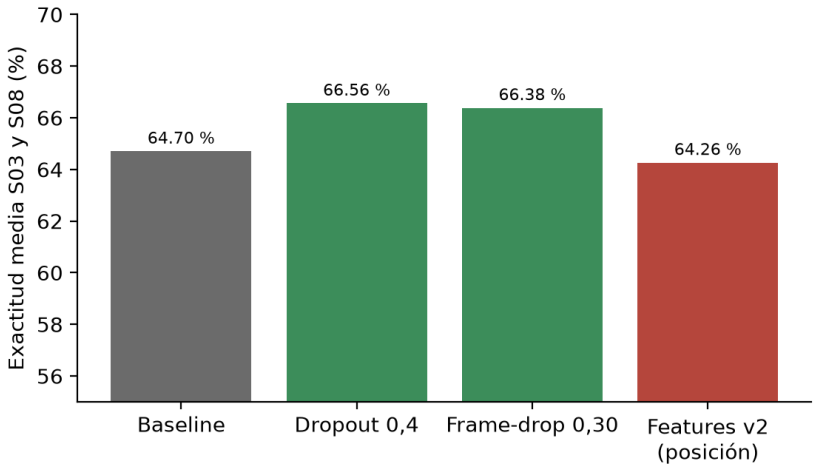


Figura *18. Exactitud media sobre las personas S03 y S08 para la configuración de referencia y tres variantes.*

- **Configuración de referencia**: 64,70 %.
- **Dropout 0,4** (en lugar de 0,2): 66,56 %, una mejora de 1,9 puntos, presente en las dos personas.
- **Descarte aleatorio de cuadros con probabilidad 0,30**: 66,38 %, una mejora de 1,7 puntos, también en las dos personas. Es la técnica que simula directamente a una persona con mala detección, el caso en que el modelo falla.
- **Representación v2 con la ubicación de la muñeca**: 64,26 %, sin mejora.

Estos resultados deben leerse con cautela: son dos particiones de diez y las diferencias son pequeñas. Indican qué variantes vale la pena confirmar, no que mejoren el modelo. [[PENDIENTE (tarea M1 del plan): confirmar con el LOSO completo la combinación de dropout 0,4 y descarte de cuadros 0,30, y, si mejora, reentrenar el modelo servido.]]

El resultado de la representación v2 merece un comentario. Que agregar la ubicación no mejore, cuando el análisis de confusiones sugiere que la ubicación es la información faltante, tiene al menos dos explicaciones posibles: que la posición de la muñeca en el encuadre no sea una buena medida de la ubicación respecto del cuerpo, o que las dos personas evaluadas fallen por la detección y no por la ubicación. Distinguir ambas requiere evaluar v2 con el LOSO completo y, eventualmente, una representación de la ubicación relativa al cuerpo.

14.7 Comparación con los antecedentes

La Tabla 3 del Capítulo 5 ubica el resultado propio respecto de la literatura. En el mismo protocolo —independiente del sujeto—, el 79,8 % supera el 74,22 % de Marais et al. (2022) y queda por debajo del 98,53 % de Akdag y Baykan (2024), obtenido con un modelo convolucional 3D sobre el video completo. El 99,4 % de Mindlin (2021) corresponde a un protocolo dependiente del sujeto y no es comparable.

14.7.1 Línea base no neuronal

Para justificar el uso de una red recurrente, es necesario compararla con un modelo más simple. La línea base prevista es un bosque aleatorio (sección 2.6) entrenado sobre un resumen de cada secuencia —media, desvío, mínimo y máximo de cada característica—, evaluado con el mismo protocolo LOSO. [[PENDIENTE (tarea M2 del plan): ejecutar la línea base y completar este apartado con su resultado. Si el bosque aleatorio obtiene un resultado cercano al de la BiLSTM, la conclusión es que el orden temporal aporta poco con esta representación, lo que también es un resultado relevante.]]

14.8 Desempeño en tiempo real

14.8.1 Latencia

[[PENDIENTE (tarea F3 del plan): medir la latencia con el panel de rendimiento del cliente, en tres escenarios — servidor local, servidor en la red local y servidor desplegado — y registrar latencia total, cómputo del servidor y red, a 8 y 15 cuadros por segundo. Presentar los resultados en una tabla.]]

14.8.2 Brecha entre el entrenamiento y el uso en tiempo real

La exactitud del LOSO mide la capacidad del modelo para clasificar una seña completa, recortada de antemano, grabada en condiciones de laboratorio. El uso en la aplicación difiere de esa situación en tres aspectos, que constituyen la principal limitación conocida del sistema:

1. **Frecuencia de muestreo.** El modelo se entrenó con secuencias a 30 cuadros por segundo (sección 9.2). El cliente envía, por defecto, 8 cuadros por segundo. Una seña de un segundo ocupa 30 cuadros en el entrenamiento y 8 en la aplicación: el modelo la ve "acelerada" casi cuatro veces.
2. **Segmentación.** En el entrenamiento, cada secuencia es exactamente una seña, de principio a fin. En la aplicación no se sabe dónde empieza y termina una seña: la sesión clasifica una ventana deslizante de los últimos 8 cuadros con manos detectadas (sección 11.5), que puede contener el final de una seña, el comienzo de otra o una transición.
3. **Condiciones de captura.** LSA64 se grabó con fondo blanco, ropa negra, guantes de color e iluminación controlada. Una cámara web doméstica captura escenas más variadas.

Ninguno de estos aspectos se refleja en la cifra del LOSO, y es esperable que el desempeño percibido en la aplicación sea menor. [[PENDIENTE (tareas M5 y F3 del plan): (a) alinear la frecuencia — enviar a 15 cuadros por segundo o volver a entrenar con un muestreo equivalente al del cliente —; (b) reemplazar la ventana fija por una segmentación según la presencia de las manos, que acumule la seña completa y clasifique al bajar las manos; (c) medir el desempeño real con un protocolo simple: un conjunto de señas realizadas frente a la cámara por personas que no estén en LSA64, registrando aciertos, errores y señas no detectadas.]]

14.9 Discusión

Los resultados permiten extraer algunas conclusiones sobre el modelo y sobre el proceso.

El protocolo de evaluación determina la cifra. La misma familia de modelos obtiene valores cercanos al 99 % o en torno al 75 % según se mezclen o no las personas entre entrenamiento y prueba (Capítulo 5). En este trabajo, un reparto único habría permitido informar 86 % eligiendo, sin malicia, la ejecución más favorable. La cifra que se informa, 79,8 % ± 8,9 %, es menor, pero es la que corresponde al uso real del sistema: reconocer a personas que el modelo nunca vio.

La calidad de la detección es el principal factor limitante. La exactitud por persona se asocia con la retención de cuadros, las clases con peor desempeño son en buena medida las que más datos perdieron, y las señas bimanuales, con dos manos que detectar, funcionan peor. Mejorar la detección —con otros umbrales, con el modo de seguimiento de MediaPipe o con la recuperación de los videos descartados— tiene, probablemente, más potencial que modificar la red.

La ubicación es la información faltante más plausible. Las confusiones apuntan a señas que difieren en la ubicación, y la representación v1 descarta ese parámetro por diseño. La primera prueba de agregarlo no dio resultado, pero la prueba fue parcial y la representación de la ubicación, mejorable.

Los resultados negativos también son resultados. La hipótesis de la grilla temporal era razonable y resultó falsa. Haberla evaluado con un protocolo riguroso evitó adoptar un cambio que empeoraba el modelo en 3,3 puntos, y dejó documentado por qué no conviene volver a proponerlo en su forma simple.

14.10 Síntesis

El modelo alcanza una exactitud independiente del sujeto de 79,8 % ± 8,9 % sobre las 64 señas de LSA64, con una variación entre personas que está asociada con la calidad de la detección de las manos. Las confusiones se concentran en señas que comparten configuración. Preservar la grilla temporal con ceros empeora el modelo; el dropout más alto y el descarte aleatorio de cuadros muestran mejoras preliminares que deben confirmarse. La principal limitación del sistema es la diferencia entre las condiciones del entrenamiento y las del uso en tiempo real, que deberá medirse y reducirse en la etapa final del proyecto.

Conclusiones

Conclusiones generales

Esta tesina se propuso desarrollar una aplicación web basada en inteligencia artificial que traduzca a texto, en tiempo real, señas aisladas de la Lengua de Señas Argentina. El resultado es un sistema funcional de tres capas: un cliente web que captura el video de la cámara y lo envía por WebSocket; un backend en Python que detecta las manos con MediaPipe, construye un vector de 138 características por cuadro y clasifica la secuencia con una red BiLSTM; y un pipeline de entrenamiento y evaluación sobre el conjunto de datos LSA64, que comparte con el backend el código de extracción de características.

Respecto de los objetivos específicos:

1. **Diseño de la arquitectura.** Se diseñó e implementó una arquitectura cliente-backend-modelo con un módulo de código compartido, que garantiza por construcción que el preprocesamiento sea el mismo en el entrenamiento y en la inferencia. El diseño separa el estado de la aplicación del estado de cada conexión, y admite modelos de distinto formato mediante adaptadores (Capítulos 8 y 11).
2. **Entrenamiento de la red.** Se entrenó una BiLSTM de 290 880 parámetros sobre LSA64, con un proceso determinista y repartos de datos siempre por persona (Capítulos 9 y 10).
3. **Evaluación independiente del sujeto.** El modelo alcanzó una exactitud de 79,8 % ± 8,9 % en una validación leave-one-subject-out de 10 particiones, sobre 64 clases. Se analizaron los errores por persona y por clase, y se evaluaron hipótesis de mejora (Capítulo 14). Este objetivo no figuraba en el anteproyecto: se incorporó al comprobar que un reparto único producía cifras que variaban hasta 7,5 puntos entre ejecuciones casi idénticas.
4. **Aplicación web.** Se desarrolló un cliente accesible, que cumple los criterios de nivel AA de WCAG 2.2 aplicables, con dibujo del esqueleto de las manos, métricas de latencia y reconexión automática (Capítulo 12).
5. **Despliegue.** El sistema está empaquetado en una imagen Docker pensada para la capa gratuita de una plataforma en la nube. **PENDIENTE: completar con el resultado del despliegue — tarea D1 del plan.**

Inconvenientes resueltos y camino recorrido

El proyecto partió de un prototipo de escritorio de terceros y de un diagrama de arquitectura. El camino hasta el sistema actual estuvo marcado, más que por la construcción de

componentes nuevos, por la detección y corrección de errores que no producían ningún mensaje de error y que, sin embargo, invalidaban los resultados:

- La **extracción de características duplicada** en el prototipo, que se resolvió con el módulo compartido y la versión del esquema verificada al cargar cada modelo.
- La **orientación inconsistente de la imagen** entre captura, entrenamiento e inferencia, que intercambiaba las manos en el vector sin que ningún control lo advirtiera, y que se resolvió con una convención explícita y verificable.
- El **etiquetado incorrecto** de LSA64, que entrenaba con 50 clases artificiales en lugar de las 64 señas reales, y la consiguiente **fuga de datos** entre entrenamiento y prueba.
- El **entrenamiento no reproducible**, que impedía distinguir una mejora real del ruido.
- El **estado compartido entre usuarios** en el backend, invisible en el diagrama de clases.
- Un **flag de presentación que alteraba el resultado** del entrenamiento, al cambiar el orden de las clases.

El aprendizaje común a todos estos casos es que, en un sistema de aprendizaje automático, los errores más graves no se manifiestan como fallas, sino como números verosímiles. Un modelo entrenado con etiquetas incorrectas o con fuga de datos produce una exactitud que parece razonable. La defensa contra este tipo de error no es la revisión del código en abstracto, sino un conjunto de prácticas: verificaciones explícitas de los supuestos (que no haya fuga, que el esquema coincida, que el caché sea compatible), pruebas de regresión para cada error encontrado, reproducibilidad estricta y un protocolo de evaluación fijo y exigente.

Contribuciones y aportes

A la comunidad sorda y a la sociedad. El trabajo ofrece una herramienta de código abierto que demuestra la viabilidad de reconocer señas de la LSA en tiempo real con una cámara común y un navegador, y que puede servir como base para herramientas de apoyo a la comunicación en ámbitos concretos —atención al público, salud, educación—, con un vocabulario adaptado a cada uno.

A otros estudiantes e investigadores.

- Un sistema completo, documentado y con pruebas, que integra un modelo de reconocimiento en una aplicación web, algo poco frecuente en los antecedentes sobre LSA.
- Una evaluación independiente del sujeto sobre LSA64 con un modelo liviano basado en landmarks, con resultados por persona y por clase, y las predicciones disponibles para análisis posteriores.
- La documentación de un caso concreto, con evidencia, de cómo el protocolo de evaluación cambia las conclusiones, incluida la revisión del protocolo del principal antecedente citado.

- Las glosas de LSA64 en castellano, con la corrección de la seña 008 y criterios explícitos para las traducciones ambiguas.
- Un registro de errores metodológicos frecuentes en proyectos de reconocimiento de señas y de cómo detectarlos, útil para quien encare un trabajo similar.

Futuro del proyecto

Los siguientes puntos quedan abiertos para continuar el trabajo, ordenados aproximadamente según su impacto esperado:

1. **Cerrar la brecha con el uso real.** Segmentar las señas según la presencia de las manos en lugar de usar una ventana fija, alinear la frecuencia de muestreo entre entrenamiento y uso, y medir el desempeño con personas reales frente a la cámara (sección 14.8).
2. **Mejorar la detección de las manos.** Evaluar el modo de seguimiento de video de MediaPipe, otros umbrales de confianza y la recuperación de los videos descartados, que es el factor más asociado con los errores (sección 14.5).
3. **Representar la ubicación respecto del cuerpo.** Incorporar puntos del cuerpo con el modelo de pose de MediaPipe para expresar la posición de las manos de forma independiente del encuadre, y evaluar si reduce las confusiones entre señas que difieren en la ubicación.
4. **Ampliar el vocabulario y los datos.** Capturar un conjunto de datos propio con más personas, en condiciones reales y con consentimiento informado, orientado a un ámbito de uso concreto; incorporar el alfabeto manual completo.
5. **Otras arquitecturas.** Probar modelos basados en atención (Transformers) y redes convolucionales sobre grafos del esqueleto, que son el estado del arte en reconocimiento basado en landmarks.
6. **Detección en el cliente.** Ejecutar MediaPipe en el navegador para reducir el cómputo del servidor y el ancho de banda, con una verificación automática de que produce los mismos vectores que la implementación en Python (sección 8.8.1).
7. **Reconocimiento continuo y componentes no manuales.** A más largo plazo, avanzar hacia el reconocimiento de frases y la incorporación de la expresión facial, que en la LSA forma parte del significado.
8. **Participación de la comunidad sorda.** Cualquier evolución del sistema debería diseñarse y evaluarse junto con personas sordas usuarias de la LSA, que son quienes pueden determinar si la herramienta resulta útil y en qué situaciones.

ANEXOS

Apéndices del trabajo

Anexo A — Vocabulario de señas

Este anexo lista las 64 señas de LSA64 que reconoce el sistema, con su glosa en castellano utilizada en la aplicación, la glosa en inglés con que se publica el conjunto de datos, la cantidad de manos con que se realiza, la cantidad de secuencias que superaron el filtrado de detección (sobre 50 videos por seña) y la exactitud obtenida en las predicciones agrupadas de la validación leave-one-subject-out (sección 14.4). Los criterios de traducción de las glosas se explican en la sección 9.1.

Tabla *17.* *Vocabulario de LSA64 con glosas, cantidad de manos, secuencias conservadas y exactitud LOSO.*

N.º	Glosa (castellano)	Glosa publicada (inglés)	Manos	Secuencias	Exactitud LOSO
001	Opaco	Opaque	1	50	70,0 %
002	Rojo	Red	1	44	100,0 %
003	Verde	Green	1	40	95,0 %
004	Amarillo	Yellow	1	50	84,0 %
005	Brillante	Bright	1	50	82,0 %
006	Celeste	Light-blue	1	39	61,5 %
007	Colores	Colors	1	47	78,7 %
008	Rosa	Pink	1	48	68,8 %
009	Mujer	Women	1	36	91,7 %
010	Enemigo	Enemy	1	40	65,0 %
011	Hijo	Son	1	44	75,0 %
012	Hombre	Man	1	23	47,8 %
013	Lejos	Away	1	50	88,0 %
014	Cajón	Drawer	1	28	100,0 %
015	Nacer	Born	1	27	81,5 %
016	Aprender	Learn	1	50	54,0 %
017	Llamar	Call	1	47	97,9 %
018	Espumadera	Skimmer	1	50	78,0 %
019	Amargo	Bitter	1	50	76,0 %
020	Dulce de leche	Sweet milk	1	50	92,0 %
021	Leche	Milk	1	50	96,0 %
022	Agua	Water	1	49	100,0 %
023	Comida	Food	1	41	48,8 %
024	Argentina	Argentina	1	50	94,0 %

N.º	Glosa (castellano)	Glosa publicada (inglés)	Manos	Secuencias	Exactitud LOSO
025	Uruguay	Uruguay	1	50	94,0 %
026	País	Country	1	50	92,0 %
027	Apellido	Last name	1	46	71,7 %
028	Dónde	Where	1	40	100,0 %
029	Burlarse	Mock	2	27	14,8 %
030	Cumpleaños	Birthday	1	48	95,8 %
031	Desayuno	Breakfast	2	50	76,0 %
032	Foto	Photo	2	49	79,6 %
033	Hambre	Hungry	1	32	81,2 %
034	Mapa	Map	2	50	82,0 %
035	Moneda	Coin	2	50	84,0 %
036	Música	Music	2	50	90,0 %
037	Nave	Ship	1	50	90,0 %
038	Ninguno	None	1	50	80,0 %
039	Nombre	Name	1	49	85,7 %
040	Paciencia	Patience	1	22	59,1 %
041	Perfume	Perfume	1	45	66,7 %
042	Sordo	Deaf	1	50	78,0 %
043	Trampa	Trap	2	50	88,0 %
044	Arroz	Rice	2	32	75,0 %
045	Asado	Barbecue	2	50	86,0 %
046	Caramelo	Candy	1	46	67,4 %
047	Chicle	Chewing-gum	1	50	88,0 %
048	Fideos	Spaghetti	2	50	54,0 %
049	Yogur	Yogurt	2	50	72,0 %
050	Aceptar	Accept	2	48	52,1 %
051	Agradecer	Thanks	2	49	71,4 %
052	Apagar	Shut down	1	50	88,0 %
053	Aparecer	Appear	2	39	48,7 %
054	Aterrizar	To land	2	50	88,0 %
055	Atrapar	Catch	2	50	98,0 %
056	Ayudar	Help	2	47	70,2 %
057	Bailar	Dance	2	34	97,1 %
058	Bañarse	Bathe	2	47	87,2 %
059	Comprar	Buy	1	48	75,0 %
060	Copiar	Copy	2	40	77,5 %
061	Correr	Run	2	49	73,5 %
062	Darse cuenta	Realize	1	29	69,0 %

N.º	Glosa (castellano)	Glosa publicada (inglés)	Manos	Secuencias	Exactitud LOSO
063	Dar	Give	2	43	88,4 %
064	Encontrar	Find	1	50	96,0 %

Dos glosas en castellano no pudieron confirmarse con una fuente independiente y se adoptaron por ser la traducción más directa: la seña 009, publicada como *Women*, se glosa "Mujer", y la 048, *Spaghetti*, se glosa "Fideos".

PENDIENTE: el anteproyecto preveía incluir en este anexo imágenes representativas de las señas. Se sugiere agregar una figura con un cuadro de cada una de algunas señas de ejemplo, tomadas de los videos de LSA64 (licencia CC BY-NC-SA 4.0, citando la fuente), en particular de los pares más confundidos de la Tabla 15.

Anexo B — Detalle de la red neuronal

Este anexo complementa la sección 10.1 con las dimensiones de cada capa de la red, su implementación y el contenido del archivo del modelo.

B.1 Capas y parámetros

Tabla *18. Capas de la red BiLSTM, dimensiones y cantidad de parámetros.*

Capa	Entrada	Salida	Parámetros	Cálculo
LSTM hacia adelante	(T, 138)	128	137 216	4 × (138 × 128 + 128 × 128 + 2 × 128)
LSTM hacia atrás	(T, 138)	128	137 216	Ídem
Concatenación	128 + 128	256	0	
Dropout (p = 0,2)	256	256	0	Sólo activo durante el entrenamiento
Lineal	256	64	16 448	256 × 64 + 64
Total			**290 880**	

El factor 4 de las capas LSTM corresponde a las tres compuertas y a la memoria candidata, cada una con su propia matriz de pesos para la entrada, otra para el estado oculto y sus sesgos (en PyTorch, dos vectores de sesgo por compuerta).

B.2 Implementación

La red se define en `common/models/lsa64.py` y se utiliza tanto en el entrenamiento como en el backend:

```
class BiLSTMClassifier(nn.Module):
    def __init__(self, input_size, hidden_size, num_classes, dropout=0.2):
        super().__init__()
        self.lstm = nn.LSTM(input_size=input_size, hidden_size=hidden_size,
                            batch_first=True, bidirectional=True)
        self.dropout = nn.Dropout(dropout)
        self.fc = nn.Linear(hidden_size * 2, num_classes)

    def forward(self, x, lengths):
        packed = pack_padded_sequence(x, lengths.cpu(), batch_first=True,
                                      enforce_sorted=False)
        _, (hidden, _) = self.lstm(packed)
        features = torch.cat([hidden[-2], hidden[-1]], dim=1)
        return self.fc(self.dropout(features))
```

El método `forward` recibe el lote de secuencias completadas con ceros y la longitud real de cada una. `pack_padded_sequence` hace que la recurrencia se detenga en el último cuadro

válido de cada secuencia, de modo que el relleno no contamina el estado final. `hidden[-2]` y `hidden[-1]` son los estados finales de la dirección hacia adelante y hacia atrás, respectivamente.

B.3 Contenido del archivo del modelo

El archivo `backend/models/modelo_lsa64_lstm.pt` es un diccionario de PyTorch con los siguientes campos:

Tabla *19***.** *Campos del archivo del modelo entrenado.*

Campo	Contenido
`model_state_dict`	Pesos de la red
`label_classes`	Nombres de las 64 clases, en el orden de las neuronas de salida
`feature_version`	Versión del esquema de características (`v1`)
`feature_vector_length`	Longitud del vector por cuadro (138)
`hidden_size`	Unidades LSTM por dirección (128)
`frame_step`	Muestreo temporal usado al extraer (2)
`max_frames`	Longitud máxima de secuencia (120)
`min_seq_len`	Longitud mínima de secuencia (8); el backend la usa como tamaño de la ventana
`dataset`, `train_size`, `val_size`, `test_size`, `samples`	Trazabilidad: conjunto de datos, tamaños del reparto y lista de videos usados

Con estos campos, el backend reconstruye la red y valida su compatibilidad sin depender de ninguna configuración externa.

Anexo C — Guía de instalación y ejecución

Este anexo resume los pasos para reproducir el sistema a partir del repositorio. La versión completa y actualizada está en el archivo `README.md` del repositorio. **PENDIENTE: agregar la dirección del repositorio y la carpeta compartida "Fuentes del proyecto" exigida por la facultad.**

C.1 Requisitos

- Python 3.12.
- Git.
- Una cámara web y un navegador moderno (Chrome, Edge o Firefox) para usar la aplicación.
- Opcionalmente, Docker.
- Para reentrenar el modelo: el conjunto de datos LSA64, que se descarga desde la página de sus autores.

C.2 Descarga del modelo de MediaPipe

El archivo de pesos de MediaPipe no se versiona. Se descarga una única vez con:

```
pip install mediapipe
python common/download_model.py
```

El script guarda el archivo en `common/models/hand_landmarker.task`.

C.3 Ejecución del backend y del cliente

```
python -m venv .venv-backend
.venv-backend\Scripts\activate
pip install -r backend/requirements.txt
uvicorn backend.app.main:app --port 8000
```

Con el servidor en marcha, la aplicación queda disponible en `http://localhost:8000`. El servicio `http://localhost:8000/health` informa si el modelo de MediaPipe y el modelo entrenado se cargaron correctamente, y `http://localhost:8000/docs` muestra la documentación de la API.

C.4 Ejecución con Docker

```
docker build -t traductor-lsa .
docker run --rm -p 8000:8000 traductor-lsa
```

La imagen descarga el modelo de MediaPipe durante la construcción e incluye el modelo entrenado, de modo que no requiere pasos adicionales.

C.5 Reentrenamiento del modelo

```
python -m venv .venv-ml
.venv-ml\Scripts\activate
pip install -r ml/requirements.txt

# Etapa 1: extracción de características (una sola vez, ~47 minutos)
python ml/extract_features.py --dataset-dir ".lsa64_cache/extracted/all"

# Etapa 2: entrenamiento del modelo que se sirve (segundos)
python ml/train_lsa64.py --labels-map ml/lsa64_labels.json

# Validación leave-one-subject-out (~55 minutos)
python ml/evaluate_loso.py
```

El entrenamiento guarda el modelo en `backend/models/modelo_lsa64_lstm.pt` y su resumen en el archivo `.json` del mismo nombre. La validación guarda su reporte en `ml/reports/loso_lsa64.json`.

C.6 Pruebas automatizadas

```
pip install -r requirements-dev.txt
pytest tests/ ml/tests/ -v
```

C.7 Regeneración de las figuras y de este documento

```
python docs/tesina/figuras/generar_figuras.py
cd docs/tesina/build
npm install
node build.js
powershell -ExecutionPolicy Bypass -File postproceso.ps1
```

El último paso requiere Microsoft Word: actualiza los índices y exporta el documento a PDF.

Anexo D — Cronograma

La Figura 19 presenta el cronograma del proyecto. Las barras grises corresponden a la etapa de anteproyecto y relevamiento; las azules, al desarrollo de la aplicación; las naranjas, al desarrollo y la evaluación del modelo; y las rojas, a las tareas que restan hasta la entrega. La línea punteada marca la fecha de esta versión del documento.

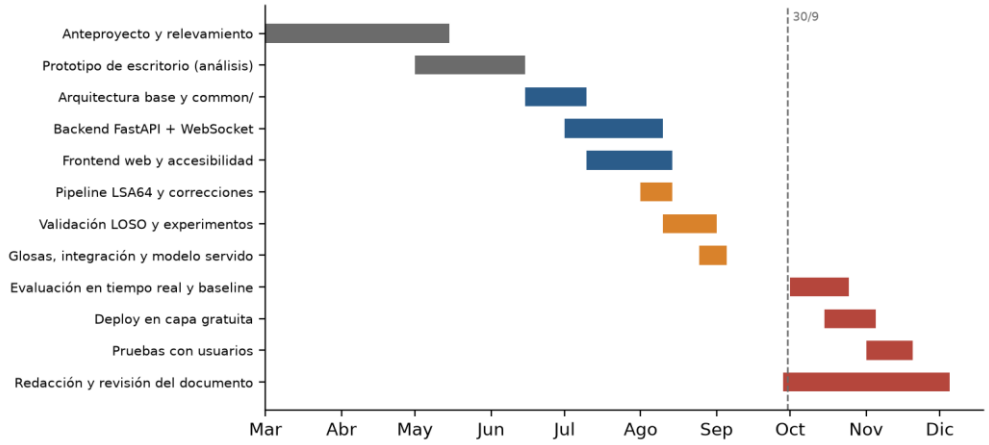


Figura *19***.** *Diagrama de Gantt del proyecto.*

PENDIENTE: las fechas anteriores a julio de 2026 son aproximadas. Ajustarlas con las fechas reales del anteproyecto, y actualizar las tareas restantes a medida que se completen. El script docs/tesina/figuras/generar_figuras.py (función fig_gantt) contiene la lista de tareas y fechas.

Tabla *20***.** *Etapas del proyecto y su estado.*

Etapa	Período	Estado
Anteproyecto y relevamiento de antecedentes	Marzo a mayo de 2026	Completada
Análisis del prototipo de escritorio	Mayo a junio de 2026	Completada
Arquitectura base y módulo compartido	Junio a julio de 2026	Completada
Backend FastAPI con WebSocket	Julio a agosto de 2026	Completada
Cliente web y accesibilidad	Julio a agosto de 2026	Completada
Pipeline de LSA64 y correcciones metodológicas	Agosto de 2026	Completada
Validación LOSO y experimentos	Agosto de 2026	Completada
Glosas en castellano e integración del modelo	Fines de agosto de 2026	Completada
Evaluación en tiempo real y línea base	Octubre de 2026	Pendiente

Etapa	Período	Estado
Despliegue en capa gratuita	Octubre a noviembre de 2026	Pendiente
Pruebas con usuarios	Noviembre de 2026	Pendiente
Redacción y revisión del documento	Octubre a diciembre de 2026	En curso

Bibliografía

Akdag, A., y Baykan, O. K. (2024). Enhancing signer-independent recognition of isolated sign language through advanced deep learning techniques and feature fusion. *Electronics, 13*(7), 1188. https://doi.org/10.3390/electronics13071188

Beck, K., Beedle, M., van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, M., Grenning, J., Highsmith, J., Hunt, A., Jeffries, R., Kern, J., Marick, B., Martin, R. C., Mellor, S., Schwaber, K., Sutherland, J., y Thomas, D. (2001). *Manifiesto por el desarrollo ágil de software*. https://agilemanifesto.org/iso/es/manifesto.html

Bengio, Y., Simard, P., y Frasconi, P. (1994). Learning long-term dependencies with gradient descent is difficult. *IEEE Transactions on Neural Networks, 5*(2), 157–166. https://doi.org/10.1109/72.279181

Breiman, L. (2001). Random forests. *Machine Learning, 45*(1), 5–32. https://doi.org/10.1023/A:1010933404324

Camgöz, N. C., Hadfield, S., Koller, O., Ney, H., y Bowden, R. (2018). Neural sign language translation. En *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (pp. 7784–7793).

Chacon, S., y Straub, B. (2014). *Pro Git* (2.ª ed.). Apress. https://git-scm.com/book/es/v2

Docker Inc. (s.f.). *Docker documentation.* Recuperado el 30 de septiembre de 2026, de https://docs.docker.com

Federación Mundial de Sordos. (s.f.). *Our work*. Recuperado el 30 de septiembre de 2026, de https://wfdeaf.org

Fette, I., y Melnikov, A. (2011). *The WebSocket Protocol* (RFC 6455). Internet Engineering Task Force. https://doi.org/10.17487/RFC6455

Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* [Tesis doctoral, University of California, Irvine].

Géron, A. (2022). *Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow* (3.ª ed.). O'Reilly Media.

Goodfellow, I., Bengio, Y., y Courville, A. (2016). *Deep learning*. MIT Press. https://www.deeplearningbook.org

Hastie, T., Tibshirani, R., y Friedman, J. (2009). *The elements of statistical learning: Data mining, inference, and prediction* (2.ª ed.). Springer.

Hochreiter, S., y Schmidhuber, J. (1997). Long short-term memory. *Neural Computation, 9*(8), 1735–1780. https://doi.org/10.1162/neco.1997.9.8.1735

Kingma, D. P., y Ba, J. (2015). Adam: A method for stochastic optimization. En *3rd International Conference on Learning Representations (ICLR)*. https://arxiv.org/abs/1412.6980

Kohavi, R. (1995). A study of cross-validation and bootstrap for accuracy estimation and model selection. En *Proceedings of the 14th International Joint Conference on Artificial Intelligence (IJCAI)* (Vol. 2, pp. 1137–1143).

Koller, O. (2020). *Quantitative survey of the state of the art in sign language recognition* (arXiv:2008.09918). arXiv. https://arxiv.org/abs/2008.09918

Ley 26.378 de 2008. Aprueba la Convención sobre los Derechos de las Personas con Discapacidad y su protocolo facultativo. 6 de junio de 2008. Boletín Oficial de la República Argentina.

Ley 27.710 de 2023. Lengua de Señas Argentina. 3 de mayo de 2023. Boletín Oficial de la República Argentina, N.º 35.162. https://www.argentina.gob.ar/normativa/nacional/ley-27710-383041/texto

Li, D., Rodríguez, C., Yu, X., y Li, H. (2020). Word-level deep sign language recognition from video: A new large-scale dataset and methods comparison. En *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)* (pp. 1459–1469).

Loshchilov, I., y Hutter, F. (2019). Decoupled weight decay regularization. En *7th International Conference on Learning Representations (ICLR)*. https://arxiv.org/abs/1711.05101

Lugaresi, C., Tang, J., Nash, H., McClanahan, C., Uboweja, E., Hays, M., Zhang, F., Chang, C.-L., Yong, M. G., Lee, J., Chang, W.-T., Hua, W., Georg, M., y Grundmann, M. (2019). *MediaPipe: A framework for building perception pipelines* (arXiv:1906.08172). arXiv. https://arxiv.org/abs/1906.08172

Marais, M., Brown, D., Connan, J., Boby, A., y Kuhlane, L. L. (2022). Investigating signer-independent sign language recognition on the LSA64 dataset. En *Southern Africa Telecommunication Networks and Applications Conference (SATNAC 2022)*.

Massone, M. I., y Machado, E. M. (1994). *Lengua de Señas Argentina: Análisis y vocabulario bilingüe*. Edicial.

Merkel, D. (2014). Docker: Lightweight Linux containers for consistent development and deployment. *Linux Journal, 2014*(239), Artículo 2.

Mindlin, I. (2021). *Reconocimiento de lengua de señas con redes neuronales recurrentes* [Tesina de licenciatura, Universidad Nacional de La Plata]. SEDICI. http://sedici.unlp.edu.ar/handle/10915/129853

Mozilla. (s.f.). *MediaDevices: getUserMedia() method*. MDN Web Docs. Recuperado el 30 de septiembre de 2026, de https://developer.mozilla.org/en-US/docs/Web/API/MediaDevices/getUserMedia

Naciones Unidas. (2006). *Convención sobre los derechos de las personas con discapacidad*. https://www.un.org/esa/socdev/enable/documents/tccconvs.pdf

Organización Mundial de la Salud. (2021). *World report on hearing*. https://www.who.int/publications/i/item/9789240020481

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., … Chintala, S. (2019). PyTorch: An imperative style, high-performance deep learning library. En *Advances in Neural Information Processing Systems 32* (pp. 8024–8035).

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., y Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830.

Prechelt, L. (1998). Early stopping — but when? En G. B. Orr y K.-R. Müller (Eds.), *Neural networks: Tricks of the trade* (pp. 55–69). Springer. https://doi.org/10.1007/3-540-49430-8_3

Ramírez, S. (s.f.). *FastAPI*. Recuperado el 30 de septiembre de 2026, de https://fastapi.tiangolo.com

Ronchetti, F., Quiroga, F., Estrebou, C., y Lanzarini, L. (2016a). Handshape recognition for Argentinian Sign Language using ProbSom. *Journal of Computer Science and Technology, 16*(1), 1–5. **PENDIENTE: verificar los datos de esta referencia (LSA16) contra la publicación.**

Ronchetti, F., Quiroga, F., Estrebou, C., Lanzarini, L., y Rosete, A. (2016b). LSA64: A dataset of Argentinian Sign Language. En *XXII Congreso Argentino de Ciencias de la Computación (CACIC 2016)*. https://facundoq.github.io/datasets/lsa64/

Ronchetti, F., Quiroga, F. M., Estrebou, C., Lanzarini, L., y Rosete, A. (2023). *LSA64: An Argentinian Sign Language dataset* (arXiv:2310.17429). arXiv. https://arxiv.org/abs/2310.17429

Rumelhart, D. E., Hinton, G. E., y Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature, 323*(6088), 533–536. https://doi.org/10.1038/323533a0

Russell, S., y Norvig, P. (2021). *Artificial intelligence: A modern approach* (4.ª ed.). Pearson.

Schuster, M., y Paliwal, K. K. (1997). Bidirectional recurrent neural networks. *IEEE Transactions on Signal Processing, 45*(11), 2673–2681. https://doi.org/10.1109/78.650093

Schwaber, K., y Sutherland, J. (2020). *La Guía de Scrum*. https://scrumguides.org

Sincan, O. M., y Keles, H. Y. (2020). AUTSL: A large scale multi-modal Turkish Sign Language dataset and baseline methods. *IEEE Access, 8*, 181340–181355. https://doi.org/10.1109/ACCESS.2020.3028072

Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., y Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research, 15*(56), 1929–1958.

Stokoe, W. C. (1960). *Sign language structure: An outline of the visual communication systems of the American deaf* (Studies in Linguistics, Occasional Papers 8). University of Buffalo.

Tran, D., Wang, H., Torresani, L., Ray, J., LeCun, Y., y Paluri, M. (2018). A closer look at spatiotemporal convolutions for action recognition. En *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (pp. 6450–6459).

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., y Polosukhin, I. (2017). Attention is all you need. En *Advances in Neural Information Processing Systems 30* (pp. 5998–6008).

World Wide Web Consortium. (2023). *Web Content Accessibility Guidelines (WCAG) 2.2*. W3C Recommendation. https://www.w3.org/TR/WCAG22/

Zhang, F., Bazarevsky, V., Vakunov, A., Tkachenka, A., Sung, G., Chang, C.-L., y Grundmann, M. (2020). *MediaPipe Hands: On-device real-time hand tracking* (arXiv:2006.10214). arXiv. https://arxiv.org/abs/2006.10214

Glosario

Los términos de este glosario aparecen en **negrita** en el lugar del cuerpo del documento donde se definen, indicado entre paréntesis.

Tabla *21***.** *Glosario de términos técnicos.*

Término	Definición
Alfabeto manual	Conjunto de configuraciones de la mano que representan las letras del alfabeto, usado para deletrear (sección 1.3.3).
Aprendizaje automático	Rama de la inteligencia artificial que estudia algoritmos que mejoran su desempeño a partir de datos (sección 2.2).
Aprendizaje profundo	Aprendizaje automático con redes neuronales de muchas capas, que aprenden sus propias representaciones (sección 3.2).
Aprendizaje supervisado	Aprendizaje a partir de ejemplos acompañados de la respuesta correcta (sección 2.2).
Aumentación de datos	Generación de variantes artificiales de los ejemplos de entrenamiento mediante transformaciones que no cambian su clase (sección 3.4).
BiLSTM	Red LSTM bidireccional: dos LSTM que recorren la secuencia en sentidos opuestos (sección 3.7).
Características	Valores numéricos que describen una entrada para un modelo de aprendizaje automático; en inglés, *features* (sección 2.2.1).
Clasificación	Tarea de asignar cada entrada a una de un conjunto finito de clases (sección 2.2.1).
Comunidad sorda	Personas que comparten la lengua de señas como lengua principal, con identidad y cultura propias (sección 1.1).
Dropout	Técnica de regularización que desactiva neuronas al azar durante el entrenamiento (sección 3.4).
Época	Pasada completa por todo el conjunto de entrenamiento (sección 3.3.2).
Etiqueta	Respuesta correcta asociada a un ejemplo de entrenamiento (sección 2.2).
Exactitud	Proporción de ejemplos clasificados correctamente; en inglés, *accuracy* (sección 2.5.1).
Fuga de datos	Filtración de información del conjunto de prueba al entrenamiento, que produce estimaciones optimistas (sección 2.4.2).
Inteligencia artificial	Área de la informática que desarrolla sistemas capaces de realizar tareas que requieren inteligencia (sección 2.1).
Landmarks	Puntos de referencia del cuerpo o de la mano estimados en una imagen; en este trabajo, 21 por mano (sección 4.2.2).
Leave-one-subject-out	Validación en la que cada persona queda, por turno, como conjunto de prueba de un modelo entrenado sin ella (sección 2.4.3).
Lengua de Señas Argentina	Lengua natural visoespacial de la comunidad sorda argentina, reconocida por la Ley 27.710 (sección 1.3).

Término	Definición
LSA64	Conjunto de datos público de 3200 videos de 64 señas de la LSA (sección 5.2.1).
LSTM	Red recurrente con celda de memoria y compuertas que permite aprender dependencias de largo plazo (sección 3.6).
Matriz de confusión	Tabla que cruza las clases reales con las predichas y muestra cada tipo de error (sección 2.5.3).
MediaPipe	Marco de trabajo de Google para procesos de percepción; su solución de manos estima 21 puntos por mano (sección 4.3).
Red neuronal artificial	Modelo compuesto por unidades simples interconectadas cuyos parámetros se ajustan a partir de datos (sección 3.1).
Redes neuronales recurrentes	Redes que procesan secuencias manteniendo un estado que resume lo visto (sección 3.5).
Regularización	Conjunto de técnicas que limitan el sobreajuste (sección 3.4).
Sobreajuste	Situación en que un modelo aprende particularidades del entrenamiento que no generalizan (sección 2.3).
WebSocket	Protocolo de comunicación persistente y bidireccional entre navegador y servidor (sección 6.3).